Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

# Master's Thesis Nr. 647

Systems Group, Department of Computer Science, ETH Zurich

Benchmarking LLM-Driven Iterative Optimisation of Deployed Backend Services

by

David Scherrer

Supervised by

Prof. Dr. Ana Klimović
Pinghe Li

March 2026 – September 2026

D INFK

# Abstract

BaxBench tests LLM-generated backend applications for functional and security correctness in an isolated container [50]. These checks do not show how an application performs as a replicated service under concurrent load. On Kubernetes, code and deployment configuration jointly determine the load a service can sustain. This thesis asks how much an LLM agent can improve sustained goodput under fixed resource and iteration budgets by refining either artifact.

We present IterBench, a framework for benchmarking and refining Kubernetes deployments whose agent revises either application code or a constrained deployment specification once per iteration. It validates, deploys, and benchmarks each candidate, then returns load-test results and cluster-resource diagnostics to the continuing agent conversation. Its adaptive load profile uses rolling latency, failure rate, and goodput to detect overload, attempt recovery, and estimate the highest stable goodput level meeting its failure-rate criterion. A run that cannot recover may still record a positive transient peak during exploration.

We evaluate Claude Opus 4.8, GPT-5.5, and GLM-5.2 across seven database-backed scenarios and three technology stacks: 63 tasks, each a single trajectory of ten refinement iterations. Twenty-eight of 30 baselines without an estimate later produce one. For tasks with an estimate, the geometric-mean best-over-first ratio is 2.7×. Code refinement costs two to three times as much and fails before benchmarking more often, mostly in functional tests or builds. Among completed steps with positive estimates before and after, selected code refinements include the largest gains and a higher geometric-mean change for every model. Deployment refinement reaches benchmarking more reliably, but loses existing estimates more often. Descriptively, GPT-5.5 reaches the highest best goodput, while GLM-5.2 slightly exceeds Claude Opus 4.8 at roughly one fifth of its LLM spend; Go-net-http ranks highest for every model in this grid. Thus, within this grid, the loop finds a better deployed candidate in 58 of the 61 tasks that establish an estimate (95%), but code correctness remains the main recorded obstacle, and the loop does not always finish with the best candidate it found.

# Acknowledgements

I would like to thank Pinghe Li for supervising this thesis. His guidance and our weekly discussions shaped the work from its early iterations through to the final benchmark design. His advice on design decisions, together with his eye for the bigger picture, repeatedly helped me identify questions worth pursuing and sharpen my focus on what mattered most.

I am grateful to Prof. Dr. Ana Klimović for proposing this project and for the opportunity to carry it out in the EASL lab at the Systems Group.

I would also like to thank Max Hürlimann for the collaboration during the early phase of the project, and for the brainstorming and the exchange of ideas and findings that came with it.

Finally, the experiments reported here were run on the Emulab testbed, whose dedicated cluster allocation made this evaluation possible.

# Contents

# List of Figures

# List of Tables

# Chapter 1

# Introduction

## 1.1 From Generated Code to Deployed Services

Large language models have advanced from completing individual functions [8] to resolving repository-level software engineering issues [12] and generating complete backend applications. BaxBench [50] evaluates that capability at the application level: given a scenario and API contract, a model generates an application that is exercised by functional and security test suites (Section 2.1). Passing those suites is evidence for the behaviours they cover, not proof of correctness beyond them. This thesis reuses the task format and functional gate; it does not repeat the security suite as a second Kubernetes evaluation.

The missing dimension is deployed performance. BaxBench evaluates each candidate in an isolated container, without a sustained concurrent load test of a replicated service. That is appropriate for contract testing, but it does not establish whether the candidate remains healthy when requests overlap and its components share a finite resource budget. Passing a functional suite and sustaining concurrent load are different properties.

This thesis therefore moves evaluation from an isolated container to a multi-node Kubernetes cluster. Every candidate admitted by the functional gate is deployed and scored on its *sustained goodput*: the rate of successful requests it holds under the benchmark's failure-rate and stability conditions, rather than a transient peak reached during a load ramp (Section 2.4).

## 1.2 Two Coupled Levers: Code and Deployment

Backend applications are commonly deployed through container-orchestration platforms such as Kubernetes [7, 17], where application replicas and data-tier components share finite CPU, memory, and network capacity across worker nodes (Section 2.2). For a fixed workload and cluster, two important determinants of performance are also the two artifacts an optimisation

agent can change in this thesis: the application code and the *deployment configuration*. The latter controls replica counts, resource requests and limits, pod placement, connection pooling, and database topology (Section 2.3).

The two levers are coupled, and their symptoms are easy to confuse. An exhausted connection pool, a query without a usable index, too few replicas, and an unfortunate placement decision can all present similarly under load: latency rises and successful-request rate stops increasing. Source inspection can identify some such defects, but it cannot reliably attribute every runtime shortfall without observing the deployed service under pressure.

Existing work generally fixes one lever: LLM-driven performance optimisation changes code within a fixed execution setting [45, 10]; infrastructure agents revise configuration but score deployability rather than served capacity [55, 11]; and learned resource managers tune configuration around a fixed application [41, 40, 28] (Sections 3.3 and 3.4). None lets one agent choose between code and deployment under the same runtime objective. This thesis exposes both levers, one per refinement step, with the deployment side validated against a fixed cluster budget before it is applied.

## 1.3 From Test Feedback to Runtime Measurement

The two gaps above presuppose useful feedback. Code-generation benchmarks and iterative coding agents commonly rely on test, compiler, or self-critique signals [44, 27, 54, 53]. These support correctness, but do not describe behaviour as offered load rises or identify the limiting resource. Performance optimisation instead needs a runtime measurement together with diagnostics that help explain it.

Agentic systems have begun to close measurement-driven loops for generated system implementations in fixed deployment settings [13, 24] (Section 3.4). A deployed backend adds a boundary: its performance is a joint property of code and the configuration that places, replicates, and connects it. A measured shortfall must therefore be attributed to an artifact class before the agent can act. To our knowledge, the systems reviewed in Chapter 3 do not combine a generated backend, a separately controlled deployment specification, and iterative load-test feedback on a multi-node Kubernetes cluster. This leaves the central question:

> *To what extent can an LLM agent, given runtime feedback, improve the sustained goodput of a generated backend service on a Kubernetes cluster, under a fixed resource and iteration budget, by refining either application code or deployment configuration?*

## 1.4 This Work: IterBench

We present IterBench, a Kubernetes-based iterative benchmarking framework that extends BaxBench to study that question. Here, *agent* denotes one continuing model conversation per experiment: the model interprets the latest outcome, chooses a refinement lever when routing permits, and proposes a revised artifact. The framework, rather than the model, owns experiment state, validation, deployment, measurement, and failure routing. Given a scenario and target cluster, IterBench runs two phases:

1. A **baseline phase** in which the agent generates application code and an initial deployment specification, which are validated, deployed, and benchmarked under adaptive load.

2. An **iterative refinement phase** in which the agent receives the preceding deployment's goodput, endpoint-level load-test statistics, latency distributions, and resource-utilisation diagnostics, and revises either the application code *or* the deployment specification before the deploy-and-benchmark cycle is repeated.

Only one artifact is regenerated in a refinement iteration; the latest validated version of the other is reused. This supports step-level attribution to the *lever class*, although a code revision can change many lines and a specification revision can change several fields. The deployment artifact is a constrained YAML schema (`spec.yaml`) covering replication, resource allocation, placement, connection handling, and data-tier configuration. IterBench renders manifests deterministically, validates requested capacity against the target cluster, and manages resource lifecycle (Chapter 4).

Every completed deployment is measured by the same adaptive Explore-and-Refine load profile. It searches across the capacity range without per-application request-rate tuning and returns an operational sustained-goodput estimate under explicit failure-rate and stability conditions (Section 4.5.5). Before the research questions use that estimate, Section 6.1 characterises the instrument's repeatability, outcome agreement, phase behaviour, and wall-clock cost. This is evidence about the instrument's operational robustness, not a comparison against an independent ground-truth capacity measurement.

Sustained goodput is one objective among several an operator might choose; latency against a service-level target, resource efficiency, or monetary cost per unit of served capacity are also defensible. It is used here because it is a single scalar that remains comparable across the observed capacity range and responds to the principal runtime failure modes this loop is designed to expose. Infrastructure cost is held fixed rather than included in the objective. LLM expenditure is logged separately so that achieved goodput can be read alongside the one-time model cost of producing a trajectory.

The empirical scope is a fixed benchmark grid, not a representative sample of production services. The evaluation crosses three models, seven scenarios,

and three language/framework stacks, for 63 tasks with one generation draw and ten refinement iterations per task. Three scenarios come from BaxBench; four are produced by the adapted scenario-construction pipeline, and all seven require a database. The workloads are synthetic and contract-derived, experiments use one dedicated cluster topology, and no multi-tenant background workload is introduced. Scenario provenance also overlaps with model evaluation: GPT-5.5 authored parts of the four generated scenarios, while GLM-5.2 supplied one of their calibration implementations (Section 5.2). Results therefore provide evidence about average performance over this fixed, balanced evaluation grid. Because each task is run once and the grid is purposively selected rather than randomly sampled, the study does not estimate within-task generation variance or support population-level claims about all models, frameworks, or production services.

## 1.5 Research Questions

The evaluation separates the loop's primary outcome from the mechanisms and failure modes that help explain it. RQ1 and RQ2 address the core optimisation behaviour; RQ3 is a diagnostic analysis of the same fixed dataset. Descriptive comparisons across technology stacks and scenarios, together with the cost analysis, are reported as additional results rather than research questions:

**RQ1** What measured sustained-goodput improvement does iterative refinement discover within the fixed budget, how often does it recover an unsustained baseline, and how do the trajectories differ across models (Section 6.2)?

**RQ2** How do code and deployment refinement differ in pre-benchmark failure, recovery or collapse of a sustained estimate, and observed goodput change when both adjacent measurements are positive (Section 6.3)?

**RQ3** Which harness-recorded pipeline failures are most prominent, and how does their frequency vary across stages, models, and tasks (Section 6.4)?

## 1.6 Contributions

This thesis makes the following contributions:

1. **An iterative benchmark framework** (IterBench) that extends BaxBench with Kubernetes deployment, adaptive load testing, and a stateful baseline-and-refinement loop in which the agent chooses between code and deployment refinement. Deterministic failure routing (Section 4.4.2) preserves artifact lineage and records the stage at which an attempted iteration stopped.

2. **A constrained deployment specification** (`spec.yaml`) that exposes a structured search space covering replica count, resource requests and limits, database topology, connection handling, and pod placement, while the framework enforces cluster-capacity validation and deterministic manifest rendering.

3. **An adaptive load profile** (*Explore–Refine*) that produces an operational sustained-goodput estimate without per-application request-rate tuning, together with an empirical characterisation of its repeatability and phase outcomes (Sections 4.5.5 and 6.1).

4. **A descriptive experimental evaluation** across three current LLMs, seven database-backed scenarios, and three language/framework stacks, covering all 63 tasks and tracking LLM expenditure alongside goodput. The evaluation reports both aggregate trajectories and stage-level failure and refinement outcomes rather than reducing each task to a final pass/fail result.

The scenario-authoring and functional-test-calibration stages build on AutoBaxBuilder [51]; their inherited logic is not presented as a standalone contribution of this thesis. The framework, the adapted construction pipeline, the analysis code, and the aggregate result tables from which every figure and number in Chapter 6 is derived are available at `https://github.com/djscherrer/IterBench`. The complete per-iteration artifact tree, roughly 16 GB of generated source, rendered manifests, load-test output, and diagnostics, is retained and available on request rather than distributed with the repository.

## 1.7 Results at a Glance

The 63 tasks contain 693 planned candidate iterations, of which 632 reach the load-test stage and 61 stop earlier at a recorded pipeline stage. Each task is run once ($n = 1$). Re-measurement agrees on the estimate/no-estimate outcome for 90.4% of 625 matched candidates; among candidates positive in both runs, the median symmetric difference is 2.9% (Section 6.1).

Within the fixed budget, the loop discovers a higher measured candidate in 58 of the 61 tasks that ever record a positive sustained estimate, with a geometric-mean best-over-first ratio of 2.7×. It also finds a sustained candidate after 28 of 30 unsustained baselines. These are best-of-trajectory, not guaranteed final, improvements: six tasks that begin healthy improve and still end without a sustained estimate.

GPT-5.5 reaches the highest aggregate best goodput. GLM-5.2 slightly exceeds Claude Opus 4.8's aggregate at roughly one fifth of the mean LLM expenditure per task. Code refinement fails before benchmarking more often and costs two to three times as much per step, but contains the largest gains

and more recoveries; deployment refinement fails earlier less often, yet crosses from a positive estimate to no sustained estimate proportionally more often. These are grid-level point summaries over 21 matched task cells per model, not population-level rankings or causal lever effects.

Go-net-http has the highest geometric-mean best observed goodput for every model in this grid, an association with the evaluated stack rather than an inherent framework ceiling. Recorded failures are dominated by code-stage functional-test and build failures, although the taxonomy does not classify a completed benchmark reporting no sustained estimate as a pipeline failure. Within the tested scope, the loop therefore discovers substantially better deployed candidates, while code correctness remains the main obstacle to a tunable state.

## 1.8 Thesis Outline

Chapter 2 introduces BaxBench, Kubernetes, deployment levers, performance vocabulary, and feedback-driven iterative refinement. Chapter 3 positions this thesis against code-generation benchmarks, iterative refinement, verification, infrastructure management, and LLM-driven performance optimisation. Chapter 4 describes scenario and workload construction, the benchmark architecture and iterative protocol, the deployment specification, and the adaptive load profile. Chapter 5 fixes the evaluated models, scenarios, cluster, and budgets. Chapter 6 characterises the measurement instrument and answers the three research questions, before Chapter 7 interprets the findings, limitations, and practical implications. Chapter 8 answers the overarching question, consolidates the scope limitations, and outlines future work.

# Chapter 2

# Background

Chapter 1 stated the problem this thesis addresses and the gaps it closes. This chapter supplies the technical vocabulary the remainder assumes. It is selective rather than encyclopaedic: each concept appears because it is used to define the benchmark, to interpret a result, or to explain a design choice.

Section 2.1 introduces BaxBench, the code-generation benchmark this thesis extends. Section 2.2 introduces Kubernetes and the deployment setting, and Section 2.3 the deployment decisions exposed to the agent. Section 2.4 defines the load-testing vocabulary and the goodput objective, and Section 2.5 the feedback-driven improvement loop. Chapter 3 instead compares this work with prior systems.

## 2.1 BaxBench Task Model and Correctness Gate

BaxBench [50] is the benchmark this thesis builds on. It defines 392 tasks across 28 *scenarios*. A scenario describes a fixed backend workload through an API contract; a recipe-sharing service with upload, comment, and rating endpoints is one example. Each scenario is paired with 14 *frameworks*, combinations of a programming language and a web or application framework spanning six languages, so the same scenario can be implemented and compared across technology stacks.

For a scenario–framework pair, a model generates a complete application. BaxBench then applies two separate forms of checking. A *functional test suite* exercises the API contract end to end and checks that responses behave as the contract requires. A separate *security test suite* probes known vulnerability classes, such as SQL injection or broken authentication. The distinction matters for this thesis: the functional suite provides the correctness gate applied before performance measurement, whereas security testing is part of the original benchmark and is not reconstructed as a second Kubernetes evaluation here.

The evaluation combines two sources of inputs. ClickCount, Recipes,

and PETSTORE are pre-existing, expert-designed BAXBENCH scenarios that retain the workloads shipped with that benchmark. The other four scenarios and their load-test workloads are constructed offline by an LLM-driven pipeline that builds on AUTOBAXBUILDER [51]. That pipeline, the adaptations made to it, and the load-test generation stage added to it are described in Section 4.2.

## 2.2 Kubernetes and Container Orchestration

Moving from one container on one host to a deployed service introduces replicated application and data-tier components, finite resources shared among them, and overlapping requests from a load generator. Kubernetes supplies the orchestration layer for that setting and deploys every candidate in this thesis. It follows a desired-state model: an operator declares what should be running, and controllers continuously reconcile actual cluster state towards that declaration [7, 20].

Several Kubernetes primitives recur throughout the thesis. A *Pod* is the smallest deployable unit, containing one or more containers scheduled together on a worker. A *Deployment* maintains a declared set of Pod replicas. A *Service* provides a stable network identity and sends traffic to selected endpoints that Kubernetes considers ready. A *Namespace* gives namespaced objects a separate administrative scope; the benchmark creates one disposable Namespace per candidate, but the Namespace alone is not a security or network isolation boundary [21, 15, 23, 19].

Pods declare CPU and memory *requests* and *limits*. The scheduler places Pods by comparing their requests, rather than current usage, with each worker's allocatable capacity; CPU requests also influence relative CPU share under contention. Limits instead constrain runtime use: a low CPU limit can throttle execution, while memory limits are enforced reactively and can trigger an out-of-memory kill. A Pod whose requests fit no permitted worker remains unschedulable. Placement therefore affects both which components compete on a machine and which network paths they use [22].

Running is not the same as serving. Kubernetes can use *liveness* probes to decide when to restart a container and *readiness* probes to decide whether a Pod should receive Service traffic [18]. A container that repeatedly exits is eventually subjected to restart backoff, commonly surfaced as `CrashLoopBackOff`; one that runs but never becomes ready remains outside the Service's ready endpoints. Both outcomes occur here as deploy-stage failures (Section 6.4), and neither is visible to a validator that checks only whether a requested layout fits the cluster.

## 2.3 Deployment Configuration as a Performance Lever

Once an application is deployed, its performance depends on both the code and the configuration of the service around it. The agent in this thesis controls a bounded set of deployment decisions; the exact fields and validation rules are specified in Section 4.5.3. They can be understood as six related families.

**Application replicas.** Replica count determines how many instances run behind the backend Service. Too few replicas can leave requests queued behind busy instances. Additional replicas can increase parallelism until another resource becomes the bottleneck, but they also consume cluster capacity and multiply downstream load, especially database connections.

**Per-Pod resources.** CPU and memory requests affect whether and where Pods can be scheduled; limits constrain runtime consumption. Requests set too high reserve scheduling capacity that other useful replicas cannot claim; requests set too low can leave a Pod with a weak CPU share under contention. Limits set too low can throttle or terminate the application. The relevant choice is therefore the relationship between workload demand and the cluster's finite capacity, not one universally correct value [22].

**Runtime and database tuning.** The specification exposes allow-listed application settings such as connection pool size, worker-process count, and `GOMAXPROCS`, as well as PostgreSQL memory, parallelism, planner, and write-ahead-log settings. These are declarative levers, but an application setting has an effect only if its runtime or code reads it, and database settings can move rather than remove a bottleneck [39].

**Placement.** Placement specifies which workers a Pod may use and whether replicas should be spread or allowed to co-locate. Co-location can make a small deployment efficient, but it also makes replicas compete for the same CPU, memory, and network path. Spreading replicas can reduce that contention while narrowing the set of feasible assignments and potentially fragmenting per-node headroom. The best choice depends on the workload and on which components share the cluster.

**Data-tier topology.** The backend can use a standalone database or a primary with streaming read replicas, and the deployment may additionally enable an application-level cache. Read replicas are not a transparent capacity increase. The primary remains the only writable copy, and a replica raises throughput only if the application deliberately routes read-only queries to it, which is a property of the application code rather than of the deployment.

Streaming replication is asynchronous by default, so a read served by a replica may reflect stale state [38]. Provisioning replicas that the application never queries consumes capacity that the primary or application tier could have used.

**Connection handling.** PostgreSQL starts a server process for each client connection, and each database server enforces its own `max_connections` ceiling [37, 36]. The aggregate client demand from all backend replicas must fit within the primary's available connection slots, so scaling the application tier can exhaust the primary before it exhausts CPU. PgBouncer can multiplex many client connections onto fewer server connections. In *session* pooling, a server connection is assigned for the client session; in *transaction* pooling, it returns to the pool after each transaction, so applications cannot generally assume that session-scoped state persists. Recent PgBouncer versions can track protocol-level prepared statements when explicitly configured, but SQL-level prepared statements and several other session features remain incompatible with transaction pooling [34]. The configuration evaluated here does not enable prepared-statement tracking, and this thesis observes a driver incompatibility directly (Section 6.5).

These families can be changed through the deployment specification without rewriting source code, although their effect may depend on application behaviour. This makes deployment configuration a control surface distinct from code while explaining why both must be observed in the same running system.

## 2.4 Measuring Performance Under Load

A *load test* creates concurrent demand and records how a service responds as that demand changes. In this thesis, Locust runs simulated users that follow the scenario's verified request flows, grounding the load workload in the same API contract used by the functional tests. The workload defines what a user does; the load profile controls how many users are active [25, 26].

This is a *closed-loop* workload model: a simulated user starts its next task only after the previous task completes and its configured wait time elapses. The controller therefore sets user concurrency, not an independent request-arrival rate; achieved requests per second also depend on response time and on how many requests each task issues. Throughout this thesis, *offered load* consequently means concurrent-user pressure under this model, and the measured capacity is specific to it [42, 26].

Two measured quantities are central. *Throughput* is the rate of completed request attempts. *Latency* is the time from issuing a request to receiving its response, usually summarised with percentiles such as p50 or p95 because a mean can hide a long tail of requests waiting behind a saturated resource.

As concurrent-user load rises, some resource eventually becomes limiting: CPU, memory, network capacity, database connections, or storage. The region where additional users stop producing proportionally more completed work is referred to here as the deployment's *capacity knee*. Near it, queues can grow and latency can rise sharply; beyond it, throughput may plateau, decline, or collapse as requests slow, time out, or fail. The limiting resource is the deployment's bottleneck at that load level.

Raw throughput is not sufficient as an objective, because a system can complete many requests while failing a substantial fraction. In this thesis, **goodput** is the observed rate of requests that the Locust workload marks successful, excluding failed and timed-out requests. This differs from the SLO-attainment definition used in serving-system work, where goodput is the maximum arrival rate satisfying a target fraction of per-request latency objectives [56]. Here, failure rate and short-term stability constrain which load levels the profile accepts; latency bounds the search but is not applied as a final acceptance condition (Section 4.5.5). Goodput remains a runtime signal, not a substitute for the functional gate: tests decide whether a candidate is admitted to measurement, and goodput then measures that candidate under concurrent load.

*Sustained goodput* is the best stable level accepted in the load profile's *Refine* phase under those health conditions, rather than a transient peak seen during *Explore*. It is therefore an operational estimate under this controller and workload model. Section 4.5.5 gives the exact phases, thresholds, and aggregation procedure.

## 2.5 Feedback-Driven Iterative Refinement

Rather than treating generation as a single attempt, an iterative system produces a candidate, evaluates it, feeds observations back, and proposes a revision. In this thesis, the *agent* is the language model embedded in that stateful loop; its weights do not change, but the current artifacts and recorded outcomes become input to the next decision. This is an instance of the general monitor–analyse–plan–execute feedback pattern [14]; particular LLM systems are compared in Chapter 3.

The stopping rule depends on the objective. A correctness-gated loop can stop when a test suite passes. A performance-gated loop has no unique pass condition, because another candidate may improve the objective. It must instead stop when improvement stalls, a budget is exhausted, or an external criterion is reached. This thesis uses a fixed budget; its value is specified in Section 5.4 and its consequences are revisited in Section 7.4.

# Chapter 3

# Related Work

This chapter positions IterBench along four dimensions of prior work: backend code-generation benchmarks, feedback-guided generation and verification, deployment and configuration optimisation, and LLM-driven performance optimisation. Section 3.5 then identifies the combination of mutable artifacts, runtime feedback, and execution environment that distinguishes this thesis.

## 3.1 LLM Code Generation and Backend Evaluation Benchmarks

Evaluation of large language models for software engineering has moved from single-function synthesis [8, 6] to repository-level issue resolution [12] to complete, security-checked backend applications [50] (Section 2.1). CWEval [33] and SecRepoBench [43] extend the security dimension with outcome-driven test oracles and repository-scale, fuzz-tested code-agent evaluation, respectively. CodeFlowBench [52] extends the correctness dimension to *multi-turn* construction, where later functionality reuses earlier turns' output, and finds significant degradation as dependency complexity grows.

Together, these benchmarks characterise whether generated software is correct, secure, or constructible over multiple turns. The code-generation benchmarks reviewed here evaluate programs in isolated execution settings, without orchestrated deployment, shared cluster resources, or load testing (Section 2.2). None measures *sustained goodput* when an application is deployed as a replicated service on Kubernetes, which is the gap our work addresses.

AutoBaxBuilder [51] addresses a related but different bottleneck: the cost of constructing new benchmark scenarios, API contracts, functional tests, and security checks. Its contribution is an LLM-driven benchmark-construction pipeline rather than a runtime evaluation of generated services. This thesis reuses and adapts that offline construction process for four additional scenarios and their workloads, while retaining three pre-existing,

expert-designed BaxBench scenarios and their shipped workloads. The contribution here is the subsequent deployment-and-performance loop.

## 3.2 Feedback-Guided and Verification-Gated Generation

Section 2.5 introduces the general propose-evaluate-revise pattern our own method instantiates. Self-Refine uses an LLM's own critique [27]; Reflexion instead verbalises task feedback, which may be external or internally simulated, and stores the resulting reflections for later trials [44]. Specialised for software engineering, SWE-agent [54] and OpenHands [53] close the loop with compiler and test signals from a sandboxed shell.

A related but distinct line constrains or verifies model output rather than revising it from execution feedback. Synchromesh [35] uses constrained semantic decoding to enforce implemented syntax, scope, and typing rules. Clover [48] checks consistency among code, docstrings, and formal annotations, while VERGE [46] applies SMT-guided refinement to decomposed claims in natural-language reasoning. These methods provide stronger guarantees where their constraints apply, but their applicability to complete backend applications has not been established.

Our agent instead revises one of two artifact classes—application code or a constrained deployment specification—using quantitative signals from a multi-node Kubernetes deployment under load: sustained goodput, latency percentiles, and resource utilisation. The load test supplies the numeric performance objective, while BaxBench functional tests and schema validation gate candidate correctness and validity. This provides empirical correctness and runtime evidence rather than formal guarantees, while remaining applicable to complete deployed backends.

## 3.3 Deployment and Configuration Optimisation

The autonomic-computing control pattern introduced in Section 2.5 predates LLM agents [14]. In Kubernetes [7, 17] (Sections 2.2–2.3), the Horizontal Pod Autoscaler adjusts replica count from observed metrics within a fixed workload template [16], without revising database topology, connection pooling, pod placement, or application code.

Other systems search or learn configuration policy while holding the application fixed. OpenTuner [1] ensembles search techniques over a program's numeric parameter space. Autopilot automates horizontal and vertical scaling from historical utilisation [41]; FIRM localises resource contention and reprovisions affected microservices [40]; Decima learns cluster scheduling policy with deep reinforcement learning [28]; and OtterTune learns DBMS knob settings [49]. These systems do not revise both application code and a

deployment specification for an arbitrary HTTP service, and none uses an LLM to choose those revisions.

Closer to infrastructure as code (IaC), IaCGen performs inference-time repair using format, syntax, and live-deployment feedback, raising deployment success from 20.8–30.2% on the first attempt to 54.6–91.6% within ten; human guidance raises all evaluated models above 90% within 25 attempts [55]. TerraFormer instead combines supervised fine-tuning with verifier-guided reinforcement learning over syntax, deployability, and policy compliance [11]. Both advance LLM-generated IaC, a broader area surveyed by Srivatsa et al. [47], but neither optimises the provisioned service's runtime performance or its application code.

These systems establish useful infrastructure-management capabilities, but they leave the joint problem open: learned resource managers hold the application and specification fixed, while IaC-generation agents refine a specification for validity or compliance rather than measured service capacity. Our deploy probe serves that validation purpose only as a precondition: after a candidate is schedulable and ready, the loop measures sustained goodput and can refine either the deployment specification or application code.

## 3.4 LLM-Driven Systems and Performance Optimisation

Shypula et al. [45] curate PIE, over 77,000 human performance-improving edit pairs from competitive programming, and study prompting and fine-tuning strategies including performance-conditioned generation and self-play. Execution provides reliable scoring rather than an online measurement loop. SWE-Perf [10] moves performance optimisation to repository scale with 140 instances derived from real pull requests and evaluates direct, pipeline-based, and agent-based patch generation. Its performance tests score produced patches rather than explicitly driving a deploy-measure-revise loop.

Glia [9] combines LLM reasoning with measured infrastructure feedback: its multi-agent architecture designs request-routing, scheduling, and autoscaling mechanisms for a GPU cluster serving LLM inference. It generates new inference-serving algorithms rather than filling a constrained deployment specification for general backends.

VibeServe [13] combines an outer planner with an inner implementation, correctness, and performance loop. It approaches vLLM and SGLang performance for Llama-3.1-8B on an H100 and reports larger gains on non-standard model–workload–hardware combinations. Liu et al. [24]'s *Just-in-Time Systems* (Jitskit) similarly iterates planning, coding, and critic-guided reflection to synthesise key-value stores that outperform established systems on its tested configurations.

Our framework shares this iterative structure—propose, gate on correctness, benchmark, and feed diagnostics forward—but at a different level of the stack: PIE and SWE-Perf optimise *source code* in isolation; VibeServe and Jitskit synthesise *system implementations*; IterBench revises either *application code* or *Kubernetes deployment configuration*, one artifact per iteration, for BaxBench backends, using *sustained goodput* under an adaptive load profile as its runtime objective.

## 3.5 Positioning of This Work

Table 3.1 summarises how representative systems relate to our framework by role, feedback signal, environment, and mutable artifact. Code-generation benchmarks establish correctness, security, and multi-turn quality but not cluster performance. Refinement and verification methods improve reliability without a quantitative infrastructure objective. Configuration systems and IaC generation tune fixed applications or target validity and compliance, while Glia, VibeServe, and Jitskit optimise or synthesise specialised system implementations.

Table 3.1: Comparison of representative approaches.

| **Approach** | **Role and artifact** | **Signal use and objective** | **Environment** |
|---|---|---|---|
| BaxBench | Benchmark; application code | Scoring: functional and security tests | Docker |
| SWE-Perf | Benchmark; code patch | Scoring: performance tests | Repository-level tests |
| IaCGen | Agent; IaC template | Revision: deployment feedback | Multi-cloud |
| Glia | Agent; system algorithms | Revision: serving performance | GPU cluster |
| VibeServe | Agent; serving runtime | Revision: correctness and performance | Target hardware |
| IterBench | Agentic benchmark; code or deployment specification | Revision: sustained goodput | Multi-node Kubernetes |

The intersection left open by these lines is *iterative optimisation of deployed, LLM-generated backend services* on a multi-node Kubernetes cluster. The agent can revise either application code or a validated deployment specification, one artifact per iteration, and each successfully deployed candidate is scored by measured sustained goodput rather than by static tests, self-critique, deployability alone, or a fixed benchmark application. To our knowledge, no prior system combines all of these dimensions in one evaluation loop.

# Chapter 4

# Methods

## 4.1 Method Overview and Evaluation Objective

This chapter describes the methodology of IterBench, our iterative agentic benchmark setup. It defines the concrete evaluation machinery: the offline construction pipeline for additional scenarios and workloads, the cluster components on which candidates run, the iterative workflow that generates and refines them, and the measures by which we compare iterations.

**Research question.** We ask whether an LLM agent can, for a fixed BaxBench backend scenario and a Kubernetes cluster with **fixed capacity**, iteratively improve **sustained goodput** by refining either application code or deployment configuration. Each experiment encompasses a *scenario* (the application behaviour and API to implement), a *framework* (the language and library stack on which the application runs), and a *model configuration* (the LLM and generation settings used to propose application code and cluster specifications). Successive iterations produce candidates, each consisting of application code and a Kubernetes deployment specification. After validation and a successful deploy on the cluster, each candidate undergoes a performance test that measures its sustained goodput.

**What we optimise.** The conceptual distinction between throughput, goodput, and sustained goodput is defined in Section 2.4. Operationally, this thesis measures the objective with an adaptive Locust load profile that raises or lowers offered load using live failure-rate, latency, and goodput statistics until the system approaches saturation (Section 4.5.5). We compare iterations by the load profile's operational, short-horizon sustained-goodput estimate, rather than by raw request rate at an arbitrarily chosen load. Throughout the thesis, "sustained goodput" is shorthand for this controller-accepted estimate; it is not a long-duration endurance guarantee or a final score constrained by every latency objective used while searching for saturation.

**Road map.** Section 4.2 describes the offline construction pipeline and the verified workloads it produces. Section 4.3 describes the benchmark architecture and the roles of its components. Section 4.4 defines the benchmark state, experimental units, artifact lineage, failure handling, responsibility split, and agent context. Finally, Section 4.5 specifies the iteration stages and their contracts, while Section 4.6 explains the outcome measures and how the resulting trajectories are analysed. Appendix A summarises the terminology and visual cues used in the custom figures.

## 4.2 Offline Scenario and Workload Construction

To evaluate how well different language models implement backend systems, the fixed grid combines two input sources. ClickCount, Recipes, and Petstore are pre-existing, expert-designed BaxBench scenarios that reuse the workloads shipped with that benchmark. The other four scenarios and their corresponding load-test workloads are constructed offline, before the model-evaluation experiments, by the adapted pipeline described here. All seven scenario–workload pairs are then reused across every model and framework.

The construction process for the four additional scenarios builds on the generative approach of AutoBaxBuilder [51], introduced in Section 2.1. This thesis extends that setup with a load-test generation and verification phase. For each pipeline-authored scenario, it generates a Locust workload and verifies its structural validity, API coverage, and behaviour against reference implementations. We also adapt the inherited generation and repair loops by replacing isolated model calls with continuing conversations and structured feedback after an unsuccessful check. Each conversation retains the preceding artifacts, while the feedback identifies the failed check and includes a compact diagnostic excerpt, for example, a parsing or validation error, a failed functional test, or HTTP and container diagnostics. A repair turn can therefore revise the preceding artifact in light of the specific problem, rather than restart from the full task description.

The following subsections describe these adaptations and extensions. The remaining details of the inherited scenario and functional-test pipelines are described in the AutoBaxBuilder paper [51]. Reference implementations serve only as internal calibration and validation targets; they are never candidates in the later model-evaluation experiments.

### 4.2.1 Construction Pipeline

The pipeline generates a scenario idea, admits it only after a novelty review, and produces a validated API contract. It then constructs functional tests and calibrates them against three LLM-generated reference implementations. Finally, it authors a Locust script from the scenario context and API contract,

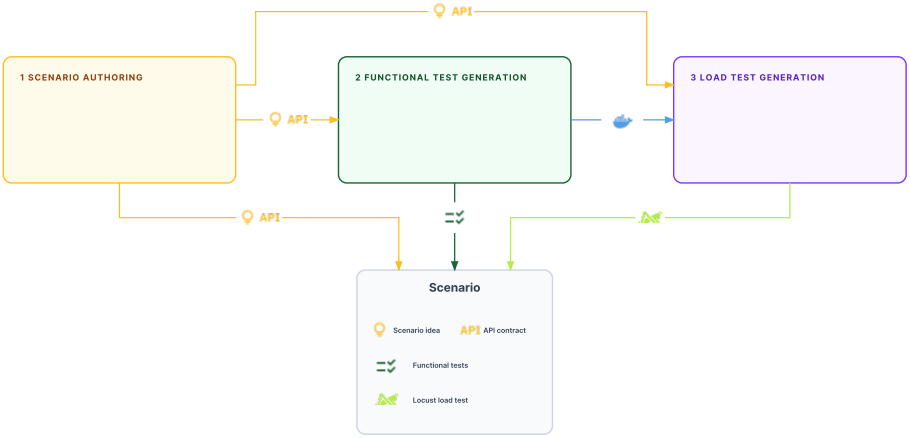



Figure 4.1: Three-stage offline construction pipeline. The arrows identify the artifacts passed between scenario authoring, functional-test generation, and load-test generation.

then verifies it against the calibrated reference implementations produced by the preceding functional-test stage. Security-test construction from the original AutoBaxBuilder setup is omitted from this thesis. The following subsections describe the adaptations to scenario and API-contract construction and functional-test generation, followed by the added load-test generation stage.

### 4.2.2 Scenario Authoring and API Specification

**Inherited workflow.** At a high level, this stage follows AutoBaxBuilder: an LLM proposes a scenario candidate, an independent novelty check compares it with the existing scenario set, and an accepted candidate is elaborated into an OpenAPI contract for downstream artifact generation [51]. Figure 4.2 shows this candidate-admission and API-contract workflow. The basic decomposition into candidate generation, novelty review, and API-contract construction is inherited; it is not a contribution of this thesis.

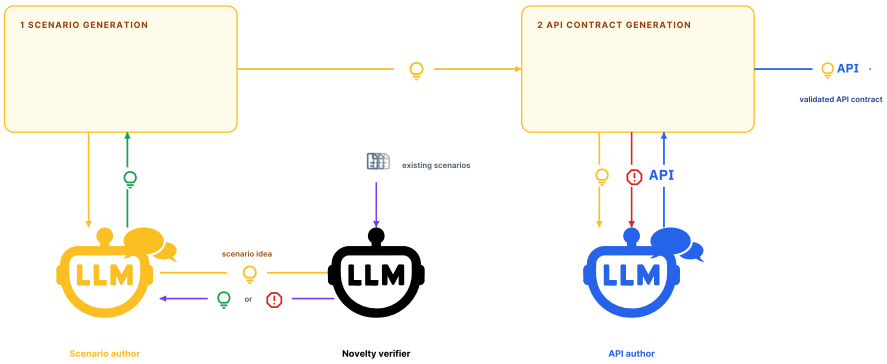



Figure 4.2: Scenario and API-contract construction: novelty admission followed by API authoring and validation.

**Adaptations in this thesis.** This thesis changes how the inherited workflow carries context and handles unsuccessful attempts. The scenario author and API-specification author each maintain a durable conversation. A repair therefore retains the initial prompt and previous candidates while receiving new feedback from the current failure record. For example, a duplicate or inconclusive novelty verdict returns the closest matches and the verifier's reason, so that the next scenario-author turn can avoid repeating the rejected candidate. The API author similarly receives the relevant parsing or validation error from the failure record, allowing the subsequent turn to address that error directly.

### 4.2.3 Functional-Test Generation and Calibration

**Inherited workflow.** This stage adopts AutoBaxBuilder's functional-test calibration workflow: it generates a functional-test suite and reference implementations, executes the suite against those implementations, and uses black-box calibration followed by white-box judgment to decide whether a failure requires a test or implementation repair [51]. Figure 4.3 shows the three inherited stages and their repair routes. In the first stage, the functional-test builder receives the scenario description and API contract and derives a suite of executable functional tests. A set of LLM reference-implementation builders, potentially configured with different models, generates the reference implementations. The harness then builds each implementation into a Docker image and executes the suite against it. The calibration cycle and the distinction between test-side and implementation-side failures are inherited from AutoBaxBuilder. They are not contributions of this thesis.

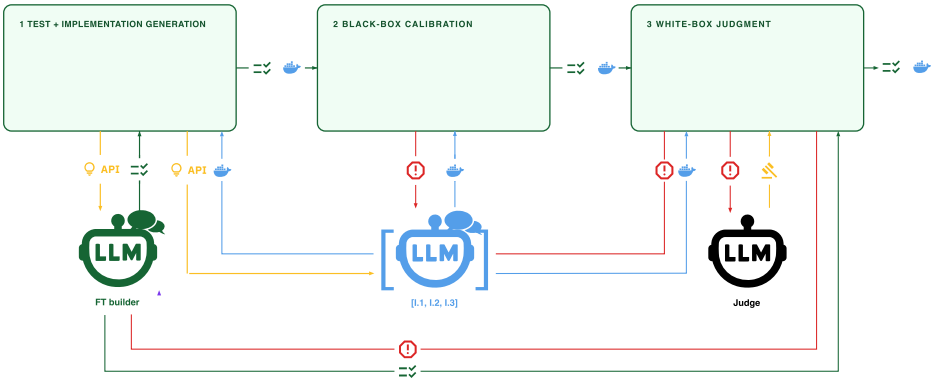



Figure 4.3: Functional-test generation and reference-implementation calibration across the three inherited stages.

**Adaptations in this thesis.** This thesis changes how the inherited repair loop preserves state and communicates evidence. One durable conversation is maintained for the functional-test builder (the leftmost agent in Figure 4.3) and one for each reference-implementation builder (the bracketed $[I.1, I.2, I.3]$ group in the same figure). When execution or judgment identifies a failure, the relevant evidence, such as test outcomes, harness and container diagnostics, and, for white-box decisions, the judge's attribution, is appended to the conversation of the responsible agent. The previously generated suite or implementation remains in that conversation history, so it does not need to be repeated in a refinement prompt. This preserves prior context while keeping each repair focused on newly observed evidence. The resulting calibrated reference implementations are then passed to load-test generation.

### 4.2.4 Load-Test Generation and Verification

For the four pipeline-authored scenarios, we extend the adapted AutoBaxBuilder pipeline with a load-test generation and verification stage. It reuses the reference implementations from the preceding functional-test stage as verification targets for the generated workload. The stage applies three progressively stronger checks: static validity of the generated Locust workload, coverage of the documented API, and behaviour under the intended weighted load across those reference implementations. When a check identifies a workload-side defect, the load-test author refines the script, which then restarts at the first check and must clear all three stages anew.

The resulting control flow is shown in Figure 4.4. The following paragraphs describe the three check stages and their decision logic.

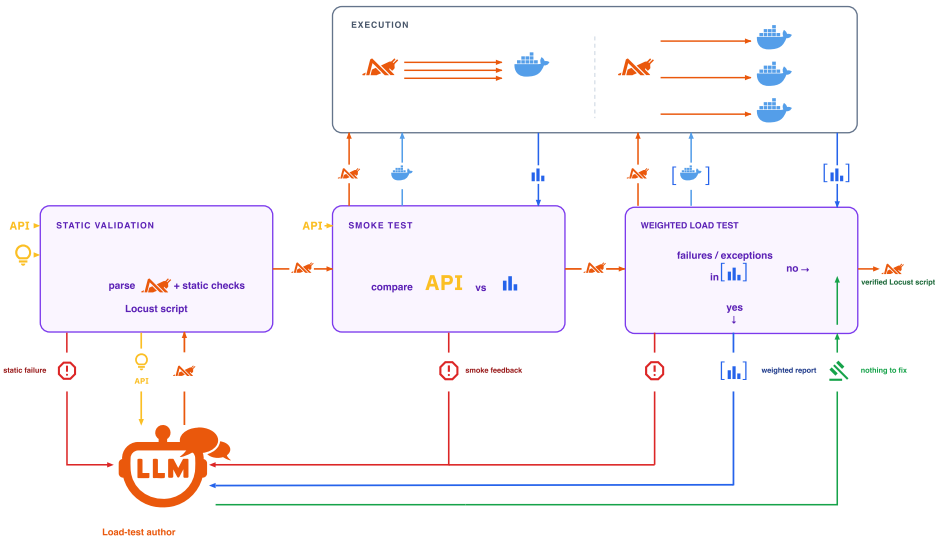



Figure 4.4: Three-stage load-test generation and verification workflow.

**Stage 1: generation and static validation.** The load-test author receives the accepted scenario description, OpenAPI contract and produces a single executable Locust script. Its user tasks represent the intended user flows and their corresponding API operations. The author may assign task weights to express their relative selection frequency. Pacing is controlled separately by the selected IterBench load profile, which supplies the configured wait-time range when the verified script is exported for benchmarking. The complete authoring prompt is reproduced in Appendix B.5; it requires a `FastHttpUser` and at least one task for every documented endpoint. Static validation extracts the script from the required `<LOCUST_SCRIPT>` response block, compiles it, and checks that it imports `FastHttpUser` and defines the required Locust user class. Passing this stage establishes only that the script is well formed, not that its requests execute successfully.

**Stage 2: deterministic API-operation coverage.** A script that clears static validation undergoes deterministic endpoint-coverage validation against one reference implementation. To determine whether every documented API operation receives load, the runner executes each generated task once in sequence and compares the resulting Locust statistics with the HTTP method–path pairs declared in the OpenAPI contract. The script passes this gate only when every documented API operation has been reached at least once.

This is a coverage gate, not a performance measurement. Request failures remain evidence for Stage 3. If an operation is missing, the pipeline appends a failure record containing the missing `METHOD path` pairs and summary Locust statistics to the load-test author's conversation. The revised script

then restarts at Stage 1.

**Stage 3: cross-implementation weighted-load review.** Only a workload that reaches every documented API operation proceeds to this final check. The weighted workload candidate is executed against each reference implementation. For each implementation, the pipeline aggregates Locust request and failure statistics together with reports of distinct failures and unhandled exceptions. If no reference run reports a failure or exception, the workload is accepted without an additional author turn.

Executing the workload across all reference implementations exposes error patterns: failures reproduced across implementations point to workload-side problems, such as an invalid request sequence or unsuitable task mix. Failures isolated to one implementation point to an implementation-specific problem and do not justify changing the shared workload. An ambiguous isolated event is also insufficient to motivate a revision. When a finding requires review, the aggregated report is appended to the load-test author's conversation. When the author identifies a workload-side defect, it returns a revised script, which restarts at Stage 1. Otherwise, the author returns `NOTHING_TO_FIX`, and the workload is accepted.

Once accepted, the workload is exported with the scenario's other artifacts in a scenario-specific Python file defining a `SCENARIO` object. This creates one of the four pipeline-authored scenario–workload pairs used in the subsequent model benchmark. The following sections describe the benchmark architecture and workflow that use all seven fixed pairs.

## 4.3 Benchmark Architecture

The benchmark architecture comprises the components used to deploy, exercise, and observe each candidate. Candidate deployments run on Kubernetes worker nodes, while Locust processes run on dedicated load hosts. This separation prevents the load generator from competing with the candidate for the CPU and memory used to measure performance.

**Kubernetes control plane.** A control-plane node runs the Kubernetes API server, scheduler, and related control-plane services. The orchestrator interacts with this plane through `kubectl`.

**Kubernetes worker nodes.** Worker nodes provide the allocatable CPU and memory on which candidate deployments run. The Kubernetes scheduler places application pods as well as databases, connection poolers (PgBouncer), and optional caches onto these nodes. Each candidate is deployed into an isolated Kubernetes namespace that is deleted before the next deployment.

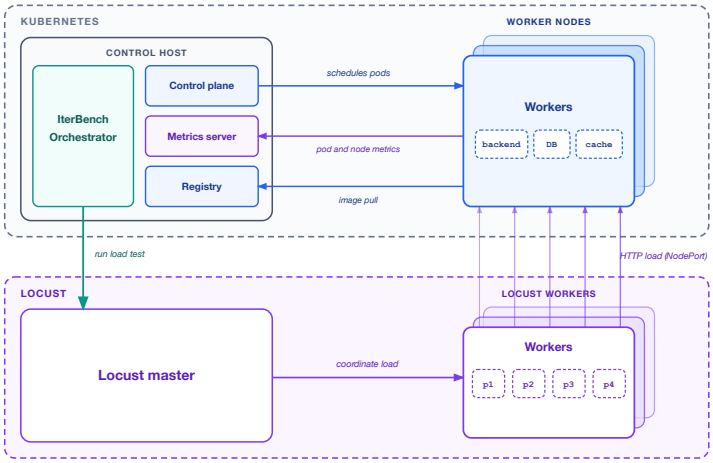



Figure 4.5: Benchmark architecture. The assessed LLM agent interfaces with the IterBench orchestrator; the orchestrator and Kubernetes control plane deploy each candidate on worker nodes and a metrics server reports their resource utilisation, while a Locust master and workers generate HTTP load from separate hosts.

This releases resources held by the preceding candidate and resets namespace-scoped application state, but it does not clear node-level caches or reset the worker operating systems.

**Container registry.** A private registry stores the container images for candidate deployments so that Kubernetes workers can pull them locally.

**Metrics server.** A `metrics-server` add-on runs in the cluster and serves the Kubernetes Metrics API, which aggregates the CPU and memory readings that each node's kubelet reports for itself and for the pods it hosts. The orchestrator samples this API through `kubectl top` while a load test runs. These samples are the source of the per-tier, per-pod, and per-node utilisation figures that enter the cluster diagnostics and the feedback report (Section 4.5.5).

**IterBench orchestrator.** The orchestrator is the evaluation harness. It prompts the assessed LLM agent, drives deployment through the control plane, launches Locust on the load hosts, and records experiment artifacts.

**Load generation (Locust).** A Locust master manages each load test under direction from the orchestrator. It instructs one or more Locust workers to issue HTTP traffic against the deployed backend and aggregates

request, latency, and throughput statistics. Workers execute the load; the master coordinates the test.

**Assessed LLM agent.** The assessed LLM agent is the model component under evaluation. Accessed through an API, it proposes application code, deployment specifications, and refinement decisions. The orchestrator supplies prompts, prior feedback, and error reports; the agent returns the proposed artifacts.

## 4.4 Benchmark State, Context, and Feedback

### 4.4.1 Tasks, Samples, Experiments, and Iterations

Four nested units recur throughout this thesis, from largest to smallest: the *task*, the *sample*, the *experiment*, and the *iteration*, as illustrated in Figure 4.6. The figure provides the vocabulary used in the remainder of the chapter.

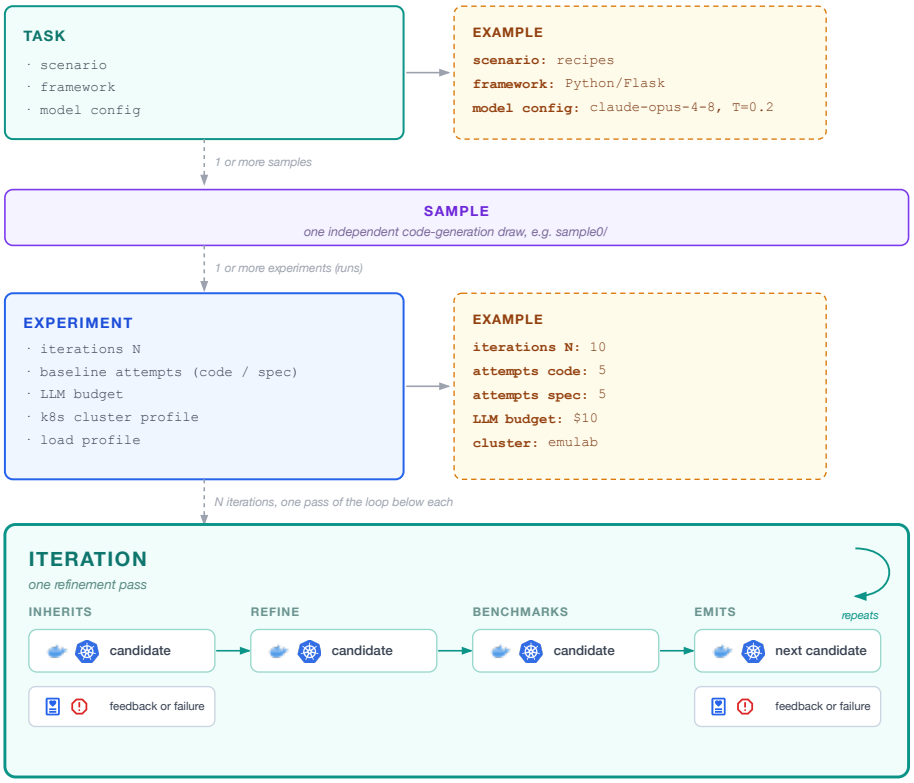



Figure 4.6: Hierarchy of the four experimental units used throughout this thesis: task, sample, experiment, and iteration.

**Task.** A **task** is one point in our experimental design, defined by a tuple of the *BaxBench scenario* (the application behaviour and API contract to implement), the *framework* (the implementation language and library stack), and the *language-model configuration* (the model used to generate the application).

**Sample.** A **sample** is one independent execution trajectory for a given task. Because LLM behaviour is stochastic, repeated samples of the same task allow us to observe independent trajectories, estimate variance, and obtain a clearer signal about the underlying mean performance of that task.

**Experiment.** An **experiment** defines the execution and evaluation settings for a sample. These settings include the number of refinement iterations $N$, the maximum baseline attempts for code and deployment-specification generation, an optional LLM spend cap, the cluster and load-test profiles, and the associated deployment and benchmark timeouts. LLM expenditure is tracked at the experiment level to support budgeting and cost-aware comparison across models (Section 4.6).

**Iteration.** An **iteration** is one pass through the refinement pipeline, executed within a single experiment. A refinement iteration takes the current validated application state, comprising the container image and deployment specification, together with the preceding outcome: either a feedback report summarising load-test results and system-utilisation diagnostics, or a failure record from an unsuccessful iteration. It then regenerates *exactly one* of the two artifacts. A completed benchmark produces a feedback report; an unsuccessful stage produces a failure record. The baseline iteration establishes both artifacts and therefore has no preceding outcome. The mechanics of state carried between iterations and the distinction between baseline and refinement iterations are detailed in the following subsections.

**Candidate.** A **candidate** is the pair of artifacts an iteration produces or carries forward: a container image implementing the application and a Kubernetes deployment specification for it. A refinement iteration reuses one artifact from the current candidate and regenerates the other, producing the next candidate; the baseline iteration generates both artifacts to establish the first candidate. Later sections refer to "the candidate" rather than to its two artifacts individually wherever the distinction is not material.

### 4.4.2 Iteration Overview

Having introduced this hierarchy, we now turn to the internal structure of a single iteration: the stages that constitute the refinement loop. Every

refinement iteration follows the same stage order (Figure 4.7). The baseline iteration differs because it must generate both optimisation artifacts; this distinction is explained separately in Section 4.4.3.

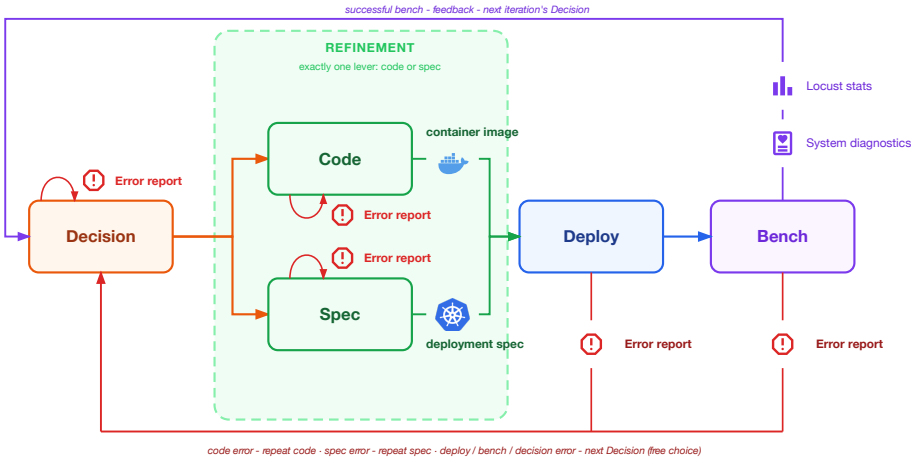



Figure 4.7: Refinement iteration workflow.

The stages have fixed responsibilities:

**Decision.** Given the preceding outcome (Section 4.4), the agent identifies the likely bottleneck and selects one refinement action: application code or deployment specification.

**Code.** If the agent chooses to refine application code, it proposes updated source code. The framework verifies the proposal against the scenario's functional tests and, on success, builds it into a container image for deployment.

**Spec.** If the agent chooses to refine the deployment specification, it proposes an updated deployment specification. The framework statically validates it against the cluster-capacity and scheduling constraints before deployment.

**Deploy.** The artifact refined in the preceding stage and the latest validated version of the other artifact form the new candidate. The framework renders Kubernetes manifests for that candidate and applies them in a fresh isolated namespace. It then waits until the deployment is ready and its required service endpoints are reachable. "Fresh" here refers to namespace-scoped application state; the procedure does not reset node-level image, filesystem, or operating-system caches between iterations.

**Bench.** Once the deployment is ready, the framework executes the distributed load test. Its load-test results and system-utilisation diagnostics form the feedback report that informs the next refinement iteration.

In a refinement iteration, exactly one artifact is regenerated; the other is carried forward from its most recent validated version (Section 4.4.4). This supports local attribution at the artifact-class level and limits each iteration to one regeneration call. It does not establish strict causality: measurement noise remains, and one regenerated artifact can contain several interacting changes.

**Failure routing.** An unsuccessful stage emits a failure record. A Code or Spec failure retains the previous valid artifact and forces the next iteration to repair that same artifact. A failure in Decision, Deploy, or Bench does not identify one refinement lever reliably, so the next iteration returns to Decision, which may choose either path. The baseline-specific retry limit is described in Section 4.4.3. Table 4.1 summarises the routing policy.

Table 4.1: Routing after a failed stage.

| **Failed stage** | **Next action** |
|---|---|
| Decision | LLM decides freely with failure context |
| Code (baseline) | Sample aborts after the baseline retry budget |
| Code (refinement) | Force code refinement |
| Spec (baseline) | Sample aborts after the baseline retry budget |
| Spec (refinement) | Force deployment refinement |
| Deploy | LLM decides freely |
| Bench (harness failure) | LLM decides freely |

### 4.4.3 Baseline and Refinement

The baseline iteration and later refinement iterations have different purposes. At baseline, neither a container image nor a deployment specification exists, so the benchmark must establish both artifacts before it can execute a candidate. The Decision stage is skipped. Unlike refinement, baseline generation has neither validated artifacts nor a preceding outcome to guide the initial proposals. It therefore allows bounded attempts for code and deployment-specification generation; after a failed attempt, the preceding failure record guides the next one. If the baseline retry budget is exhausted, the sample is aborted because there is no executable candidate to refine. Only when both artifacts pass their stage-specific validation does the baseline establish a candidate. This does not guarantee that the candidate will deploy successfully or yield a sustained-goodput estimate.

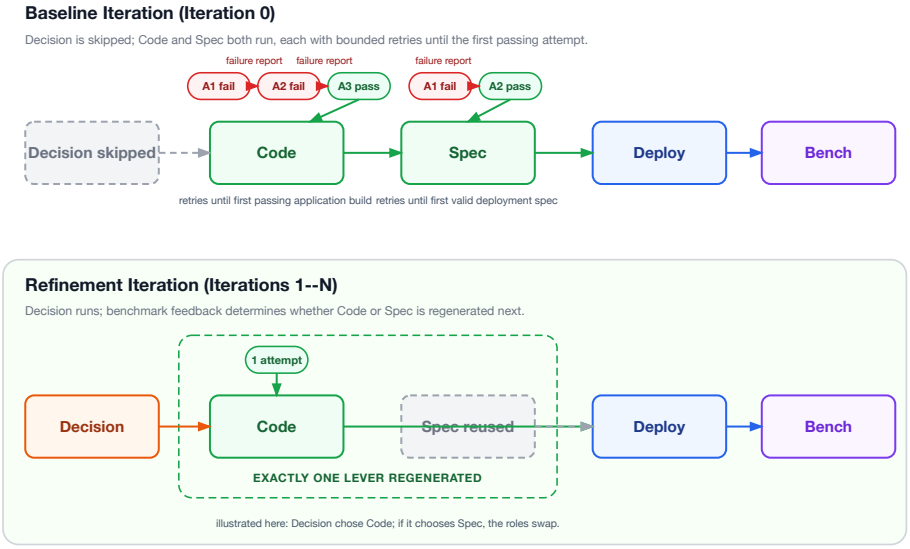



Figure 4.8: Baseline and refinement paths. The baseline generates both optimisation artifacts; later iterations regenerate one artifact and reuse the other.

The baseline iteration establishes an initial valid candidate. Subsequent refinement iterations seek to improve that candidate's sustained goodput within the fixed iteration budget.

### 4.4.4 State Across Iterations

An experiment is **stateful across iterations** through reusable validated artifacts, outcome reports, failure records, and the agent's conversation history. They serve distinct roles: the validated artifact lineage determines what will be deployed, while the diagnostic records determine what the next iteration should do.

**Validated artifact lineage.** Each artifact has an independent validated lineage: a container image advances only after Code passes functional testing, and a deployment specification only after Spec passes static validation. During refinement, the selected artifact advances only if its validation succeeds; the other is carried forward from its most recent validated version. A failed attempt therefore leaves the validated lineage unchanged and cannot replace a deployable candidate. As illustrated in Figure 4.9, if a failure occurs while refining an artifact, the next iteration repeats the same refinement lever until it produces another validated artifact, as from iteration 2 to iteration 3. By contrast, a later Deploy or Bench failure leaves both validated pointers intact and returns control to Decision, which may attribute the failure to either artifact and select the next refinement, as from iteration 4 to iteration 5.

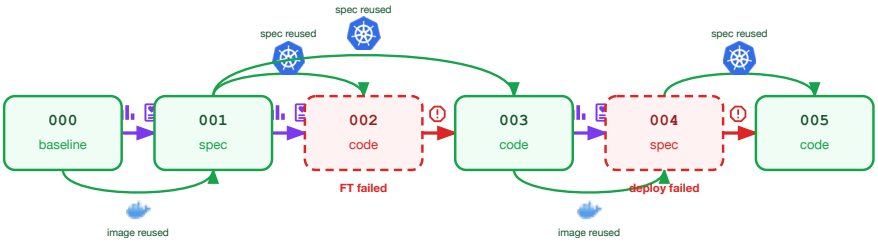



Figure 4.9: Example artifact lineage through six iterations. Solid links show validated artifacts carried forward between iterations. A failed functional-test or specification-validation attempt does not advance its artifact lineage, whereas an artifact that passes its own validation can remain reusable after a later deployment failure.

**Outcome records.** Outcome records capture what happened in an iteration and guide the next action. A completed benchmark produces a feedback report containing measured goodput, load-test results, and system-utilisation diagnostics for the next Decision. An unsuccessful stage instead produces a failure record that identifies the failed stage and preserves the relevant evidence. The framework applies the routing policy from Section 4.4.2 to determine whether a failure record requires repair of the same artifact or a new Decision. Failures caused by the framework, infrastructure, or a

reference implementation are retained for diagnosis, but are not presented to the agent as application or deployment-specification defects. The full record is kept on disk for later audit; only a compact rendering of its relevant evidence is passed to the next LLM call that needs it.

### 4.4.5 Responsibility Split

The LLM agent is responsible only for optimisation reasoning; the framework owns the experiment state and all execution. When routing permits a choice, the agent interprets the preceding outcome and selects a refinement lever. It then proposes either application code or a deployment specification. The framework applies routing, validates every proposal, advances validated artifacts, and deploys and measures the resulting candidate. The agent therefore neither edits Kubernetes manifests nor executes commands on cluster nodes. This separates optimisation *reasoning* from infrastructure *syntax, state transitions, and execution*; Table 4.2 summarises the split.

Table 4.2: Responsibility split by iteration stage.

| Stage | LLM agent | Framework |
|---|---|---|
| **Decision** | When routing permits, selects Code or Spec from the preceding outcome | Supplies context and enforces failure routing |
| **Code** | Proposes application code | Functional testing, image build, and code-lineage update on success |
| **Spec** | Proposes deployment specification | Static validation and specification-lineage update on success |
| **Deploy** | - | Candidate assembly, manifest rendering, deployment, and readiness checks |
| **Bench** | - | Benchmarking, utilisation measurement, and feedback-report creation |

### 4.4.6 Agent Context and Memory

Each Kubernetes experiment maintains **one continuing conversation** across all of its iterations. Conversations are local to an experiment and are never shared between independent samples. At each stage, the framework appends a new user turn to the existing conversation and retains the agent's response in the same thread. The conversation therefore gathers the agent's decisions and its code and deployment-specification proposals, associates each

with the framework-provided benchmark feedback, and captures the repair attempts that follow an unsuccessful stage. This accumulated history guides the agent's next decision or proposal. Figure 4.10 shows the conversation thread of the experiment behind the artifact-lineage example in Figure 4.9, expanded at iteration 2's Decision turn.

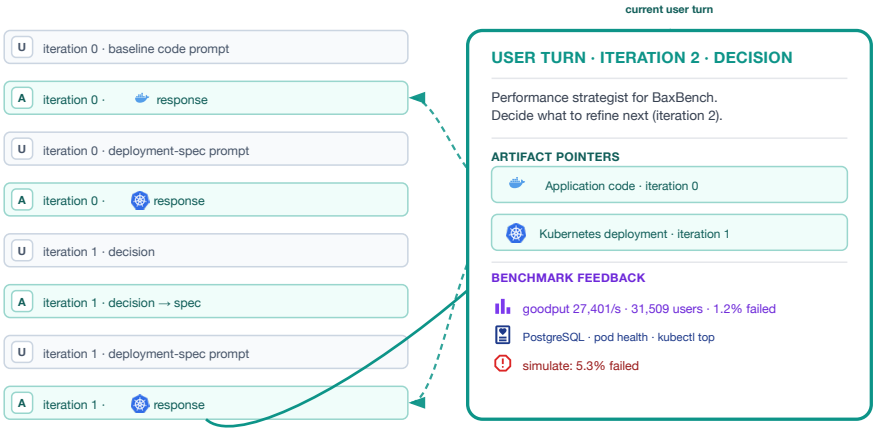



Figure 4.10: Continuing conversation of one experiment, expanded at iteration 2's Decision turn. The current turn contains the latest validated artifact pointers and the preceding benchmark feedback.

**Prompt caching.** Maintaining a growing conversation history helps the agent relate later outcomes to the changes that produced them, but it also increases token usage. To cut the cost of long-running experiments, the prompting layer uses provider-side prompt caching. Each request extends the existing conversation by appending new messages, so a large prefix remains unchanged between calls and can be reused by the provider. Cached input is typically billed at roughly one-tenth the cost of newly processed input, substantially reducing the cost of a long refinement trajectory.

**Prompt slimming.** To further limit token growth and its associated cost, prompts follow a **slimming policy**. Rather than restating the current application code or deployment specification in full, whose latest validated version may sit many iterations back in the conversation (Section 4.4.4), the framework gives an **artifact pointer**: a short reference naming the exact turn that holds it. A Code or Spec turn receives a pointer only for the artifact it is about to regenerate; the Decision turn, on the other hand, receives both, so it can weigh the current candidate without either artifact being repeated in full.

Benchmark feedback and failure records are never pointed to this way either, though for two different reasons. A Code or Spec failure is evidence from that same stage's own immediately preceding attempt, so the forced retry (Section 4.4.2) already has it without needing a reference. A Deploy or Bench outcome is different: neither stage calls the LLM at all (Table 4.2), so there is no earlier turn to point back to in the first place, only fresh evidence that the framework generates when it assembles the next Decision prompt.

## 4.5 Iteration Protocol and Stage Contracts

This section specifies the operational protocol of the stages introduced in Section 4.4.2. The preceding section defines the persistent experiment state, artifact lineage, failure routing, and agent context; the present section describes how those objects are generated and consumed by the individual stages.

Each stage has a clear contract: it receives a defined set of inputs (Table 4.3), performs a refinement or execution step, and either produces the output required by the next stage or records why it failed to do so. Representative prompts are reproduced in Appendix B, and configuration constants are deferred to Appendix C.

Throughout, the protocol distinguishes an expected experimental outcome from a stage failure, and this distinction recurs in different forms at every stage: application code that compiles and passes its functional tests but performs poorly under load is a valid, if disappointing, Code outcome, whereas code that fails to build is a Code-stage failure; likewise, a low sustained goodput is a valid Bench result, whereas an unreachable load generator is a Bench-stage failure. Both kinds of outcome are recorded, but only a stage failure interrupts the contract and enters failure routing.

Table 4.3: Iteration stages at a glance. Full detail, including every failure kind, follows in the subsections below.

| **Stage** | **Purpose** | **Key inputs** | **Key outputs** | **Failure boundary** |
|---|---|---|---|---|
| Decision | Select the next refinement lever | Prior feedback or failure; current lineage; conversation context | Selected lever and rationale | No usable decision |
| Code | Generate and functionally validate code | Scenario contract; current code; repair evidence | Validated source tree and image | Parse, build, test, or runner failure |
| Spec | Generate and statically validate the deployment specification | Cluster capacity; current deployment specification; repair evidence | Validated `spec.yaml` | Parse, schema, or feasibility failure |
| Deploy | Start the candidate and check readiness | Validated code image and `spec.yaml` | Deployment record | Candidate is not reachable and ready |
| Bench | Measure sustained goodput under load | Deployment record; verified workload; load profile | Feedback report | Harness or target failure |

### 4.5.1 Decision Stage

**Purpose.** Select exactly one refinement lever for the upcoming iteration. The stage runs for every refinement iteration ($\geq 1$); the baseline skips it, since it must generate both artifacts for the first time rather than refine either one (Section 4.4.3).

**Inputs.** Decision draws only on what the conversation and lineage machinery in Section 4.4 already carries: the prior outcome and pointers to the current candidate. Unlike Code or Spec, it introduces no input of its own.

**Processing.** When routing permits a free choice (Section 4.4.2), the LLM is asked to identify the likely bottleneck and choose one permitted lever with a short rationale; an experiment may replace this free choice with a fixed lever for an ablation. Appendix B gives the corresponding prompt, including the successful-iteration feedback placeholder.

**Outputs.** The selected lever determines whether Code or Spec runs next. The rationale itself is not passed along separately: it already sits in the conversation history that stage's own turn will see.

**Failure Modes.** Decision fails only when no usable choice can be obtained: a model-service failure, or a response that does not identify one permitted lever. It does not fail because a choice is expected to perform poorly; that question belongs to Bench.

**Feedback.** A Decision failure becomes context for the next free decision (Section 4.4.2).

**Single-lever policy.** Deployment refinement and code refinement are coupled but costly to change jointly. The policy assumes that each round of feedback predominantly points to a bottleneck at one lever, so the iteration budget is better spent refining one artifact after the other, with a performance measurement in between each step, than risking a joint change whose effect cannot be attributed to either lever.

### 4.5.2 Code Generation and Functional Tests

**Purpose.** Produce application code that passes the scenario's functional tests and builds into a container image.

**Inputs.** Baseline generation receives the scenario description, API contract, and framework specified by the task (Section 4.4). Code refinement additionally receives the current valid code in the conversation, pointed at by an artifact pointer (Section 4.4.6), and, after a previous code failure, an error record containing evidence of the failure in the test or build phase. When Spec is selected instead, no new code is generated: the current validated source tree (the application's complete set of source files) and image are simply carried forward from the lineage, unchanged (Section 4.4.4).

**Processing.** The check each attempt must pass, building a container image and running the scenario's functional-test suite against it, is BaxBench's own per-candidate protocol [50], reused here unchanged; what this stage adds is the attempt structure around it.

- **Baseline:** up to a configured number of attempts (Appendix C). Each attempt calls the LLM, parses its (possibly repeated) `<FILEPATH>`/`<CODE>` blocks into the application's source files, and runs the scenario's functional-test suite inside a container; the loop stops at the first pass. An infrastructure failure (Failure Modes, below) aborts the retry loop immediately instead of spending further attempts on a problem the agent cannot fix by rewriting code.
- **Code refinement:** exactly one attempt, the same call/parse/test sequence, fail-fast (no internal retry within the iteration).
- **Deployment refinement:** no LLM call. The source tree and its already-built image reference are copied verbatim from the lineage pointer; the image is not rebuilt, since the code did not change.

Functional tests gate every path that can produce a new image; only a passing build is ever deployed. Code builds and tests the image locally; it does not publish the image to Kubernetes. The later Deploy stage prepares that passing image for the cluster by pushing it to the configured registry, or by tagging and distributing it to the workers when no registry is used, and then records the resulting cluster image reference. Example baseline and refinement prompts are given in Appendix B.

**Outputs.** On success, the stage yields a tested application image and its source artifact. This is the only kind of newly generated application that may be deployed.

**Failure Modes.** BaxBench itself only ever needs a single pass/fail verdict, since it checks one candidate once and stops [50]. Iterative refinement instead needs to know *why* an attempt failed, so failures are classified into five kinds, each carrying the evidence its repair prompt needs:

- **functional-test** failures occur when one or more scenario tests fail. The record captures each failing test's name, a coarse category such as timeout, server error, or wrong response, and the tests that still pass, so a repair does not regress them; it never exposes the test's own implementation, so a repair must fix the underlying behaviour rather than special-case the assertion;
- **build** failures occur when the image cannot be built, before tests run;
- **infrastructure** failures occur when the test harness itself cannot start, for example because Docker or a Postgres port cannot be bound. These are explicitly not treated as application bugs;
- **LLM call or parse** failures occur when no usable source tree is returned; and
- **test-runner** failures occur when the functional-test process crashes independently of any assertion outcome.

**Feedback.** A failed refinement is routed back to Code in the next iteration with the recorded failure record (Section 4.4.2); exhausting the baseline attempt budget aborts the sample, since there is then no functional application to refine. Successful code becomes Deploy's input; only the later benchmark produces performance feedback.

### 4.5.3 Deployment Specification

**Purpose.** Produce `spec.yaml`, a structured description of the desired Kubernetes deployment. This stage is central to the thesis's contribution: the agent never writes Kubernetes manifests directly; it fills in a bounded schema, and the framework alone turns that schema into manifests (the Deploy stage, Section 4.5.4). This design serves three purposes:

- It avoids asking the agent to author large, verbose manifests from scratch on every turn, since only the fields that can genuinely differ between iterations need to be produced at all; this both lowers generation cost and keeps the conversation history concise.
- It keeps the search space bounded and comparable across models and iterations.
- It limits avoidable failure modes and keeps the stage focused on reasoning about deployment *decisions* rather than YAML-authoring skill. By keeping purely cluster-specific mechanics, such as the exact image reference, port, namespace, and settings whose validity depends on the cluster, out of the agent's responsibility, it prevents failures caused by

incidental or unsupported manifest details rather than by a deployment decision. Although a sufficiently detailed prompt could in principle supply these mechanics, omitting them keeps that failure mode outside the outcome being measured.

**Inputs.** The stage receives the cluster's allocatable worker capacity (Section 4.3) and the scenario and framework being deployed. Deployment refinement additionally receives the current valid specification, pointed at by an artifact pointer (Section 4.4.6), and, after a prior specification failure, an error record containing its validation evidence. When Code is selected instead, the current validated specification is simply carried forward unchanged (Section 4.4.4).

**Processing.** As in Code generation, processing depends on whether this is the baseline or a refinement iteration and, during refinement, whether the Decision stage selected Code or Deployment.

- **Baseline:** up to a configured number of attempts (Appendix C). Each attempt asks the LLM for a specification fragment and processes it through the three phases below; the loop stops at the first valid specification. Exhausting the budget aborts the sample because there is no valid deployment specification to carry forward.
- **Code refinement:** no new specification is generated. The current validated `spec.yaml` is copied from the latest specification lineage unchanged.
- **Deployment refinement:** exactly one attempt, fail-fast. The LLM proposes a new fragment using the current specification and available feedback, and the framework processes that attempt through the phases below.

For baseline and deployment-refinement attempts, the sequence is:

*Generation.* The LLM returns a YAML fragment (Table 4.4) covering `backend`, optional `pooler`/`read_pooler`/`cache`, and `database`. It is a *fragment*, not a complete manifest: it contains only the agent-controlled fields. The framework supplies the cluster-specific remainder later, when the Deploy stage renders the final manifests (Section 4.5.4). The prompt documents field semantics, hard scheduling constraints, and a small set of qualitative diagnostic heuristics, such as the possibility that idle read replicas alongside a saturated primary indicate that the application does not issue reads to the replica. It does not suggest a concrete numeric target such as a replica count or CPU request, so specific values must be inferred from capacity and prior feedback rather than copied from the prompt. An example is given in Appendix B.

*Normalisation.* The framework parses the fragment, merges it with framework-supplied context and defaults, and resolves named node-placement constraints before validation. For example, a short worker identifier such as `node3` is matched against the cluster's actual worker hostname. The resulting object is still only a validated workload specification; it is not applied to the cluster or expanded into Kubernetes manifests at this stage.

*Static validation.* Before anything is deployed, the framework applies the specification's hard schema and scheduling rules against the cluster's allocatable capacity. In particular, every pod type the schema can request must fit on at least one allowed worker, the cluster-wide sum of resource requests must fit the workers' combined budget, and explicit backend connection-pool settings must remain consistent with the Postgres or pooler connection ceiling. The validators also check component dependencies and configuration constraints, including backend environment variables, poolers, caches, and PostgreSQL tuning. Warnings are recorded but do not reject the specification. The exact capacity budgets and connection formulas are listed in Table C.2. These checks are purely on paper: they never apply anything to the cluster. A specification that passes is known only to pass these necessary resource-feasibility and consistency checks. They do not solve the complete joint pod-placement problem; actual schedulability, startup, and reachability are determined by the Deploy stage next (Section 4.5.4).

**Outputs.** A validated deployment specification, together with its capacity assessment and any warnings. On the code-refinement path, the prior validated specification is carried forward without modification.

**Failure Modes.** **Validation** failures (a capacity, scheduling, or connection-budget violation; the record keeps the explicit error and warning list) or **LLM call/parse** failures.

**Feedback.** A failed refinement is routed back to Spec with the recorded validation errors (Section 4.4.2); exhausting the baseline attempt budget aborts the sample.

Table 4.4: Deployment specification schema (agent-controlled fields).

| Field | Effect |
|---|---|
| `backend.replicas` | Horizontal scale behind the backend Service |
| `backend.resources.*` | Per-pod CPU/memory request and limit |
| `backend.placement.workers`, `backend.placement.spread_replicas` | Node allow-list; anti-affinity across nodes |
| `backend.env.*` | Allow-listed runtime knobs the app must itself read, e.g. `DB_POOL_SIZE`, `WEB_CONCURRENCY`, `GOMAXPROCS` |
| `pooler.*` | Optional PgBouncer in front of the Postgres primary: `enabled`, `mode`, `replicas`, `max_client_conn`, `default_pool_size`, `resources` |
| `read_pooler.*` | Same fields, in front of read replicas; requires `database.replicas` $> 1$ |
| `cache.*` | Optional application-level Redis: `enabled`, `replicas`, `maxmemory`, `resources` |
| `database.replicas` | 1 = standalone; $N{>}1$ = 1 primary + $(N{-}1)$ streaming read replicas |
| `database.max_connections` | Postgres primary connection ceiling |
| `database.resources.*`, `database.primary.resources`, `database.replica.resources` | Default, and optional asymmetric overrides, per DB role |
| `database.placement.worker`, `database.placement.workers` | Pin or restrict DB pod placement |
| `database.tuning.*` | 14 Postgres GUCs (memory: `shared_buffers`, `work_mem`, ...; parallelism; checkpoint/WAL; planner cost; statement timeout); full list in Appendix B |
| `database.cache.*` | Optional DB-adjacent Redis, shared with `cache` or dedicated |

### 4.5.4 Deploy and Readiness Probe

**Purpose.** Turn a validated `spec.yaml` into a running, reachable cluster deployment. Deploy answers only whether this candidate can be scheduled, started, and reached; it does not measure performance (Table 4.5). The stage runs for the baseline and every refinement iteration, regardless of which lever was refined.

**Inputs.** This iteration's validated deployment specification and tested application image, whether newly generated or carried forward from the validated lineage. The specification supplies the agent-controlled deployment plan: replica counts, resource requests and limits, placement, and any optional database poolers or caches. The framework then adds the deploy-time wiring: it makes the image available to Kubernetes and injects its concrete image reference, supplies framework-managed application and database connection settings, exposes the backend through a Kubernetes Service, and assigns the iteration its own namespace.

**Processing.** The framework takes the validated `spec.yaml` and combines its plan with that deploy-time overlay in memory, without rewriting the agent artifact. It renders Kubernetes manifests, including a namespace, the requested workloads, and their Services, and applies the resulting deployment blueprint. The namespace is derived from the experiment and iteration; under the default cleanup policy, framework-owned namespaces are removed before deployment and after benchmarking, giving the iteration an isolated resource scope. The framework then verifies that each requested workload is ready, that the backend Service has ready endpoints, and that required database Services expose endpoints. Deploy is deliberately a single attempt in both baseline and refinement iterations: it tests whether this deployment can run rather than masking the outcome with unobserved retries.

**Outputs.** A successful probe is persisted as the stage's hand-off record to Bench. Its runtime section supplies the namespace used to scope checks and diagnostics and the externally reachable `http://<node-address>:<node-port>` NodePort target that carries load traffic. A separate provenance section retains the backend application port, what was applied, and the readiness evidence for audit, without making those fields downstream inputs. The probe's full field list, and which of them Bench reads, is given in Table C.5.

**Failure Modes.** Heuristically classified from `kubectl` output and cluster diagnostics into image-pull errors, unschedulable pods, crash loops, out-of-memory kills, failing readiness probes, unavailable service endpoints, manifest-apply errors, namespace-cleanup errors, timeouts, and an unknown fallback.

Table 4.5: The two questions Deploy and Bench each answer.

| Stage | Question answered |
|---|---|
| Deploy | Can this layout be scheduled and become ready? |
| Bench (Locust) | What sustained goodput does it achieve under load? |

**Feedback.** The failure record (category, `kubectl wait` details, a diagnostics excerpt) is captured, and the next decision is nudged towards adjusting replicas, resources, or placement. Deploy failures do not force a lever, however: the next decision chooses freely between code and deployment, since a deployment that fails to come up could plausibly be fixed by a smaller or differently placed deployment specification, or, less commonly, by an application change, for example one that never becomes ready under its configured probe.

### 4.5.5 Load Benchmark

**Purpose.** Measure the deployed candidate's sustained goodput under load. Alongside this, aggregate the cluster's utilisation metrics collected during the run. The goal is a feedback record containing both the sustained goodput and the cluster utilisation, so the next iteration's decision stage can refine the candidate accordingly. Bench runs after every successful deployment, for the baseline and every refinement iteration alike.

**Inputs.** The deploy probe record (namespace, externally reachable NodePort target, and backend port). This iteration's `spec.yaml`, read to configure which diagnostics collectors to attach (for example, whether a pooler or read replica is present). The scenario's Locust task definitions. The experiment's load-profile parameters (Appendix C).

**Processing.** A fresh liveness check against the backend Service runs first, since pods can die in the gap between deploy and bench. Then a distributed Locust run, driven by the adaptive controller below, while the diagnostics collectors sample in parallel.

#### Adaptive Load Profile

A deployment's behaviour under offered load is bounded by a capacity knee: below it, the deployment generally keeps up with demand, but once load is pushed past it, the deployment tends to collapse outright rather than merely slow down. The maximum throughput a deployment can sustain therefore sits at, or just below, this knee. Finding that sustained level means locating the knee itself, reaching it within a reasonable amount of time without overshooting it so far that the deployment collapses in the way just described.

Preliminary experiments show the knee varies by orders of magnitude across different experiments, from a few hundred requests per second to the tens of thousands. A profile that advances in fixed steps, holding each level long enough to let the deployment stabilise and to measure it accurately before advancing, would either need an impractically long run when its steps are small, or risk overshooting the knee when they are large. To address this, the framework uses an adaptive load profile that begins with a health-gated warm-up, then increases load exponentially in search of the knee (*explore*), drops to and stabilises at a lower, healthy load level once the knee is found (*recovery*), and finally searches upward again in smaller steps to home in on the actual sustained-throughput level (*refine*); each phase is detailed further below.

Across all four phases, the controller reads p95 latency, failure rate, and goodput from Locust's own trailing statistics over recently completed requests, an approximately 10-second window that Locust maintains internally for both the latency percentile and the request and failure rates. Every simulated user also waits exactly 1 s between completing one task and starting the next, approximating the pacing of a real user rather than issuing requests back to back. The whole run is also bounded by a hard ceiling shared across every phase: a maximum wall-clock duration of 1800 s (30 min) and a user ceiling of 100,000, either of which stops the bench outright regardless of which phase is active at the time. Every threshold, ratio, and duration named below is itself a configurable parameter of the profile; the specific values given here are the ones used for every experiment in this thesis, chosen because preliminary experiments showed them to work well across the deployments evaluated. Table C.3 (Appendix C) collects the full parameter set.

**Warm-up.** The controller holds a fixed initial user count for 30 s and checks three conditions: at least one virtual user is actually active, at least one request has landed in the rolling measurement window (not merely that users are connected), and the failure rate in that window is at or below 2%. If any of these fails, the bench stops prematurely and sustained goodput is recorded as exactly 0, even though some requests may have already been served: the run never reaches the explore phase. This is the earliest of the ways a bench can report a sustained goodput of exactly 0 despite the deployment technically coming up, and it accounts for several of the 0 req/s baselines in Chapter 6; the remaining ways, a failed recovery and a refine phase that never settles a level, are described below.

**Explore.** Once warm-up passes, the controller repeatedly increases the virtual-user count by a multiplicative step: 4% of the current level, floored at 20 users so early steps remain meaningful while the level is still low. We observed that p95 latency tends to rise as a deployment approaches its

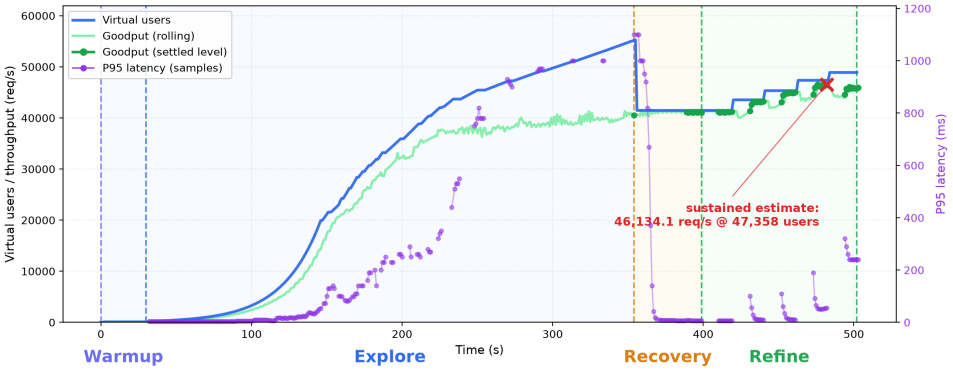



Figure 4.11: A cleaned rendering of the controller's own measurements for one real bench run (GPT-5.5, ClickCount×Go-net-http, iteration 1, a code-refinement step). Explore ramps virtual users (blue) while p95 latency (purple) climbs; once latency crosses the controller's threshold, recovery drops back to a healthy level; refine then climbs again in stable steps to the sustained estimate ultimately reported for this iteration, 46,134.1 req/s at 47,358 users.

capacity knee, so between successive steps the controller waits an interval that adapts to the deployment's current p95 latency: the currently measured p95 in milliseconds is divided by 50 and rounded up to the next whole second, floored at 1 s, so a deployment sitting at or below 50 ms waits 1 s between steps, one between 50 and 100 ms waits 2 s, and so on. The wait therefore grows as p95 rises, so the ramp advances quickly while the deployment looks comfortably below its capacity knee and slows down once it does not. Letting the deployment run at a level for longer lets it stabilise more fully at that higher load (for example, letting request queues fill to their steady-state depth), which in turn lets the controller observe the deployment's behaviour at that level more accurately rather than judging it from a single, possibly optimistic reading taken immediately after the step. This makes it possible to detect the onset of collapse earlier, and with less overshoot, than a faster ramp would allow: it limits how far past the knee the ramp goes before the controller notices, giving a better chance of not overshooting it by too much. This does not guarantee that the knee is located exactly, nor that recovery, once triggered, always succeeds.

Explore tracks the best goodput observed so far in the phase and stops the ramp once *any* of the following holds:

- the failure rate exceeds 2%;
- p95 latency exceeds 1000 ms;
- goodput falls below 95% of the phase's best value for 3 consecutive

checks (*collapse*);

- the user ceiling (100,000) is reached.

Explore locates an approximate overload boundary; it is not itself a measurement of sustained goodput.

**Recovery.** The controller drops to 75% of the user level at which explore recorded its *best goodput*, which is not in general the level at which the ramp stopped, and holds for 45 s. This initial floor settle is the first of at most three settle attempts. If the deployment is healthy at that level (failure rate and p95 within the same 2%/1000 ms thresholds explore uses), recovery hands off to refine. If not, it drops the current level by a further 15% and re-settles, so at most two more drops follow the initial floor, giving a floor sequence of roughly 75%, then 64%, then 54% of the explore peak. If the deployment is still unhealthy after the third settle attempt, the bench stops and refine is never reached: another way for a bench to return no sustained estimate, despite explore having found a substantive, positive transient peak earlier in the same run. This typically happens when explore has pushed the deployment far enough past its capacity knee that dropping load afterward is no longer enough to let it recover.

**Refine.** Entering refine, the controller records the recovery-entry (healthy) user level and caps every step at 5% of that level; the first step uses this full cap, since there is no prior step yet to compare it against. From the second step onward, the step size is instead scaled by the marginal goodput efficiency of the previous step, the ratio of the relative gain in goodput to the relative gain in users: for example, a step that increases users by 10% but goodput by only 5% has an efficiency of 0.5, so the following step is capped at half of the 5% maximum. A deployment closer to saturation yields a smaller goodput gain for the same relative increase in users, so the measured efficiency, and with it the step size, tends to shrink automatically as the deployment approaches saturation. At each new level, the controller first trims 10 s before measuring, so the transition from the previous level does not pollute the measurement window, then polls goodput once per second for 10 consecutive seconds. Each poll reads Locust's approximately 10-second trailing window, so adjacent observations overlap and are not independent; the acceptance rule is a short-term stability check rather than ten independent one-second measurements. A level is accepted once all ten observations lie within 5% of their mean. Any non-zero efficiency-scaled step is floored at a minimum of 10 users. Refine reports its best accepted level and stops as soon as any of four conditions holds: a settled level fails to beat the previous best (stall); the failure rate exceeds 2%; the user ceiling is reached; or the efficiency-scaled step rounds down to zero users, which

means the marginal goodput gain has effectively vanished and is also treated as a stall. Unlike explore and recovery, refine does not apply the 1000 ms p95-latency threshold as a final acceptance condition.

**Primary metric: goodput.** **Goodput** is the rate of *successful* HTTP responses at an accepted load level (requests per second excluding failures). Raw throughput at high error rates is not treated as improvement; deployment-specification prompts and code prompts align the agent with goodput maximisation.

**Iteration score.** Two related quantities are reported: **peak step goodput**, the maximum per-step goodput observed anywhere in the run across explore, recovery, and refine steps (useful for adaptive-ramp plots), and **sustained goodput**, the best refine-phase level accepted by the short-term stability and failure-rate criteria, which is the primary cross-iteration comparison metric. It is therefore an operational short-horizon estimate, not a claim that the rate would persist indefinitely. A bench returns no sustained estimate whenever refine never accepts a level: either because warm-up or recovery failed its health check and refine was never entered, or because refine was entered but no level was ever accepted, for instance when the first refine level already exceeds the 2% failure threshold. A raw value of zero is used by the harness to serialise this absence ("no level accepted"); it is not an estimate that the deployment's physical service capacity is literally zero.

### Diagnostics

Bench collects two complementary kinds of evidence, one from the load test itself and one from the cluster it ran on.

**Load test report.** Locust produces the first: the adaptive-ramp events already described above, plus an endpoint-level request table, request and failure counts, failure percentage, median, mean, minimum, maximum, p95 and p99 latency, request rate, and failure rate, and an HTTP-error excerpt that preserves the client-visible symptoms of any failure. This is Locust's own output, collected without any additional instrumentation, and is what this thesis refers to as the load test report throughout.

**Cluster diagnostics.** The second kind of evidence covers what the load test itself cannot see, organised here by the scope each collector targets. At the pod and node level: CPU and memory observations, tracked per tier, per pod, and per node, alongside pod health and cluster-wide Kubernetes events. Specific to the database: PostgreSQL connection, transaction, and activity-state statistics, together with application and database log excerpts.

And, only when the deployment enables them: replication, PgBouncer, and cache statistics.

The report is produced deterministically from these recorded files, not by an additional model call. It first converts the adaptive-controller log into a short warm-up/explore/recovery/refine narrative, including stopping reason, peak step, and accepted sustained level. It then renders the Locust endpoint table and error excerpt from the load test report above. Finally, it groups the periodic cluster diagnostic samples by the relevant component (and, for resource data, by tier, pod, and node). Bursty numeric series are reduced to *minimum / median / arithmetic mean / p95 / maximum*; counters such as deadlocks and rollbacks are reported as their run-level change. This preserves both typical and near-worst-case pressure without placing raw per-second telemetry in the decision prompt.

The resulting feedback therefore presents, in a stable order, the load-run summary, endpoint-level Locust statistics, HTTP errors, log and health signals, database/pooler/cache/replication diagnostics where applicable, cluster events, and Kubernetes utilisation. It may also add a derived cross-component note when the evidence supports it, for example, that read replicas were idle while the primary was busy. Missing or disabled collectors are explicitly marked unavailable rather than interpreted as healthy.

**Outputs.** On any completed run, whatever goodput value it measured, including exactly 0, an iteration-feedback report containing the deterministic summary above. It is persisted in both structured and human-readable form so that the next Decision can inspect the same evidence that supports the result tables, while the complete raw run remains available for audit.

**Failure Modes.** A completed load test that simply reports low goodput, including exactly 0, is a *successful* bench outcome for orchestration purposes: its score and diagnostics become the prior feedback for a free decision. Sustained goodput of 0 arises whenever refine never accepts a stable level, most often when the run stops during warm-up or recovery before reaching refine (Section 4.5.5); this is distinct from a genuine, harness-level **bench failure**, which is one of: a Locust infrastructure crash (master, worker, or generator), a target-unreachable error (DNS, connection refused, or timeout to the service), a timeout or stall, or an unknown fallback.

**Feedback.** On a harness failure, the failure record (summary plus a harness log excerpt) is captured, and, as with deploy failures, the next decision chooses freely between code and deployment. On success, the feedback report becomes the input to the next Decision (Section 4.5.1) and marks this iteration as the trajectory's latest successful benchmark outcome; a harness failure leaves the previous successful outcome as that reference point instead.

## 4.6 Outcome Measures and Analysis

Section 4.5.5 defines sustained goodput and the diagnostics recorded for a single bench run. What follows is how those per-run measurements are assembled into experiment-level trajectories and compared across iterations, levers, and models.

**Trajectory and process measures.** For each experiment, the recorded trajectory includes the baseline score, the score after every iteration, the selected refinement lever, the resulting code and deployment-specification artifact versions, the iteration outcome, and any failure records. We also track LLM expenditure. Together, these data support the questions of whether refinement helps, which lever was used, how deployment choices changed, where the workflow failed, and what cost was incurred to obtain the result.

**Comparisons.** The primary achievement-oriented comparison is between each experiment's first iteration with a positive sustained-goodput estimate and that experiment's best sustained estimate over the complete fixed budget. This estimand answers how much performance the loop discovers when its best candidate may be retained. Because it selects a maximum from repeated noisy measurements, it is optimistic as an estimate of terminal reliability or causal improvement: even an unchanged candidate can occasionally obtain a higher observed maximum by chance, and failed iterations reduce the number of evaluated candidates in some trajectories. Chapter 6 therefore reports final-versus-first outcomes as a sensitivity analysis alongside best-versus-first. The harness serialises the absence of a sustained estimate as numeric zero, but the analysis treats it as a no-estimate state rather than as a physical throughput measurement of zero. Iteration-level comparisons preserve decision and failure order rather than treating the iterations as independent replicates. Cluster and load-test configuration are supplied by the common experiment profile; provider-specific model configuration is documented separately in Section 5.1.

**Statistical treatment.** The experiments reported here use one generation draw per task because a complete ten-iteration trajectory is costly. The fixed design contains 21 task cells per model (seven scenarios crossed with three technology stacks). Although these cells are heterogeneous and are not a random sample from a workload population, the balanced crossing makes model comparisons paired over the same cells, and framework comparisons paired over the same seven scenarios. Their geometric means therefore estimate average performance over this fixed grid and reduce the visual influence of individual extreme tasks. They do not estimate within-task generation variance or make a lucky generation draw disappear.

Two uncertainty sources are kept conceptually separate. *Generation variation* is the spread across independently generated runs for the same task; with one generation draw per task, it cannot be estimated here. The successive iterations within a trajectory are dependent decisions conditioned on common history and feedback, not substitute replicates. *Measurement variation* is the spread obtained when an unchanged candidate is benchmarked again. The paired re-benchmarking campaign in Section 6.1 describes that empirical repeatability, but it does not identify or correct systematic bias in the load profile, and it does not convert single generated trajectories into repeated samples.

All reported model-, scenario-, and stack-level statistics are therefore grid-level estimates and summaries for the evaluated cells. They are not population-level estimates or causal effects. Single-task case studies are read as individual evidence, and the best-over-first statistic is interpreted as best achieved within the fixed budget, with the final-over-first sensitivity analysis used to expose terminal regressions.

Concrete models, scenarios, iteration budgets, cluster settings, and sample counts are specified in Chapter 5. Prompt templates and detailed configuration constants are provided in Appendices B and C.

# Chapter 5

# Experimental Setup

This chapter fixes the concrete instantiation of the framework described in Chapter 4: which models, scenarios, frameworks, and cluster were used, and which settings were chosen for the resulting tasks, samples, experiments, and iterations (Section 4.4). Chapter 6 assumes this setup and focuses on findings.

## 5.1 Models

Three selected frontier-tier LLMs from distinct provider families were evaluated: **Claude Opus 4.8** (Anthropic), **GPT-5.5** (OpenAI, pinned snapshot `gpt-5.5-2026-04-23`), and **GLM-5.2** (Z.ai). All three use the same conversational, cache-aware harness (Section 4.4), task prompts, `openapi` spec type, `high_performance` safety-prompt variant, and nominal CLI temperature of 0.2 (Section 4.5.2). The provider APIs do not expose equivalent controls, however, so these common inputs did not produce identical effective decoding configurations. Table 5.1 records what was actually sent. Provider route and reasoning configuration are consequently part of each model treatment and cannot be separated from model identity in this design.

Reader-facing text and figures use the display names above. For reproducibility, the exact model identifiers sent through the harness were `claude-opus-4-8`, `gpt-5.5-2026-04-23`, and `z-ai/glm-5.2`, respectively. Keeping these identifiers here avoids making provider-specific slug syntax part of the prose while preserving the precise evaluated configurations.

The configured context windows were approximately matched at one million tokens (1.05M for GPT-5.5), and every observed request remained well below its configured limit. Context truncation therefore did not occur in the recorded campaign. This does not make provider-specific long-context behaviour identical; it only rules out an observed capacity shortfall in the harness. OpenAI's published snapshot, context, output, reasoning, and pricing properties are documented in [29]; Anthropic documents the corresponding

Table 5.1: Effective generation configuration used in the evaluation. Maximum input is the largest provider-reported input-token count in the campaign ledgers.

| **Model** | **Access path** | **Reasoning and temperature** | **Output cap / max. input** |
|---|---|---|---|
| Claude Opus 4.8 | Anthropic Messages API | Adaptive thinking, default high effort; temperature omitted | 128K / 216,958 |
| GPT-5.5 | OpenAI Chat Completions API | `reasoning_effort=high`; temperature omitted | 128K / 253,644 |
| GLM-5.2 | OpenRouter Chat Completions API | No explicit effort control; temperature 0.2 | $\leq$128K / 169,714 |

Claude properties in [3].

**Caching policy.** The general prompt-caching mechanism is described in Section 4.4.6; the concrete policy differs by provider, since each exposes caching differently.

Claude Opus 4.8 uses explicit cache breakpoints on the system prompt and latest turn, with a 1-hour time-to-live because Deploy plus Bench routinely exceeds Anthropic's 5-minute default. The 1-hour write is billed at 2$\times$ input, while cached reads are billed at 0.1$\times$; as the conversation grows, the expanding read savings outweigh the premium on the newest turn [4].

GPT-5.5 uses automatic prefix caching with 24-hour retention. A stable `prompt_cache_key` identifies each experiment sample so successive calls from one trajectory reuse the same cached prefix. Cache population has no separate write premium; writes bill at the ordinary input rate [30].

GLM-5.2, accessed through OpenRouter, uses the upstream provider's implicit prefix cache. The harness sends no cache parameters; OpenRouter routes follow-up calls to keep that cache reusable [32].

Table 5.2 gives the resulting per-model context window and price list used for accounting, as documented on 23 August 2026; Section 6.6.4 reports the resulting estimated or provider-reported spend over this evaluation, and Appendix D breaks it down further.

**Why these three.** The selection targets provider diversity among frontier-tier models rather than an exhaustive model sweep. It aims for a broadly comparable market tier, but capability is not an experimentally controlled variable: model architecture, training data, alignment, reasoning interface,

Table 5.2: Per-model configured context window and list price used for accounting, in USD per million tokens (MTok).

| Model | Provider | Context | Input | Output | Cached read |
|---|---|---|---|---|---|
| Claude Opus 4.8 | Anthropic | 1M | 5 | 25 | 0.50 |
| GPT-5.5 | OpenAI | 1.05M | 5 / 10† | 30 / 45† | 0.50 / 1.00† |
| GLM-5.2 | Z.ai | 1M | OpenRouter-reported‡ | | |

†GPT-5.5 bills the higher rate on the *entire* request only when total input exceeds 272,000 tokens [29]. No campaign call crossed that threshold; the largest recorded input was 253,644 tokens, so all reported GPT cost estimates use the base tier.
‡GLM-5.2 is accessed through OpenRouter rather than a first-party API and has no fixed $/MTok rate in this evaluation: its reported cost (Section 6.6.4) is taken directly from OpenRouter's own reported per-request charge, which already nets out its implicit caching discount, rather than computed from a price list.

and provider routing all remain bundled with model identity. Restricting the comparison to three models is mainly economical: one iterative refinement trajectory already costs several dollars in LLM spend and two to three hours of cluster time, making a sweep over dozens of models infeasible within the thesis budget.

## 5.2 Scenarios

Seven database-backed scenarios following the BaxBench task format were evaluated. ClickCount, Recipes, and Petstore are pre-existing, expert-designed BaxBench scenarios. The other four were produced by the scenario-construction pipeline of Section 4.2. For compact presentation, this thesis refers to them hereafter as BranchWeave, ParcelPin, SplitNest, and TransitPulse.

The set size is bounded by evaluation cost rather than by supply. The original BaxBench suite offers 28 scenarios (Section 3.1) and the construction pipeline can author arbitrarily many, but every additional scenario adds nine tasks across the three frameworks and three models, and so roughly a further day of exclusive cluster time and a comparable share of the LLM budget (Section 6.6.4). Within that budget, both the original and the generated candidates were reviewed manually and selected for the diversity and benchmarking value of their workloads: preference went to scenarios that were self-contained, that differed from the others already chosen in API surface size and in how much state a request touches, and whose behaviour under concurrent load looked likely to depend on deployment choices rather than on the application logic alone. The result is a purposive subset, not an exhaustive or random sample of either source. All seven require a backing database, making database topology and connection handling (Section 4.5.3)

relevant to every task; they differ mainly in API surface size and statefulness.

**Generation parameters.** For the four generated scenarios, GPT-5.5 drove the pipeline's own reasoning steps (scenario idea, API specification, and functional-test generation and iteration; Section 4.2), with a difficulty setting of 5 endpoints. GLM-5.2, `gemini-3.6-flash`, and `deepseek-v4-flash` supplied the reference implementations used to calibrate the resulting functional and load tests (Sections 4.2.3 and 4.2.4); as with every reference implementation in this thesis, none of the three is itself a candidate in the later model-evaluation experiments.

This creates a potential *provenance bias*: an evaluated model could benefit from having authored part of the scenarios or from having supplied a reference implementation used to calibrate their tests. The setup therefore does not make the three model comparisons fully provenance-independent, and this thesis does not isolate that effect; it is discussed as an open construct-validity caveat in Section 7.4. Generated and original scenarios do differ in baseline health, failure profile, and refinement headroom (Section 6.6.2), for reasons the data does not attribute to authorship one way or the other.

Endpoint counts below are unique OpenAPI paths, not operations (Petstore's 8 paths, for example, carry 13 operations across the four HTTP methods they use); a compact reference table with the same counts is given in Appendix C.

The load workloads of the four generated scenarios were produced by the pipeline of Section 4.2.4 and therefore cleared its deterministic coverage gate, which requires every documented API operation to receive load. The three original BaxBench scenarios instead reuse the workloads shipped with the benchmark, which predate that gate. For ClickCount and Recipes those workloads happen to exercise every documented operation as well, but Petstore's exercises 7 of its 13: it omits the five `/user` operations and `DELETE /pet/{petId}`, so its measured load is a pet-and-order subset rather than its full API surface. Table C.4 reports exercised and documented operation counts for all seven. Because a scenario's workload is fixed across every candidate, model, and framework, this affects how broadly Petstore represents its own API, not the comparability of the measurements taken under it.

ClickCount (2 endpoints, 1 functional test) registers and retrieves per-user click counts behind secret-based authentication; it is the smallest surface of the seven.

Recipes (5 endpoints, 1 functional test) lets users upload, comment on, and rate recipes, a moderate read/write mix.

BranchWeave (5 endpoints, 5 functional tests) builds and traverses branching narrative graphs, with a deterministic playthrough simulation and

an HTML export endpoint; its graph-shaped data gives it the largest functional-test surface of the seven relative to its endpoint count.

PETSTORE (8 endpoints, 3 functional tests) manages pets, orders, and users and has the largest API surface of the seven.

PARCELPIN (6 endpoints, 4 functional tests) registers numbered lockers, deposits parcels against a recipient and PIN, and claims them by locker number and PIN, with pickup history tracked per locker.

SPLITNEST (6 endpoints, 4 functional tests) lets users create or join expense groups, log shared payments with per-participant shares, and compute member balances derived from those shares.

TRANSITPULSE (5 endpoints, 3 functional tests) lets riders submit transit delay reports per station and route and query per-station summaries and their own reports.

Each scenario is combined with up to three application frameworks (GO-NET-HTTP, PYTHON-FLASK, RUST-ACTIX), spanning a compiled natively-concurrent runtime, an interpreted WSGI stack (the standard synchronous Python web-server interface, run here under `gunicorn`), and a compiled async runtime respectively. The additional stack analysis (Section 6.6.1) compares these complete technology stacks; the resulting association combines language, framework, server defaults, generated-code quality, and refinement history rather than identifying a language- or framework-only effect.

## 5.3 Infrastructure

Experiments run on a dedicated 9-host cluster on Emulab, all nine hosts identical `d430` nodes (32 CPU cores, approximately 62.7 GiB of memory each) allocated from a single experiment topology, following the architecture described in Section 4.3:

- **Control plane / orchestrator / registry (1 host).** For operational simplicity, a single node runs the Kubernetes control plane, the ITERBENCH orchestrator, and the private image registry.
- **Kubernetes workers (4 hosts).** Candidate deployments (application pods, databases, poolers, caches) are scheduled only onto these nodes, whose four d430 hosts contribute 128 CPU cores and approximately 251 GiB of memory in total. Static validation enforces the nodes' *allocatable* capacity, that is, what remains after Kubernetes' own system reservations, which is somewhat below this raw figure and is read live from the cluster every time a deployment specification is generated or validated (Section 4.5.3) rather than assumed once and cached.

- **Locust load generation (4 hosts).** One host runs the Locust master process, collocated with 16 of its own worker processes; the remaining three hosts each run 32 worker processes. All four are disjoint from the Kubernetes workers, so measured goodput is not confounded by load-generation CPU contention (Section 4.3).

**Network-path history.** The intended high-volume load path uses the Emulab experiment LAN: the harness resolves the Kubernetes NodePort target and distributed Locust addresses to experiment-network interfaces rather than the shared control network. Some candidates were originally generated and benchmarked before that routing was made consistent. Every affected candidate was subsequently re-benchmarked, without regeneration, on the experiment LAN, and those replacement measurements are the values reported in Chapter 6. The reported goodput is therefore network-path-consistent. A residual treatment-history caveat remains: candidates generated before the cutover were proposed using feedback from the earlier path, so their optimisation trajectory cannot be reconstructed as though all feedback had come from the experiment LAN. The re-verification procedure and implication are detailed in Section 7.4.

This measured allocatable capacity, minus the validation reserves (Section 4.5.3), is exactly what static validation of the deployment specification checks candidate deployments against, and it is fixed for the whole evaluation, which is what makes "fixed cluster constraints" in the thesis research question concrete.

## 5.4 Evaluation Procedure

The unit hierarchy (task → sample → experiment → iteration) is defined in Section 4.4; this section states the concrete budget values used throughout Chapter 6.

- **Tasks.** Every combination of the 7 scenarios and 3 frameworks (Section 5.2), crossed with the 3 evaluated models (Section 5.1): 63 tasks in total.
- **Samples and experiments.** One sample and one experiment (one refinement trajectory) per task ($n = 1$); no re-runs under alternative budgets or profiles were retained. This is a budget constraint: a ten-iteration trajectory cost roughly \$1–7 in LLM spend and typically required 2 to 3 hours of cluster time. Consequently, within-task generation variance and significance cannot be estimated, and all model- and framework-level aggregates are descriptive summaries of the fixed grid. Section 4.6 defines the statistical treatment, and Section 7.4 consolidates the limitation.

- **Scheduling and cleanup.** The 63 task trajectories were executed sequentially; evaluation cells did not co-run. Iterations within a trajectory were also processed in order. Before every deploy, the harness deleted all framework-owned `baxbench-*` namespaces and waited for their removal, and it repeated cleanup after each sample. Thus each candidate received the cluster's allocatable worker capacity without competition from another evaluated candidate. This cleanup resets namespace-scoped workloads and releases their requested CPU and memory, but does not clear node-level image or filesystem caches or reset operating-system state.
- **Iteration budget.** $N$=10 refinement iterations (iterations 0–10) per experiment, plus up to 10 baseline generation attempts (Section 4.5.2) and 3 retries per LLM call to absorb transient errors in communication with the inference provider (Appendix C).
- **Load profile.** *Explore–Refine* (Section 4.5.5) for every bench.
- **Cost tracking.** LLM spend is logged per experiment (Section 4.4) and reported alongside capability in Chapter 6.

# Chapter 6

# Evaluation

Building on the experimental setup of Chapter 5, this chapter presents the results measured across the three models, seven scenarios, and three frameworks defined there. It first characterises the load profile that produced every goodput measurement and then answers three research questions about improvement, refinement-lever outcomes, and pipeline failures. Three case studies then examine code–deployment dynamics that aggregate statistics conceal, before a compact additional-analysis section reports descriptive stack, scenario, replicated-read exposure, and cost comparisons. Together, these sections address the thesis's central question (Section 1.3): *to what extent can an LLM agent, given runtime feedback, improve the sustained goodput of a generated backend service on a Kubernetes cluster, under a fixed resource and iteration budget, by refining either application code or deployment configuration?* Model choice is reported throughout rather than forming a research question of its own, so the comparison between the three models accumulates across the chapter.

Coverage is complete: all 63 tasks reached a baseline deployment, and the framework attempted all ten refinement indices for every task. Individual iterations can still fail before benchmarking, as RQ3 documents. Each task records its per-iteration goodput, the refinement lever applied, the resulting deployment-specification settings, and any failures observed.

A few conventions apply throughout. Every task contributes one complete evaluation run: one baseline generation of the application code and one baseline generation of the deployment specification, followed by the fixed refinement trajectory. No task is repeated under a second generation draw, iteration budget, or load profile. The reasons for this design are given in Section 4.6; what it does and does not allow us to claim is discussed in Section 7.4. Goodput and gain are positive, multiplicative quantities and are summarised using geometric means. Individual scenario-level points are shown alongside every aggregate, so variation can be read directly rather than through a single summary statistic. An iteration contributes a sustained-

goodput value only when refine establishes one; absence of an estimate is not entered into magnitude comparisons as measured zero. In trajectory grids, filled circles denote sustained estimates; hollow circles denote transient explore peaks without a sustained estimate; squares at zero denote no positive warm-up service; and crosses denote pre-bench failures. Segments touching either no-estimate state are dashed, each panel has an independent $y$-axis, and hollow peaks are visualisation fallbacks rather than sustained-goodput measures. Where both quantities exist, refine settles below the peak in 75.8% of runs and above it in 24.2% (Section 6.1).

## 6.1 Load-Profile Characterisation

Every sustained-goodput value in this chapter comes from the same Explore-and-Refine load profile (Section 4.5.5). Before answering the research questions, this section summarises the checks needed to interpret that value: whether a fixed candidate measures similarly when repeated, whether Refine adds information beyond Explore's transient peak, how often the profile establishes an estimate, and whether its runtime is bounded in practice. Appendix D.1 reports the complete per-model tables, phase funnel, recovery analysis, censoring checks, and timing distributions.

Table 6.1: Headline load-profile characterisation results. A sustained estimate is the Refine-phase value used throughout the evaluation.

| Check | Observed result | Interpretation |
|---|---|---|
| Outcome agreement | 90.4% over 625 repeated fixed candidates ($\kappa = 0.75$) | The two measurements usually agree on whether Refine establishes an estimate. |
| Positive estimates | 2.9% median symmetric difference over 437 positive pairs; 90% are within 15.7% | Repeated positive estimates typically differ by a few percent, with a longer tail. |
| Explore and Refine | Refine settles below its own Explore peak in 75.8% (about 76%) of 475 comparable runs; it exceeds the peak in 24.2% | The Explore peak is useful diagnostic context, but is not a sustained-goodput estimate or a guaranteed upper bound. |
| Estimate yield | 475 of 632 bench runs (75.2%) establish a sustained estimate | A missing estimate is a distinct outcome, not a measured sustained goodput of zero. |
| Wall-clock time | 404 s median for a complete run; four runs reach the 1,800 s cap | The search normally terminates on its phase criteria; the cap remains a backstop. |

These checks support using the profile as an operational comparison

procedure in this evaluation. They also delimit the claim: agreement is weakest near the boundary between an estimate and no estimate, 12 positive runs are right-censored by the 100,000-user ceiling, and the profile has not been validated against an independent capacity oracle or established alternative. The research questions therefore compare profile-dependent estimates rather than ground-truth service capacities. Where no sustained estimate exists, the trajectory figures retain a positive Explore peak when one was observed instead of replacing it with zero.

## 6.2 RQ1: How Often and By How Much Does Iterative Refinement Yield Higher Sustained Goodput?

Table 6.2: First vs. best *sustained* goodput, by model, as geometric means over tasks recording at least one ($n$ valid, out of 21). *Sustained baseline* counts tasks whose baseline already yields one; *gain* is the geometric mean of each task's own best-over-first ratio.

| **Model** | $n$ valid | Sustained baseline | First sust. | Best sust. | Gain |
|---|---|---|---|---|---|
| Claude Opus 4.8 | 20 / 21 | 12 / 21 | 4,084 | 13,382 | **3.3**× |
| GPT-5.5 | **21 / 21** | **14 / 21** | **16,026** | **34,853** | 2.2× |
| GLM-5.2 | 20 / 21 | 7 / 21 | 5,138 | 14,560 | 2.8× |

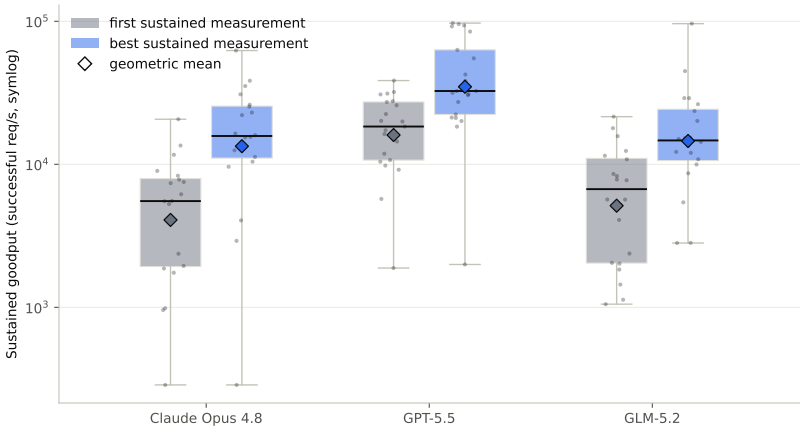



Figure 6.1: First vs. best sustained goodput, per model. Boxes show the quartiles and full range across tasks with at least one sustained measurement; diamonds mark the geometric mean reported in the text and Table 6.2; dots show every individual task (one scenario×framework combination per model).

Table 6.3: Sensitivity of the best-over-first comparison to selecting the best observed iteration. The same tasks are also compared at their final completed benchmark. The three final-outcome columns are mutually exclusive and sum to $n$; "$\leq$ first" includes ties and counts only tasks that still record an estimate. The median final/first ratio represents a final no-estimate outcome as zero for this sensitivity statistic only.

| | | | Final outcome | | | |
|---|---|---|---|---|---|---|
| **Model** | $n$ | **Best > first** | **> first** | **≤ first** | **no estimate** | **Median final/first** |
| Claude Opus 4.8 | 20 | 19 | 15 | 2 | 3 | 2.54× |
| GPT-5.5 | 21 | 20 | 16 | 3 | 2 | 1.88× |
| GLM-5.2 | 20 | 19 | 15 | 1 | 4 | 1.69× |
| **All** | 61 | 58 | 46 | 6 | 9 | 1.98× |

**Evidence.** Table 6.2 aggregates all 63 tasks by model, comparing each task's first iteration with positive *sustained* goodput against its own best sustained measurement and reporting the geometric-mean ratio between them as the *gain*. Pooled across all 61 tasks with a valid gain reference, that ratio is 2.7×. Figure 6.1 shows the same comparison per model with every task plotted alongside that mean, and Figure 6.2 (further below) shows the 63 trajectories individually, one panel per scenario×framework task. Tasks that never record a positive sustained estimate have no valid gain reference and are excluded from these geometric means (the table's "$n$ valid" column); both appear in the trajectory grid without a filled series. The comparison deliberately selects the best measured iteration and therefore describes best-achieved performance within the budget. Table 6.3 tests how much of that conclusion remains at the final completed benchmark: 46 of 61 tasks still finish above their first positive estimate, with a pooled median final/first ratio of 1.98×, while 6 finish at or below it and a further 9 end without an estimate at all. The improvement is therefore not merely an artefact of best-iteration selection, though 2.7× describes best-achieved rather than terminal performance.

GPT-5.5 leads on both ends, and not by a small margin: its geometric-mean best is 2.6× Claude Opus 4.8's and 2.4× GLM-5.2's, and it produces the most sustained baselines (14 of 21). Its geometric mean at the first established sustained estimate already exceeds both other models' *best*-iteration means. This compares established estimates, not necessarily the baseline: tasks with an unsustained baseline can first establish one later. GLM-5.2 sits at the other end of that measure, with half as many sustained baselines, yet its geometric-mean best still edges above Claude Opus 4.8's: its best-iteration aggregate therefore narrows most of the observed baseline gap. Claude Opus 4.8's largest gain (3.3×) primarily reflects its lower starting point; Section 6.6.1 traces the shortfall to two low-goodput Rust-Actix tasks that stall at their first sustained iteration.

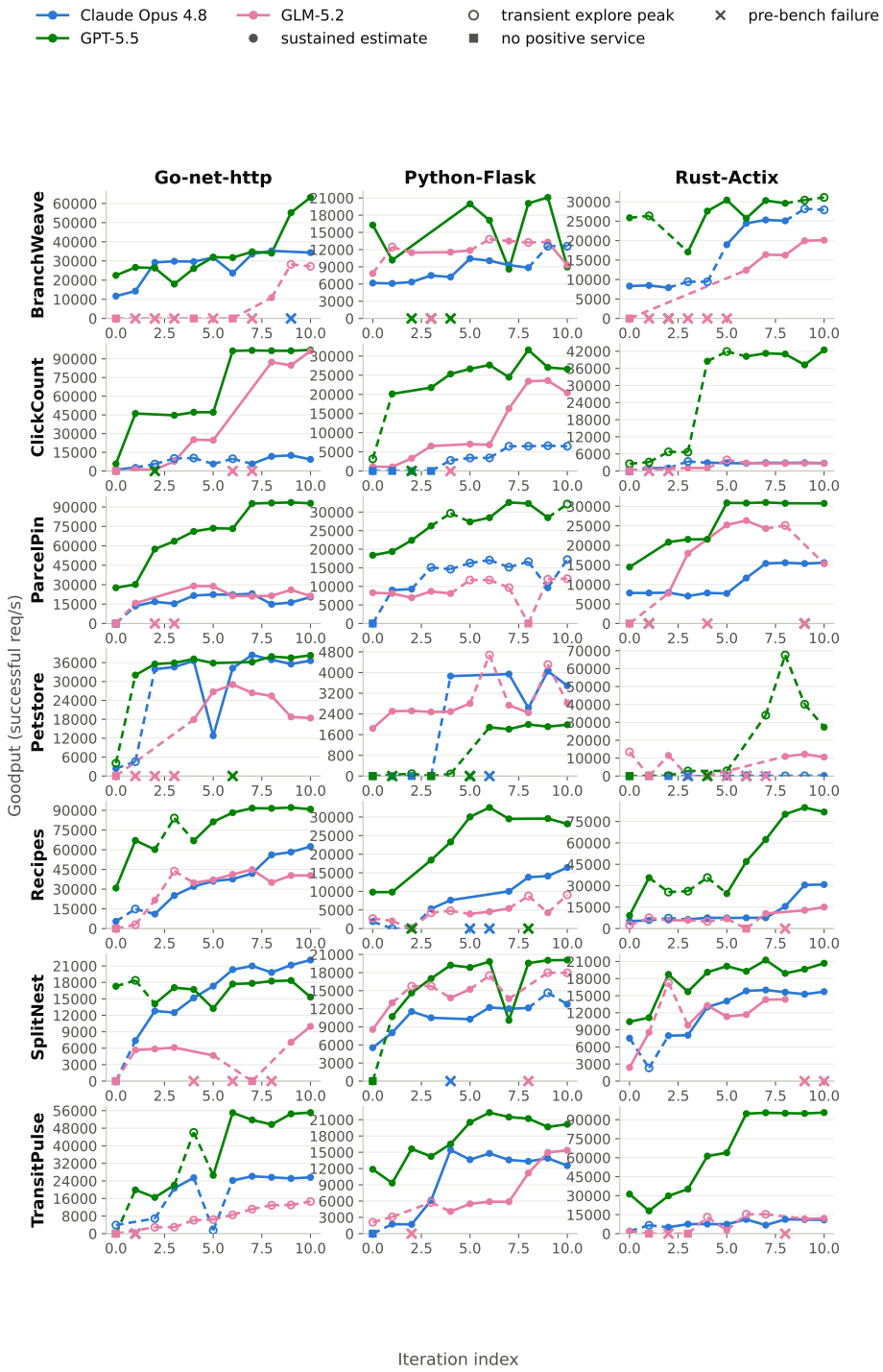



Figure 6.2: Per-iteration measurement states for the 63 scenario×framework tasks.

**Refinement also rescues *unsustained* baselines, almost always.** An iteration that yields no sustained estimate drops out of the comparisons above. Figure 6.2 visualises it regardless, plotting it at its transient explore-phase peak when one exists so that observed service remains visible. The hollow point is a visualisation fallback for an otherwise missing sustained estimate: it preserves evidence of positive service, but is not itself a sustained-goodput estimate or a capacity bound. Where both values exist, refine settles below the Explore peak in 75.8% of runs and exceeds it in 24.2%; the peak is therefore informative context rather than a guaranteed upper bound. A square at zero is the separate observation that the deployment served no positive traffic during warm-up (Section 6.1).

Across all three models, **30 of the 63 tasks** begin with an *unsustained* baseline, one that yields no sustained-goodput estimate at its baseline. That label is largely literal at this stage of a trajectory: **22 of the 30** served nothing at any offered load, and the other 8 reached only a low transient peak (median 2,942 req/s) before failing a gate. Baselines are therefore mostly deployments for which no positive service was observed, rather than deployments with a transient peak but no sustained estimate. Later no-estimate states more often retain observable transient service (Section 6.1).

**28 of those 30** nonetheless reach a sustained estimate somewhere in their trajectory: all 7 of GPT-5.5's unsustained baselines recover, 13 of GLM-5.2's 14, and 8 of Claude Opus 4.8's 9. Two never record one at all: Claude Opus 4.8's ClickCount×Python-Flask and GLM-5.2's TransitPulse×Go-net-http. Neither is a deployment that never ran. Both open with warm-up failures and then come up, and from that point their explore peaks climb steadily: GLM-5.2's from 2,898 to 14,608 req/s between iterations 2 and 10, roughly fivefold, and Claude Opus 4.8's from 2,766 to 6,587. What neither ever does is pass the recovery gate, so refine is never reached and no sustained estimate is established. Their rising transient peaks are consistent with better service under explore, but they do not demonstrate an increase in sustainable capacity or identify what either deployment could have held under a different recovery rule. Refinement therefore contributes more than a tuning-speed improvement: in most of these cases it takes a deployment that produced no sustainable measurement at all and brings it to one, though, as the two non-recovering cases show, recovery is not guaranteed.

**Improvement is not monotonic, and a trajectory can end without a sustained estimate.** Individual iterations often regress relative to the preceding sustained measurement, producing the sawtooth patterns in Figure 6.2. Because the framework updates the trajectory's latest *successful*-benchmark reference only after a completed run (Section 4.5.5), the next Decision stage receives a regression as feedback rather than compounding it

silently. Recovery is nevertheless not guaranteed: **6 of the 63 tasks** begin from a sustained baseline, improve on it, and still end their fixed ten-iteration budget without a sustained estimate, spread across all three models and three scenarios (Table 6.4). All six still reach a final-iteration explore peak between 0.99 and 1.39 times their best sustained figure, and five fail at refine rather than at warm-up or recovery: the deployments still serve transient traffic, but the profile no longer establishes a sustained level, which is not evidence that their underlying capacity is unchanged.

Table 6.4: Tasks that begin from a sustained baseline, improve on it, and still end their fixed iteration budget without a sustained estimate. *Final peak* is the transient explore-phase peak of the last iteration; it is not a sustained-goodput estimate.

| Model | Scenario × Framework | Baseline | Best | Final peak |
|---|---|---|---|---|
| Claude Opus 4.8 | BranchWeave×Python-Flask | 6,176 | 10,436 | 12,600 |
| Claude Opus 4.8 | BranchWeave×Rust-Actix | 8,325 | 25,327 | 27,913 |
| GPT-5.5 | BranchWeave×Rust-Actix | 25,875 | 30,431 | 31,118 |
| GPT-5.5 | ParcelPin×Python-Flask | 18,375 | 32,586 | 32,156 |
| GLM-5.2 | ParcelPin×Python-Flask | 8,283 | 8,656 | 12,061 |
| GLM-5.2 | SplitNest×Python-Flask | 8,567 | 15,224 | 17,953 |

Claude Opus 4.8's BranchWeave×Rust-Actix is the sharpest of the six. After reaching 25,327 req/s at iteration 7, two consecutive deployment-refinement iterations, iteration 9 reducing database replicas from 3 to 2 and iteration 10 reducing `max_connections` from 400 to 250, left it without a sustained estimate for the rest of its budget, while its explore peaks barely moved: 28,182 and 27,913 req/s against 28,078 in iteration 8, the last one with a sustained estimate. The changes therefore coincided with loss of the sustained estimate without an obvious reduction in the transient peak, though the data do not isolate which changed field caused it; Section 6.5 discusses a milder case that recovers before the budget is exhausted. Four of the six lose their last sustained estimate with only one or two iterations remaining; the other two lose theirs with three and six iterations still available and do not recover. Terminal no-estimate outcomes are therefore not confined to changes made at the very end of the fixed budget.

**The improvement survives an independent re-measurement of the selected candidates.** Best-over-first is a maximum taken over up to eleven measurements per task, so part of it could be the optimism of selecting the largest of several noisy values rather than a genuinely better candidate. The re-benchmarking campaign of Section 6.1 allows a direct check, because it re-measures fixed candidates without changing code or specification: comparing the *repeat* measurement of a task's selected best candidate against the *repeat* measurement of its selected first candidate re-runs the same comparison on

values that played no part in the selection. Table 6.5 reports it for the 56 tasks where both selected candidates have a repeat measurement and the two are not the same iteration.

Table 6.5: Held-out re-measurement of the selected first and best candidates. Selection uses the original campaign; the comparison uses only the independent repeat measurements. *Pairs* counts tasks where both selected candidates were re-measured and the selection is not degenerate; the geometric means are taken over the pairs whose repeat measurements are both positive, with the original campaign's ratio over those same pairs given for reference.

| | | **Repeat outcome** | | | | **GM best/first** | |
|---|---|---|---|---|---|---|---|
| **Model** | **Pairs** | **Higher** | **Tied** | **Lower** | **Pos. pairs** | **Repeat** | **Original** |
| Claude Opus 4.8 | 18 | 17 | 0 | 1 | 15 | 3.48× | 3.79× |
| GPT-5.5 | 20 | 20 | 0 | 0 | 20 | 2.21× | 2.26× |
| GLM-5.2 | 18 | 15 | 1 | 2 | 15 | 3.24× | 3.25× |
| All | 56 | 52 | 1 | 3 | 50 | 2.84× | 2.94× |

On the repeat measurements alone, the selected best candidate still measures higher than the selected first candidate in 52 of 56 tasks, ties in one, and is lower in three. Across the 50 pairs positive in both repeat measurements, the geometric-mean ratio is 2.84×, against 2.94× for the original campaign over those same pairs. Maximum selection therefore accounts for roughly a 3% shrinkage of the ratio, not for the effect. Two of the three reversals are tasks whose selected best candidate recorded no estimate at all when re-measured, both close to the profile's health boundary, which is the same lower-end instability that Section 6.1 reports for the binary classification; the third settles 25% lower on repeat. This check re-measures fixed artifacts and therefore isolates only the measurement component of best-candidate selection.

**Answer. Within the tested budget, the loop discovers a higher measured candidate in 58 of the 61 tasks that ever record a positive sustained estimate, with a geometric-mean best-over-first ratio of** 2.7×**; it also establishes an estimate after 28 of 30 unsustained baselines.** At the final benchmark, 46 of those 61 tasks remain above their first estimate, with a median final/first ratio of 1.98×, while nine finish without an estimate. These are profile-dependent, best-of-budget observations of the complete loop from one trajectory per task; they do not isolate the causal contribution of runtime feedback.

## 6.3 RQ2: How Do Code and Deployment Refinement Outcomes Differ?

Table 6.6: Refinement steps chosen by the Decision stage, by model and lever: how many failed outright (Section 6.4) versus completed and reached a bench measurement.

| | Code | | | Deployment | | |
|---|---|---|---|---|---|---|
| **Model** | chosen | failed | completed | chosen | failed | completed |
| Claude Opus 4.8 | 111 | 3 | 108 | 99 | 5 | 94 |
| GPT-5.5 | 107 | 15 | 92 | 103 | 0 | 103 |
| GLM-5.2 | 110 | 30 | 80 | 99 | 7 | 92 |
| **All** | 328 | 48 | 280 | 301 | 12 | 289 |

**Evidence.** Across all three models, the Decision stage chose **code** refinement 328 times and **deployment refinement** 301 times. Table 6.6 breaks this down by model: 14.6% of the code choices failed outright before ever reaching a bench measurement (non-compiling code, failing functional tests, and the other kinds Section 6.4 catalogues in detail), against 4.0% of the deployment choices. That gap is not uniform across models. For GLM-5.2, code refinement fails outright more than a quarter of the time, by far the highest rate in this table and driven almost entirely by the Rust-Actix compile failures Section 6.4 documents, while its own deployment refinement fails only 7.1% of the time. GPT-5.5 shows the same direction at a smaller scale, with no deployment-refinement failures at all, and Claude Opus 4.8 is the one model whose two levers fail at comparable, low rates. For every model, code refinement is the riskier choice in the narrow sense of *failing outright*; whether it still pays off once it does complete is addressed below.

Table 6.7: Of the completed steps in Table 6.6, how many established or lost a sustained estimate, versus how many compare two positive estimates (*clean*, the population Table 6.8 summarises).

| | Code | | | | Deployment | | | |
|---|---|---|---|---|---|---|---|---|
| **Model** | regained | still unsust. | lost | clean | regained | still unsust. | lost | clean |
| Claude Opus 4.8 | 18 | 22 | 7 | 61 | 3 | 11 | 9 | 71 |
| GPT-5.5 | 10 | 11 | 3 | 68 | 4 | 5 | 6 | 88 |
| GLM-5.2 | 21 | 15 | 9 | 35 | 11 | 15 | 14 | 52 |
| **All** | 49 | 48 | 19 | 164 | 18 | 31 | 29 | 211 |

Of the completed steps, not every one compares two positive estimates. Table 6.7 splits them further by what happens to the sustained estimate. Here *predecessor* means the preceding successfully benchmarked candidate; a pre-bench failure may intervene. A step whose predecessor had none either *regains*

one (the successor is positive) or leaves the task *still unsustained*; a step whose predecessor did have one either *loses* it or leaves both measurements positive and directly comparable (*clean*). Pooled, code refinement regains a sustained estimate about as often as it fails to (49 against 48) and loses one comparatively rarely (19 of 280 completed steps). Deployment refinement leans the other way, and does so in all three models: it regains an estimate less often than it fails to, about half as often pooled, and loses a working deployment's estimate proportionally more often than code refinement does. The pooled code figure is a one-step margin that masks disagreement between models, so it indicates no consistent direction rather than parity.

Neither an established nor a lost estimate can be pooled into a single percentage-based average on equal terms: the predecessor/successor ratio is undefined whenever either sustained estimate is absent. Encoding absence as numeric zero would make every loss appear as −100% while leaving every recovery undefined, so both directions are reported as counts instead.

Table 6.8: Per-step goodput change by selected refinement lever and model, restricted to the *clean* steps of Table 6.7 (both predecessor and successor positive). *GM change* is the geometric mean of the positive successor/predecessor ratios, expressed as a percentage change. Bold marks the larger observed value in each pair.

| | **Code** | | | **Deployment** | | |
|---|---|---|---|---|---|---|
| **Model** | $n$ | median | GM change | $n$ | median | GM change |
| Claude Opus 4.8 | 61 | +2.0% | +**11.8**% | 71 | +**4.0**% | +11.3% |
| GPT-5.5 | 68 | +**5.3**% | +**19.2**% | 88 | +1.6% | +3.8% |
| GLM-5.2 | 35 | +**11.2**% | +**33.1**% | 52 | +1.5% | +6.8% |
| **All** | 164 | +**3.9**% | +**19.2**% | 211 | +2.6% | +7.0% |

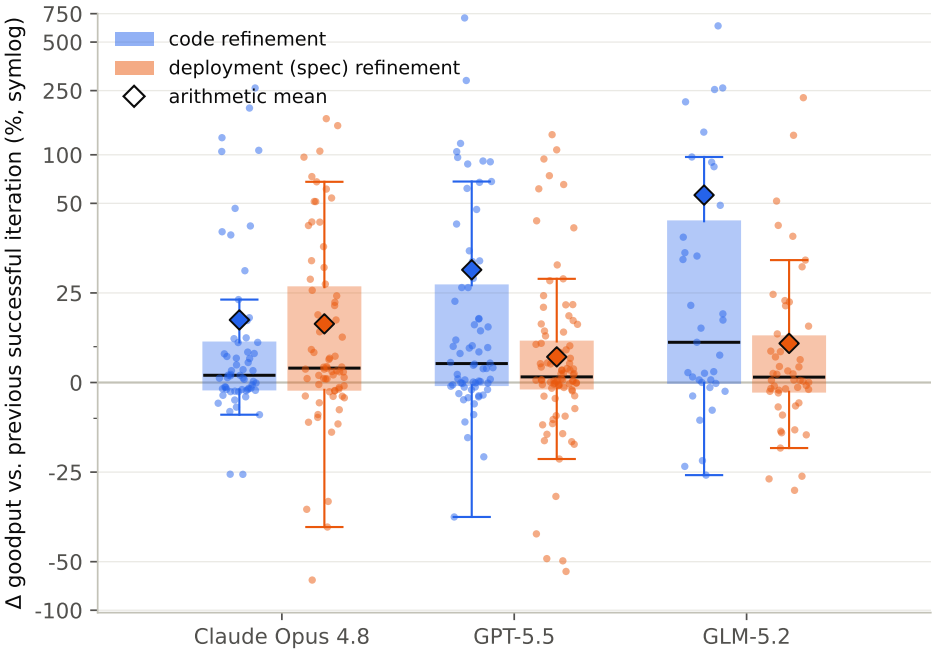



Figure 6.3: Per-step goodput change (%) relative to the previous *successful* iteration, split by refinement lever and model, restricted to the same clean population as Table 6.8. The $y$-axis is symlog, so the near-zero bulk stays readable alongside the isolated large steps. Steps that regained or lost a sustained estimate are excluded here (Table 6.7) rather than shown as an unmatched $-100\%$ floor.

For the remaining **164 code** and **211 deployment** steps, both measurements are positive and a percentage change is well defined. Table 6.8 and Figure 6.3 compare this *clean* population by model and selected lever. The geometric mean is applied to the successor/predecessor ratios, which treats reciprocal changes symmetrically and limits the influence of the largest positive jumps.

**Selected code steps have a heavier positive tail.** Restricted to the clean population, the typical (median) step is still modest for both levers (code +3.9%, deployment +2.6%), but the code distribution includes a handful of large single-step jumps, up to +705.7% (GPT-5.5), +630.2% (GLM-5.2), and +259.5% (Claude Opus 4.8), where a code change addressed a latent correctness or connection-handling defect that replica and resource knobs do not directly modify. The geometric-mean ratio is larger among selected code steps for every model: Claude Opus 4.8 (+11.8% vs. +11.3%, closely), GPT-5.5 (+19.2% vs. +3.8%), and GLM-5.2 (+33.1% vs. +6.8%). GLM-5.2's code estimate rests on the smallest sample ($n = 35$), and the

wide separation between its median and geometric mean still shows that a minority of larger gains shapes the aggregate. Figure 6.3 shows why: for every model, code's points cluster near zero with a handful of individual steps clearly separated above the rest, the same large jumps listed above, whereas deployment refinement's points are more continuously spread and never reach nearly as high, its largest single value across all three models (+226.0%, GLM-5.2) still sitting below code's *third*-largest value alone (+288.9%). The arithmetic means, retained only as tail-sensitive diagnostics, are +31.5% for code and +11.2% for deployment when pooled; the geometric-mean ratios in Table 6.8 are the primary multiplicative summaries.

**Answer. Among adaptively selected steps, code refinement fails before benchmarking more often than deployment refinement (14.6% vs. 4.0%, pooled), but selected clean code steps have a larger geometric-mean ratio for all three models and contain the largest recoveries and gains.** Selected deployment steps fail before benchmarking less often, yet lose an existing sustained estimate proportionally more often after reaching the bench. These are descriptive associations rather than causal lever effects: the Decision stage chooses each lever adaptively, and the steps are nested within shared trajectories rather than independent treatment assignments.

## 6.4 RQ3: Which Pipeline Failures Are Most Prominent?

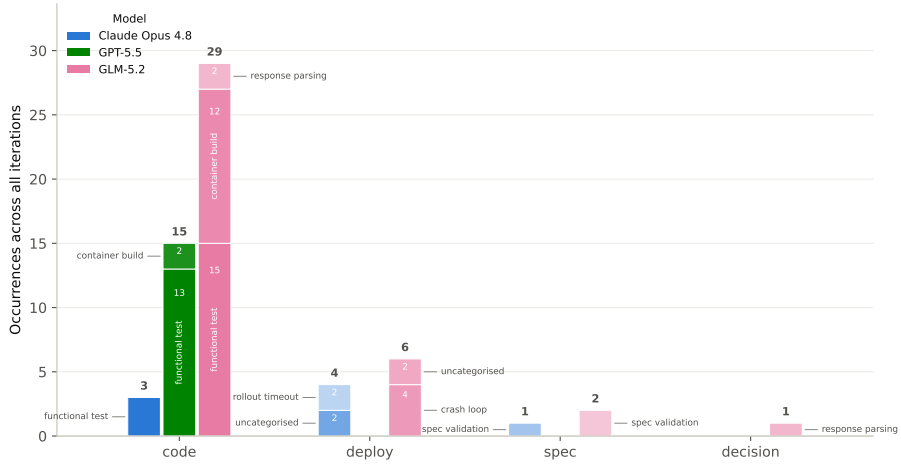



Figure 6.4: Recorded failure occurrences by phase: one bar per model within each phase, stacked and labelled by failure category, across all 63 tasks. Bold numbers are per-bar totals; smaller numbers inside a segment are that kind's own count.

**Evidence.** Figure 6.4 splits the **61** recorded failures by phase, model, and category: **47** in Code, **10** in Deploy, **3** in Spec, and **1** in Decision, so code-refinement failures alone outnumber every other phase combined. Two Code-phase categories dominate: **31** functional-test failures, split 3 / 13 / 15 across Claude Opus 4.8 / GPT-5.5 / GLM-5.2 and present for every model, and **14** container-build failures, every one of them Rust-Actix code that fails to compile and almost all from one model (12 GLM-5.2, 2 GPT-5.5, no Claude Opus 4.8). Together they account for 45 of the 61 (74%), and both are correctness problems the agent is directly responsible for rather than artifacts of the deployment path.

Failures are concentrated rather than diffuse across tasks: 32 of the 63 model–scenario–framework tasks record at least one, with 14 recording one, 11 recording two, five recording three, and two recording five. The two five-failure tasks are GLM-5.2's BranchWeave runs on Go-net-http and Rust-Actix; the former records repeated functional-test failures and the latter repeated container-build failures. This task-level concentration identifies particular model–stack trajectories, not an independent ranking of scenario difficulty.

**Five of the six recurring Deploy failures are one defect.** Six of the ten Deploy-phase failures recur rather than appearing once: crash-loop

failures four times across three GLM-5.2 tasks, and rollout timeouts twice on consecutive iterations of Claude Opus 4.8's PETSTORE×PYTHON-FLASK task. Five share one cause, readable from the recorded pod diagnostics. The generated application takes a session-scoped Postgres advisory lock in its schema-initialisation path, which runs at process start before the app serves anything, while the deployment routes the backend's write connection through PgBouncer in transaction-pooling mode. Under transaction pooling a server connection returns to the pool at the end of every transaction, so the lock and its matching unlock are not guaranteed to run on the same backend session [34]: the lock provides no mutual exclusion and can be left held on a connection already handed back. Every replica runs this path at once during a rollout, so the pods that do not reach the connection holding the lock block on it. In each of the five, roughly half the backend pods still reach `Ready` (between 6 of 12 and 16 of 30), which fits a lock held on one pooled connection and not a resource shortfall, under which no replica would start.

One specification field decides which label the failure gets. GLM-5.2's two tasks keep `statement_timeout_ms` at 30,000, so Postgres cancels the waiting statement after 30 seconds and the process exits; repeated exits are what Kubernetes records as `CrashLoopBackOff`, and both tasks log the cancellation raised from their schema-initialisation call. Claude Opus 4.8's PETSTORE task dropped the field after the baseline, so nothing cancels the wait: its initialiser runs at import time under `gunicorn`'s `preload`, before the listening socket is bound, so the affected pods never open their port, the readiness probe records `connect: connection refused`, and the rollout times out with the pods still `Running`.

The defect exists only in the combination of the two levers. In each of the three tasks a code-refinement step introduces the lock and deploys successfully at least once, a later deployment-refinement step turns it into a failure without touching the code, and a subsequent code step removes it: GLM-5.2's GO-NET-HTTP task at iterations 1, 2–3 (the first of which raises backend replicas from 12 to 16) and 4; its CLICKCOUNT task at 3, 4 (replicas 24 to 30) and 5; and Claude Opus 4.8's PETSTORE task, which carries the lock from its baseline, disables the write pooler at iteration 1, fails at 5–6 once it is re-enabled, and recovers at 7. Neither lever's edit is wrong on its own, which is why nothing upstream rejects it: the functional tests exercise the code as a single container against a direct, unpooled connection, and static deployment-specification validation examines the specification without reference to the code it will run. The same class also reaches the cluster in a form these counts never see, on a task that deploys cleanly and then fails under load without producing a single recorded entry (Section 6.5).

**The sixth failure is unrelated and deterministic.** GLM-5.2's ParcelPin×Rust-Actix code-refinement iteration 4 crashes all 50 backend pods rather than a subset. Its rewritten start-up block reads `WEB_CONCURRENCY` and falls back to zero worker threads when the variable is absent; Actix rejects a worker count of zero, so the process panics before it binds. That task's specification never sets the variable while the functional-test container always injects one, so the same binary passed 4 of 4 functional tests and could not start in the cluster. The model removed the call at the next iteration. None of the six is the resource-infeasibility case static validation is designed to catch; all six are couplings between generated code and deployment configuration that become visible only when the two are combined at cluster scale.

**The four remaining Deploy failures are an operating artifact.** The other four, recorded as uncategorised, are the same error on two tasks: Claude Opus 4.8's Recipes×Python-Flask and GLM-5.2's Petstore×Rust-Actix, iterations 5–6 in both. A deployment-refinement iteration reuses the image an earlier iteration built rather than rebuilding it, and the campaign had been restarted after those images were deleted from the build host, so the reused reference no longer resolved; both tasks recover at the next code-refinement step, which rebuilds from source. These are an operating error on our part rather than a property of the framework or of any model, and no claim in this chapter rests on them: they cost part of the fixed iteration budget but leave every measurement that was taken intact.

**All three Spec-phase rejections were correct.** Every Spec-phase failure is static validation refusing a proposal before it reached the cluster, and all three are the same error: the specification pins the entire database tier onto a single worker node whose combined CPU *requests* exceed what that node can commit (≈28,800m, 90% of its 32-core allocatable capacity, after the reserve the check holds back). Claude Opus 4.8's TransitPulse×Go-net-http task (iteration 1) asked for 38,000m across three database pods, GLM-5.2's TransitPulse×Python-Flask task (iteration 2) for 32,000m across four, and GLM-5.2's Recipes×Rust-Actix task (iteration 8) for 36,000m across three. Requests are reserved up front rather than estimated from use, so in none of the three could the pods all have been placed. The Spec stage therefore contributed nothing but correct rejections. The converse also holds: none of this dataset's Deploy-phase failures is a resource-infeasibility case this check would have been expected to catch, and on every occasion a proposal actually was infeasible, it was caught before reaching the cluster.

**Two of the three response-parsing failures are a delivery problem.** The last three failures are response-parsing failures, all GLM-5.2's, and they cover two different problems. One is a genuine format failure: on Pet-

STORE×GO-NET-HTTP (iteration 1) the model never produced the required decision block in any of its three attempts, so no lever was selected. This is the one refinement iteration of the 630 in which that happened, which is why Table 6.6 accounts for 629. The other two, both on SPLITNEST (GO-NET-HTTP, iteration 6; RUST-ACTIX, iteration 10), are truncation rather than malformation: each response stops mid-statement with no closing code fence. Both ended well below the 128,000-token ceiling the harness requested and the provider reported no length-limited stop, so their cause cannot be settled from what was logged, though it points away from a budget set too low on our side and towards an early stop upstream. They are a delivery problem rather than evidence that the model cannot follow the required output format.

**Answer. The dominant genuine failure mode is code correctness: functional-test and container-build failures account for 45 of 61 recorded failures (74%).** Five of the six recurring Deploy failures arise from one code–deployment interaction, while static validation rejects all three observed resource-infeasible specifications before deployment. This taxonomy records where the harness stops before benchmarking; a completed benchmark that yields no sustained estimate is a separate outcome rather than a pipeline failure.

## 6.5 Case Studies: Code–Deployment Dynamics and Individual Trajectories

The results so far compress each task into a handful of numbers. This section reads three trajectories in full. The first two run on the same task, where a defect that no recorded failure captures decides the outcome; the third follows a healthy trajectory through a coherent sequence of deployment-refinement decisions under a ceiling-censored measurement.

**Case study: a connection-pooling bug invisible to single-container evaluation.** GLM-5.2's RECIPES×GO-NET-HTTP baseline reaches the bench but establishes no sustained estimate: warm-up fails its health checks and explore never starts. Postgres logs from that run show the deployment producing 919 errors during the load window, dominated by two error classes: `unnamed prepared statement does not exist` and `bind message supplies N parameters, but prepared statement requires M`. This is the code–deployment coupling behind RQ3's Deploy failures (Section 6.4), with prepared-statement tracking in place of the advisory lock, and like those instances it is invisible to BAXBENCH's original single-container functional tests, which use a direct, unpooled connection. What differs is where the class lands. RQ3's instances break the rollout and are recorded as Deploy failures, whereas

this one deploys cleanly and fails only under load, so the task completes every iteration as a harness-level success and contributes nothing to the taxonomy. Iteration 1 replaces the affected parameterised SQL path; the prepared-statement errors disappear, so the artifact evidence indicates that it addresses the initial incompatibility. It still establishes no sustained estimate because connection-pool exhaustion and timeouts become the next dominant symptoms. Iteration 2 adds caching and query-path optimisations and then establishes 21,542.3 req/s. The sequence is evidence of successive bottleneck exposure: one code change removes the initial error class, after which another code change addresses the newly visible bottleneck. It does not establish that deployment refinement could not have helped in the same state.

**Case study: a plausible deployment decision that backfires.** On the same task, iteration 3's Decision stage reasons as follows (quoted verbatim from its recorded decision):[1]

> "The code is now well-optimized (0% failures, 19K/s goodput) and the bottleneck is purely infrastructure: Postgres hits 200/200 connections with PgBouncer `cl_waiting` reaching 311, and db-primary CPU saturates at 16.7 cores. [...] We should add database replicas, enable a read pooler, increase Postgres `max_connections` and PgBouncer `default_pool_size`, and potentially reduce over-provisioned backend CPU to free cluster budget for more DB resources."

The reasoning is a coherent read of the previous iteration's diagnostics (Section 4.5.5), and the resulting deployment specification does exactly what it describes: database replicas $1 \rightarrow 3$, `max_connections` $200 \rightarrow 300$, backend CPU request 2000m $\rightarrow$ 500m. The resulting deployment, however, establishes no sustained estimate. The next iteration (4, code) recovers to 34,854.6 req/s *without* reverting the topology change, so the regression is not explained by the added replicas alone. The simultaneous 4× backend CPU cut is one plausible contributor, but three fields changed together and the experiment cannot isolate any single cause. The task goes on to alternate lever choices through the rest of its budget, peaking at 44,850.9 req/s at iteration 7 before settling at 40,414.0 req/s by iteration 10, itself not a monotonic climb: the first of two consecutive code-refinement steps (iteration 8) gives back roughly a fifth of the peak, the second (iteration 9) recovers about half of what was lost, and the closing deployment-refinement step adds 20 req/s to that, leaving the trajectory below its own peak. Unlike Claude Opus 4.8's BranchWeave×Rust-Actix task (Section 6.2), this trajectory has enough budget left after its own backfire to recover before running out. Because this

[1]The log is quoted as written at generation time, so the model's own figures here (the ∼19,000 req/s goodput and the connection and CPU readings) are its original-network measurements; the goodput values in the surrounding narrative are the internal-network re-measurements of the same iterations that form this chapter's ground truth, which is why the two differ slightly.

iteration changed three specification fields at once, the observed regression cannot be attributed to any one of them. It also belongs to the model–lever group with the most observed losses of an existing sustained estimate: 14 of GLM-5.2's 92 completed deployment-refinement steps (Section 6.3).

**Case study: a coherent deployment specification trajectory under ceiling censoring.** GPT-5.5 on ClickCount×Go-net-http produces a large gain (17.0×, baseline 5,725.9 req/s to best 97,137.7 req/s), and its deployment-refinement steps move in a consistent direction across the run rather than oscillating:

Table 6.9: Deployment specification trajectory for GPT-5.5, ClickCount, Go-net-http (selected iterations).

| Iter. | Lever | Goodput (req/s) | Replicas | CPU req. | DB repl. | DB max_conn |
|---|---|---|---|---|---|---|
| 0 | baseline | 5,725.9 | 48 | 1000m | 3 | 500 |
| 6 | code | 96,363.2 | 48 | 1000m | 3 | 500 |
| 7 | deployment | 96,738.6 | 192 | 350m | 1 | 300 |
| 8 | code | 96,494.9 | 192 | 350m | 1 | 300 |
| 9 | deployment | 96,435.7 | 320 | 250m | 1 | 500 |
| 10 | code | 97,137.7 | 320 | 250m | 1 | 500 |

By iteration 6 a code-refinement step has already lifted goodput to roughly 96,000 req/s, near the profile's user ceiling and therefore right-censored (Section 6.1). The deployment-refinement steps that follow coincide with a nearly flat measured outcome while changing several deployment knobs. Between iterations 6 and 10, the agent roughly *sextuples* replica count (48 → 320) while *quartering* per-pod CPU request (1000m → 250m), trading a few large pods for many small ones, and drops database replicas from 3 to 1. The flat, ceiling-censored measurements do not establish that those settings improved goodput or that the original read replicas were unnecessary. GLM-5.2 reaches almost exactly the same observed level on its own ClickCount×Go-net-http task (96,281.8 req/s, and the single largest gain in this dataset at 66.7×), but not by the same route: its replica count ends where it started (50), it halves rather than quarters per-pod CPU (1000m → 500m), and it never provisioned the database read replicas that GPT-5.5 dropped. Its deployment-refinement steps oscillate across the run (50 → 25 → 50 → 20 → 50 replicas) instead of settling on a direction. The two models therefore arrive at the same observed level on the same task through visibly different deployment settings, while the ceiling prevents a performance comparison among those settings.

## 6.6 Additional Analysis

The three research questions cover improvement, refinement-lever outcomes, and pipeline failures. This section collects four secondary, descriptive views of the same fixed grid: variation across technology stacks, variation across scenarios, semantic-correctness exposure under replicated read routing, and LLM cost. Each subsection retains the headline evidence needed for interpretation; Appendix D provides the detailed stack, scenario, and cost breakdowns.

### 6.6.1 Technology-Stack Variation

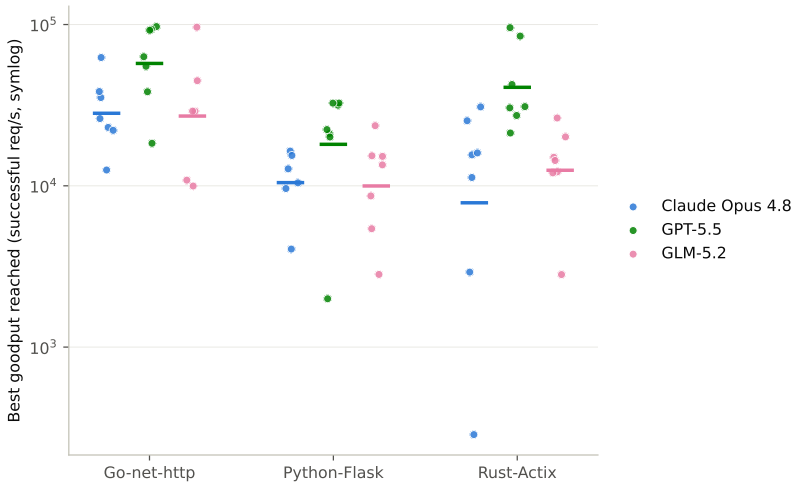



Figure 6.5: Best sustained goodput reached per framework (symlog $y$-axis), split by model. Each dot is one scenario task; the horizontal bar is the per-model geometric mean for that framework. Tasks that never establish a sustained estimate are excluded from the bar rather than plotted as a measured sustained goodput of zero.

Go-net-http has the highest geometric-mean best sustained goodput for all three models in this grid. Rust-Actix ranks second for GPT-5.5 and GLM-5.2, ahead of Python-Flask. Claude Opus 4.8 is the exception: two low positive Rust-Actix outcomes, including the table minimum of 287 req/s, pull its geometric mean below Python-Flask. By median, however, all three models share the same ordering: Go-net-http first, Rust-Actix second, and Python-Flask third. This comparison describes the complete evaluated stacks—language, framework, server defaults, generated code, and refinement history together—rather than inherent framework ceilings. Appendix D.2 gives the task coverage and outlier analysis behind the aggregate ordering.

### 6.6.2 Scenario-Level Variation in Refinement Outcomes

Table 6.10: Per-scenario outcome profiles across nine tasks (three models × three frameworks). *Baseline*, *ever*, and *final* count tasks with a sustained estimate at the corresponding point in the trajectory. Failures are recorded pre-benchmark pipeline failures (Section 6.4). Median gain is the median best-over-first-positive-estimate ratio ($n = 8$ for ClickCount and TransitPulse, $n = 9$ otherwise).

| Scenario | Baseline /9 | Ever /9 | Final /9 | Failures | Median gain |
|---|---|---|---|---|---|
| ClickCount | 3 | 8 | 8 | 7 | 8.02× |
| Recipes | 6 | 9 | 8 | 5 | 3.32× |
| BranchWeave | 7 | 9 | 5 | 16 | 1.69× |
| Petstore | 2 | 9 | 9 | 14 | 1.06× |
| ParcelPin | 5 | 9 | 6 | 7 | 1.84× |
| SplitNest | 6 | 9 | 8 | 7 | 2.04× |
| TransitPulse | 4 | 8 | 8 | 5 | 3.41× |

**Scenario-level variation in this fixed grid is multidimensional: initial viability, recovery, terminal reliability, recorded failures, and refinement headroom do not move together.** For example, Petstore and ClickCount both begin with few sustained baselines but show opposite headroom profiles, while BranchWeave begins strongly and finishes with an estimate in only five of nine tasks. The seven purposively selected scenarios support description, not a population taxonomy or causal difficulty ordering. Appendix D.3 examines these contrasts and the static endpoint and test-count proxies in detail.

### 6.6.3 Replicated Read Routing and Semantic Correctness

Replicated read routing is more common among the candidates selected by the best-of-budget comparison: 40 of the 61 selected best candidates deploy more than one database replica, against 17 of the 61 corresponding first candidates. Across the 373 benchmarked iterations that run database replicas, 372 route application reads through the framework-provided `DB_READ_HOST`.

Among runs whose read and write failure rates both remain below the load profile's failure threshold, the mean failure rate on read operations is 0.062% with replicated read routing and 0.031% with a single primary. These status-code-based figures are descriptive and cannot detect a successful response containing stale content; they therefore establish neither semantic equivalence nor a causal effect of replicated routing.

### 6.6.4 What Refinement Costs

Every result so far is stated in goodput alone, but refinement uses LLM calls, so its accounted LLM/API spend is an outcome in its own right. This section reports the campaign's recorded LLM spend and where the loop incurred it; it does not include cluster or infrastructure cost, or human effort. Spend is read from each task's recorded cost ledger. For GPT-5.5 and Claude Opus 4.8, each call is estimated from provider-reported token buckets and the documented list price; for GLM-5.2, the ledger retains OpenRouter's reported per-request charge. These figures therefore describe the evaluated access paths and pricing rules, not a general provider-price comparison. Appendix D carries the finer splits. Across the 63 tasks, accounted LLM spend totals $287.73, an average of $4.57 per task for one iteration 0–10 trajectory.

Table 6.11: Accounted LLM spend per model over its 21 tasks, against what that spend reached. *$ to first estimate* is the mean cumulative spend up to and including the iteration that first produced a sustained estimate, among tasks that ultimately reached one (Table D.8 gives the denominators and splits it by whether the baseline was already sustained). *Best goodput* is the geometric-mean best achieved goodput, repeated from Table 6.2.

| Model | Total | $ / task | $ to first estimate | Best goodput (req/s) |
|---|---|---|---|---|
| GPT-5.5 | $125.63 | $5.98 | $1.95 | 34,853 |
| Claude Opus 4.8 | $133.61 | $6.36 | $1.41 | 13,382 |
| GLM-5.2 | $28.49 | $1.36 | $0.29 | 14,560 |

**Spend and outcome are decoupled.** Cost rank does not match achieved-goodput rank. Because every model attempted the same 21 tasks, paired comparisons are possible; each *median ratio* below is the median taskwise numerator/denominator ratio of best sustained goodput where both models reached an estimate. GPT-5.5 costs about 6% less per task than Claude Opus 4.8 and reaches a higher best on 18 of 21 tasks, at a median GPT/Claude ratio of 1.99×. For a fifth of Claude Opus 4.8's per-task spend, GLM-5.2 finishes ahead on 8 of 21, at a median GLM/Claude ratio of 0.90×. GPT-5.5 leads GLM-5.2 on 20 of 21 at a median GPT/GLM ratio of 1.70×, for 4.4 times the spend.

**Most spend comes after the first estimate.** Among tasks that ultimately reach a sustained estimate, reaching it costs $0.29 to $1.95 per task on average—a minority of full per-task spend. The remaining cost accumulates across the fixed refinement budget; nonrecovering tasks remain in the total-spend figures but not in this conditional mean. Code-refinement steps cost two to three times as much as deployment-refinement steps for

every model. Provider, recovery, lever, framework, scenario, and iteration breakdowns appear in Appendix D.4.

# Chapter 7

# Discussion

## 7.1 From Candidate Discovery to Dependable Optimisation

The central result supports a bounded candidate-discovery claim. Among the 61 evaluated trajectories for which a sustained-goodput comparison is defined, 58 contain a later candidate with a higher estimate (Section 6.2). The held-out re-measurement of the selected candidates makes it unlikely that this pattern is explained by choosing the largest of several noisy measurements. It supports interpreting the pattern as reflecting differences between artifacts produced along the trajectories, rather than measurement selection alone. This is the principal sense in which the complete IterBench loop succeeds: given its fixed search budget, it usually finds a deployed candidate that measures better than the first candidate for which the profile establishes a sustained level.

That result is not a causal estimate of the value of runtime feedback. The experiment evaluates one continuing process in which model priors, diagnostics, conversation history, and repeated generation operate together. It contains no matched trajectory that withholds the diagnostics, resets the conversation, or draws an equal number of independent candidates. Held-out benchmarking addresses measurement optimism after a candidate has been selected; it does not separate the contributions of the mechanisms that produced that candidate. The defensible interpretation is consequently that the complete conversational loop discovers better candidates on this grid, not that feedback alone causes the observed improvement.

Candidate discovery is also different from dependable terminal optimisation. The trajectories frequently move from an unsustained baseline to a candidate for which the profile establishes a sustained estimate, but they do not improve monotonically and can end after losing an estimate established earlier. A best-of-budget analysis asks whether the search encountered a strong candidate; a final-iteration analysis asks whether the system preserved

or recovered one. Both are useful, but they describe different system properties. The former is appropriate for a search system that retains validated artifacts, whereas the latter exposes the current framework's lack of an explicit acceptance and rollback policy. Extending the iteration budget could provide more opportunities to recover from a late regression, but it would not guarantee a return to a known-good state. Dependable optimisation therefore requires the orchestrator to retain and, when necessary, redeploy its best validated candidate rather than equating the last proposal with the final product.

The secondary model, scenario, and technology-stack comparisons should be read within the same boundary. They reveal substantial heterogeneity in initial viability, recovery, terminal reliability, and refinement headroom, but the observed orderings combine generated-code quality, provider configuration, framework defaults, and refinement history (Sections 6.6.2 and 6.6.1). They are useful descriptions of where this particular loop succeeded or struggled; they do not establish a general model ranking, an inherent framework ceiling, or a taxonomy of backend difficulty.

## 7.2 Code and Deployment as Coupled Optimisation Surfaces

The selected lever classes exhibit different observed profiles. Code refinement fails before benchmarking more often than deployment refinement in this campaign (14.6% versus 4.0%), yet the successfully benchmarked code steps include the largest recoveries and positive changes (Sections 6.3 and 6.4). A code edit can remove an application defect that caps every deployment of that artifact; replica, resource, and placement changes cannot directly do so. Deployment refinement more often survives the upstream pipeline and usually costs less to propose, but reaching the benchmark is not equivalent to preserving sustained performance. Among completed steps, selected deployment refinements proportionally more often move a previously sustained candidate to a no-estimate outcome. Pre-benchmark correctness and post-deployment performance are therefore separate notions of reliability.

These profiles are descriptive associations rather than lever effects. The Decision stage selects a lever in response to the current trajectory, so code and deployment steps occur in systematically different states and share artifacts, diagnostics, and conversation history. The larger positive tail among selected code steps does not show what deployment refinement would have achieved in those same states, just as the lower upstream failure rate of selected deployment steps does not show that deployment refinement is intrinsically safer. Randomised or matched lever assignments would be required for that comparison.

The individual trajectories nevertheless clarify why both artifact classes

must remain in scope. Some bottlenecks reside primarily in application logic and become visible only under concurrency. Others emerge from the interaction between otherwise plausible code and deployment choices, such as assumptions about connection or session behaviour that cease to hold behind a pooler. Conversely, a coherent deployment proposal can modify several resource and topology fields at once and still reduce the measured outcome (Section 6.5). Neither isolated code testing nor static validation of a deployment specification fully characterises the pair that will execute in the cluster. The relevant optimisation surface is the deployed system, even when the experimental design changes only one artifact class at a time.

## 7.3 Relation to Prior Work and Practical Implications

Many backend-generation benchmarks test functional and security correctness in isolated execution (Section 3.1). Cluster autotuning systems, by contrast, often begin with an existing application and optimise its configuration (Section 3.3). IterBench connects these problem settings by allowing one loop to revise application code or deployment configuration and then evaluating their combination under sustained concurrent load. This is broader in artifact scope than either isolated code evaluation or configuration-only tuning, but narrower in claim than autonomous production optimisation. In particular, the experiment does not compare the LLM policy with a classical tuner, an autoscaler, or a fixed search policy, and therefore provides no evidence that it outperforms them.

The first practical implication is to separate candidate generation from candidate acceptance. An exploratory agent should be free to propose risky changes, but a production-oriented orchestrator should preserve the best validated code–specification pair, reject sufficiently large regressions, and support rollback after a sustained estimate is lost. This converts the observed best-achieved capability into a safer terminal policy without assuming that the agent will rediscover an earlier artifact from conversational context.

Second, keeping code and deployment changes mutually exclusive per iteration remains valuable for artifact-class attribution (Section 4.4). It identifies which lineage changed even though adaptive selection prevents causal attribution. The case studies also show the limit of that design: a deployment refinement can alter several schema fields simultaneously. Restricting high-risk trials to one field, or following a multi-field proposal with controlled component trials, would make regressions more diagnosable without removing the broader deployment search space.

Third, validation should follow the artifact pair into its deployed topology. The current functional gate exercises a single application container against a standalone database, while the eventual deployment can add poolers,

replicas, caches, and multiple application instances. Re-running functional and semantic checks against the rendered topology would catch interactions that neither the local code gate nor static specification validation can see. Those checks should test state transitions, freshness, and cache behaviour as well as HTTP status, because a successful response does not by itself establish that a refined system preserves the scenario's semantics.

Finally, a stronger comparative evaluation should pair the LLM loop with feedback ablations and established configuration baselines. Fixed or random lever policies would help identify the value of adaptive selection; fresh-context and no-diagnostic conditions would separate runtime evidence from repeated sampling; and HPA/VPA-style mechanisms or classical search would establish whether LLM-guided proposals add value beyond conventional Kubernetes tuning. The reported LLM expenditure shows that generation cost can be tracked alongside capability, but a practical comparison must also include cluster occupancy, operator effort, and the cost of failed trials.

## 7.4 Threats to Validity

The evaluation supports descriptive claims about the complete IterBench loop on its fixed grid. The following threats delimit the construct being measured, the causal interpretations available, the reliability of the observed summaries, and their generalisability.

### 7.4.1 Construct Validity

Sustained goodput is useful for comparing candidates, but it does not fully describe service quality or a deployment's physical capacity. The objective does not directly include tail latency, resource efficiency, cost per served request, security, or burst performance. Some of these signals are recorded as diagnostics, but they do not affect the objective. The best-of-budget summary instead measures the loop's refinement capability within the fixed budget: whether refinement produces an improved candidate at any point in the trajectory. It does not measure the quality of the final candidate. For this summary to describe a usable result, the system must retain and return the candidate selected as best.

The main additional limitation concerns semantic correctness. Every new version of the generated code must pass functional tests locally before deployment. Those tests use one application container and a standalone database. They are not rerun against the full deployed topology, which can include multiple application instances, database replicas, connection poolers, and caches. Section 6.6.3 explains why this gap matters by examining replicated read routing. Goodput excludes responses with unsuccessful HTTP status codes, but it can still count a successful response that contains stale content. Reported goodput therefore measures successful-response

throughput in the deployed topology. It does not verify that the deployed system preserves the behaviour checked by the local functional tests.

The cost measure is also narrow. It records LLM expenditure but excludes cluster use, load-generation infrastructure, storage, repeated verification, recovery work, and operator time. The values also depend on provider prices, cache behaviour, and routing at the time of the study. They therefore describe only the recorded LLM cost of this campaign. They are not serving costs and cannot, by themselves, establish a return on investment (Section 6.6.4).

### 7.4.2 Internal and Causal Validity

The design contains no matched condition that removes runtime feedback, resets conversation state, fixes or randomises lever choice, or samples an equivalent number of independent candidates without diagnostics. RQ1 therefore identifies what the complete process discovers, not the causal contribution of any one component. RQ2 compares outcomes following adaptively selected levers, not independent treatment assignments. Shared lineage and conversational context also make iterations within a trajectory dependent. These constraints do not invalidate the recorded transitions, but they rule out counterfactual claims about feedback or the unselected lever.

The reported measurements are network-path-consistent because affected candidates were re-benchmarked on the experiment LAN (Section 5.3). Some of those candidates had already been generated from diagnostics collected through the earlier network path, however. Re-measurement makes their outcomes comparable but cannot recreate a uniform feedback history. This residual difference belongs to the treatment that produced the candidates and should be removed in a replicated campaign.

### 7.4.3 Reliability and Statistical Conclusion Validity

Each task contributes one generated trajectory. Within-task generation variance is therefore unknown, and aggregating heterogeneous task cells does not turn them into independent samples from a workload population. The reported model, stack, and lever summaries are point descriptions of the grid rather than estimates with population-level uncertainty.

Independent re-measurement indicates that positive estimates are usually close enough to support the broad comparisons, while also showing that the boundary between an estimate and no estimate is less stable for lower-goodput candidates (Section 6.1). Recovery and loss counts should accordingly be read as outcomes under this particular profile, not exact classifications of physical capacity. Ceiling-bound runs provide lower bounds rather than measured saturation points, and the small number of wall-clock-bound runs are truncated measurements. These effects qualify task, model, and stack summaries whose selected values lie at either boundary.

The fixed refinement budget is part of the estimand rather than a neutral implementation detail. A different budget could alter the best candidate found, the terminal outcome, and the mix of selected levers. Maximum selection is also optimistic by construction, although the held-out comparison indicates that measurement selection alone does not explain the observed improvement (Section 6.2). Neither a larger budget nor repeated measurement would solve the separate absence of automatic best-candidate retention.

### 7.4.4 External and Operational Validity

The scenarios were purposively selected to span workload shapes, not sampled to represent backend services statistically. They are database-backed and do not cover important compute-, memory-, coordination-, or burst-dominated services. All candidates also run on one cluster topology and hardware profile. The observed refinement profiles may therefore depend on the database, connection, network, and capacity bottlenecks made salient by this particular environment; the same deployment decisions need not be feasible or effective elsewhere.

Model identity is bundled with provider access, reasoning interface, decoding controls, routing, and snapshot availability (Section 5.1). Scenario provenance overlaps with evaluation as well, because evaluated model families contributed to construction or calibration of part of the scenario set (Section 5.2). Model comparisons are therefore neither provider-independent treatments nor provenance-independent capability estimates.

Finally, sequential execution and namespace cleanup prevented intentional competition between evaluation cells, but did not reset node-level image and filesystem caches or operating-system state (Section 5.4). The recorded image-lifecycle failures show that campaign operations can also consume part of a fixed iteration budget (Section 6.4). Explicit image provenance, independently verified cleanup, and repeated campaigns on freshly prepared infrastructure would make those operational assumptions auditable rather than implicit.

# Chapter 8

# Conclusion

## 8.1 Answer and Contribution

Code that passes functional checks in isolation need not remain effective as a replicated service under concurrent load. This thesis therefore asked to what extent an LLM agent can improve the sustained goodput of a generated backend on Kubernetes, under fixed cluster and iteration budgets, by refining either its code or deployment configuration. To study that question, it introduced IterBench: a framework that maintains validated artifact lineages, routes failures to the responsible stage, and places code and deployment refinement under the same runtime objective and feedback loop.

**Among the 61 evaluated tasks for which a sustained-goodput comparison is defined, the complete loop finds a higher-estimated deployed candidate in 58 cases.** The geometric-mean best-over-first ratio is 2.7×, and the loop finds a sustained candidate after 28 of 30 unsustained baselines. The result is not equivalent to dependable terminal optimisation: 46 of those 61 tasks finish above their first estimate, while nine finish without an estimate. The evidence thus supports a best-achieved capability claim about the loop under its fixed budget, not a claim of reliable terminal optimisation.

The two refinement levers exhibit different observed profiles. Code refinement is the principal source of pre-benchmark failure, but it also contains the largest recoveries and positive gains. Deployment refinement reaches measurement more reliably, yet more often moves a previously sustained candidate to a no-estimate outcome once benchmarked (Section 6.3). Correctness remains the dominant recorded barrier overall: functional-test and build failures outweigh failures at later pipeline stages, while static specification validation cannot reject defects that arise only when otherwise valid code and deployment choices interact (Section 6.4). IterBench's contribution is therefore not only a score, but an experimental setting in which those transitions remain tied to validated artifacts and regressions remain visible

rather than being hidden by a single final result.

Taken together, the results show that generated code and deployment specification should be evaluated as a coupled deployed system. IterBench makes that coupling measurable: within the evaluated grid, it finds substantially better candidates in most comparable trajectories while exposing the regressions, correctness failures, and evidential boundaries that a credible account of autonomous optimisation must preserve.

## 8.2 Scope of the Claims

The full limitations are consolidated in Section 7.4. Each of the 63 tasks contributes one trajectory ($n = 1$) from a purposive, database-heavy grid, and all candidates run on one cluster topology. The design includes no matched no-feedback, fresh-context, random-policy, or classical-tuner control, so it evaluates the complete loop rather than the causal contribution of feedback, memory, generation, or lever choice. Its primary result selects the best profile-dependent estimate within a fixed budget and is consequently more optimistic than terminal reliability. Moreover, functional tests run against a single-container topology rather than the deployed configuration; successful-response goodput therefore does not verify semantic equivalence when replicas or caches are introduced. The claims describe what IterBench discovers on this fixed grid, not population-level model or stack rankings, performance across cluster scales, or guaranteed autonomous production optimisation.

## 8.3 Future Work

**Repeat trajectories and isolate mechanisms.** Repeated trajectories per task would quantify generation variance and attach uncertainty to model, stack, and lever comparisons. Matched ablations should remove runtime feedback, reset conversation state, and randomise or fix lever selection; comparisons with independent sampling, classical search, and autoscaling baselines would then separate the value of runtime evidence from model priors and repeated generation.

**Strengthen deployed-pair validation.** The existing warm-up and readiness gate already rejects candidates that do not become healthy in the cluster, but it does not establish that the resulting code–deployment pair remains functionally correct in the cluster. Future work should add post-deployment end-to-end checks that validate the semantics of returned responses. Extending this validation to distributed settings could enable further improvements.

**Expand the controllable service surface.** Although the deployment specification exposes most relevant infrastructure levers, it leaves service-internal configuration outside the agent's action space. Future work could allow coordinated control over application and runtime settings such as connection pools, prepared-statement behaviour, worker counts, and cache or storage parameters, alongside code and deployment specification changes. This would test whether the observed coupling extends beyond the current schema. Because it enlarges the action space and can create interactions hidden from static validation, it should be paired with the stronger deployed-service gates and richer lineage and attribution records described above.

**Broaden the evaluation scenarios.** Future grids should add compute-, memory-, coordination-, and burst-bound workloads, expand the deployment space beyond relational CRUD services, and repeat matched tasks across cluster sizes. Objectives combining goodput with tail latency, resource efficiency, or serving cost would test whether the observed improvements survive a broader definition of performance.

**Strengthen the measurement instrument.** The current load-profile setup would benefit from repeated settle windows, a higher load ceiling with explicit censoring labels, and elapsed-time bounds on every phase. These changes would address the observed ceiling effects and phase-control failures while making the resulting measurements more robust.

# Bibliography


[1] Jason Ansel, Shoaib Kamil, Kalyan Veeramachaneni, Jonathan Ragan-Kelley, Jeffrey Bosboom, Una-May O'Reilly, and Saman Amarasinghe. OpenTuner: An extensible framework for program autotuning. In *Proceedings of PACT*, 2014.

[2] Anthropic. Model IDs and versioning. `https://platform.claude.com/docs/en/about-claude/models/model-ids-and-versions`. Accessed: 2026-08-25.

[3] Anthropic. Models overview. `https://platform.claude.com/docs/en/about-claude/models/overview`. Accessed: 2026-08-23.

[4] Anthropic. Pricing. `https://platform.claude.com/docs/en/about-claude/pricing`. Accessed: 2026-08-23.

[5] Anysphere, Inc. Cursor: Models and pricing. `https://cursor.com/docs/models-and-pricing`. Accessed: 2026-08-26.

[6] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

[7] Brendan Burns, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. Borg, omega, and Kubernetes: Lessons learned from three container-management systems over a decade. *ACM Queue*, 14(1):70–93, 2016.

[8] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[9] Pouya Hamadanian, Pantea Karimi, Arash Nasr-Esfahany, Kimia Noorbakhsh, Joseph F. Chandler, Ali ParandehGheibi, Mohammad Alizadeh, and Hari Balakrishnan. Glia: A human-inspired AI for automated

systems design and optimization. *arXiv preprint arXiv:2510.27176*, 2025.

[10] Xinyi He, Qian Liu, Mingzhe Du, Lin Yan, Zhijie Fan, Yiming Huang, Yin Zheng, Zejian Yuan, and Zejun Ma. SWE-Perf: Can language models optimize code performance on real-world repositories? In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, 2026.

[11] Prithwish Jana, Sam Davidson, Bhavana Bhasker, Andrey Kan, Anoop Deoras, and Laurent Callot. TerraFormer: Automated infrastructure-as-code with LLMs fine-tuned via policy-guided verifier feedback. In *Proceedings of ICSE*, 2026.

[12] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *Proceedings of ICLR*, 2024.

[13] Keisuke Kamahori, Shihang Li, Simon Peter, and Baris Kasikci. VibeServe: Can AI agents build bespoke LLM serving systems? *arXiv preprint arXiv:2605.06068*, 2026.

[14] Jeffrey O. Kephart and David M. Chess. The vision of autonomic computing. *Computer*, 36(1):41–50, 2003.

[15] Kubernetes Authors. Deployments. `https://kubernetes.io/docs/concepts/workloads/controllers/deployment/`. Accessed: 2026-08-23.

[16] Kubernetes Authors. Horizontal pod autoscaler. `https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/`. Accessed: 2026-08-23.

[17] Kubernetes Authors. Kubernetes. `https://kubernetes.io`. Accessed: 2026-08-23.

[18] Kubernetes Authors. Liveness, readiness, and startup probes. `https://kubernetes.io/docs/concepts/workloads/pods/probes/`. Accessed: 2026-08-23.

[19] Kubernetes Authors. Namespaces. `https://kubernetes.io/docs/concepts/overview/working-with-objects/namespaces/`. Accessed: 2026-08-23.

[20] Kubernetes Authors. Objects in Kubernetes. `https://kubernetes.io/docs/concepts/overview/working-with-objects/`. Accessed: 2026-08-23.

[21] Kubernetes Authors. Pods. https://kubernetes.io/docs/concepts/workloads/pods/. Accessed: 2026-08-23.

[22] Kubernetes Authors. Resource management for pods and containers. https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/. Accessed: 2026-08-23.

[23] Kubernetes Authors. Service. https://kubernetes.io/docs/concepts/services-networking/service/. Accessed: 2026-08-23.

[24] Shu Liu, Alexander Krentsel, Shubham Agarwal, Mert Cemri, Ziming Mao, Soujanya Ponnapalli, Alexandros G. Dimakis, Sylvia Ratnasamy, Matei Zaharia, Aditya Parameswaran, and Ion Stoica. The time is here for just-in-time systems: Challenges and opportunities. *arXiv preprint arXiv:2605.24096*, 2026.

[25] Locust Authors. Locust: A modern load testing framework. https://locust.io. Accessed: 2026-08-23.

[26] Locust Authors. Writing a locustfile: User wait time. https://docs.locust.io/en/stable/writing-a-locustfile.html. Accessed: 2026-08-23.

[27] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. In *Proceedings of NeurIPS*, 2023.

[28] Hongzi Mao, Malte Schwarzkopf Venkata, Ning He, Srikanth Kandula, and Mohammad Alizadeh. Learning scheduling algorithms for data processing clusters. In *Proceedings of NSDI*, 2019.

[29] OpenAI. GPT-5.5 Model. https://developers.openai.com/api/docs/models/gpt-5.5. Accessed: 2026-08-23.

[30] OpenAI. Model guidance: Using GPT-5.5. https://developers.openai.com/api/docs/guides/latest-model?model=gpt-5.5. Accessed: 2026-08-23.

[31] OpenAI. Model guidance: Using GPT-5.6. https://developers.openai.com/api/docs/guides/latest-model. Accessed: 2026-08-25.

[32] OpenRouter. Prompt caching. https://openrouter.ai/docs/guides/best-practices/prompt-caching. Accessed: 2026-08-23.

[33] Jinjun Peng, Leyi Cui, Kele Huang, Junfeng Yang, and Baishakhi Ray. CWEval: Outcome-driven evaluation on functionality and security of LLM code generation. *arXiv preprint arXiv:2501.08200*, 2025.

[34] PgBouncer Authors. PgBouncer features. `https://www.pgbouncer.org/features.html`. Accessed: 2026-08-23.

[35] Gabriel Poesia, Oleksandr Polozov, Vu Le, Ashish Tiwari, Gustavo Soares, Christopher Meek, and Sumit Gulwani. Synchromesh: Reliable code generation from pre-trained language models. *arXiv preprint arXiv:2201.11227*, 2022.

[36] PostgreSQL Global Development Group. PostgreSQL 17 Documentation: Connections and authentication. `https://www.postgresql.org/docs/17/runtime-config-connection.html`. Accessed: 2026-08-23.

[37] PostgreSQL Global Development Group. PostgreSQL 17 Documentation: How connections are established. `https://www.postgresql.org/docs/17/connect-estab.html`. Accessed: 2026-08-23.

[38] PostgreSQL Global Development Group. PostgreSQL 17 Documentation: Log-shipping standby servers. `https://www.postgresql.org/docs/17/warm-standby.html`. Accessed: 2026-08-23.

[39] PostgreSQL Global Development Group. PostgreSQL 17 Documentation: Server configuration. `https://www.postgresql.org/docs/17/runtime-config.html`. Accessed: 2026-08-23.

[40] Haoran Qiu, Subho S. Banerjee, Saurabh Jha, Zbigniew T. Kalbarczyk, and Ravishankar K. Iyer. FIRM: An intelligent fine-grained resource management framework for SLO-oriented microservices. In *Proceedings of OSDI*, 2020.

[41] Krzysztof Rządca, Paweł Findeisen, Jacek Świderski, Przemysław Zych, Przemysław Broniek, Jarek Kusmierek, Paweł Nowak, Beata Strack, Piotr Witusowski, Steven Hand, and John Wilkes. Autopilot: Workload autoscaling at Google. In *Proceedings of EuroSys*, 2020.

[42] Bianca Schroeder, Adam Wierman, and Mor Harchol-Balter. Open versus closed: A cautionary tale. In *Proceedings of NSDI*, pages 239–252, 2006.

[43] Chihao Shen, Connor Dilgren, Purva Chiniya, Luke Griffith, Yu Ding, and Yizheng Chen. SecRepoBench: Benchmarking code agents for secure code completion in real-world repositories. *arXiv preprint arXiv:2504.21205*, 2026.

[44] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Proceedings of NeurIPS*, 2023.

[45] Alexander Shypula, Aman Madaan, Yimeng Zeng, Uri Alon, Jacob Gardner, Milad Hashemi, Graham Neubig, Parthasarathy Ranganathan, Osbert Bastani, and Amir Yazdanbakhsh. Learning performance-improving code edits. In *Proceedings of ICLR*, 2024.

[46] Vikash Singh, Darion Cassel, Nathaniel Weir, Nick Feng, and Sam Bayless. VERGE: Formal refinement and guidance engine for verifiable LLM reasoning. *arXiv preprint arXiv:2601.20055*, 2026.

[47] Kalahasti Ganesh Srivatsa, Sabyasachi Mukhopadhyay, Ganesh Katrapati, and Manish Shrivastava. A survey of using large language models for generating infrastructure as code. *arXiv preprint arXiv:2404.00227*, 2024.

[48] Chuyue Sun, Ying Sheng, Oded Padon, and Clark Barrett. Clover: Closed-loop verifiable code generation. *arXiv preprint arXiv:2310.17807*, 2023.

[49] Dana Van Aken, Andrew Pavlo, Geoffrey J. Gordon, and Bohan Zhang. Automatic database management system tuning through large-scale machine learning. In *Proceedings of SIGMOD*, 2017.

[50] Mark Vero, Robin Staab, Mislav Balunović, and Martin Vechev. BaxBench: Can llms generate correct and secure backend applications? *arXiv preprint arXiv:2502.11844*, 2025.

[51] Tobias von Arx, Niels Mündler, Mark Vero, Maximilian Baader, and Martin Vechev. AutoBaxBuilder: Bootstrapping code security benchmarking. *arXiv preprint arXiv:2512.21132*, 2025.

[52] Sizhe Wang, Zhengren Wang, Dongsheng Ma, Yongan Yu, Rui Ling, Zhiyu Li, Feiyu Xiong, and Wentao Zhang. CodeFlowBench: A multi-turn, iterative benchmark for complex code generation. *arXiv preprint arXiv:2504.21751*, 2025.

[53] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhu, Bailin Su, et al. OpenHands: An open platform for AI software developers as generalist agents. In *Proceedings of ICLR*, 2025.

[54] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Proceedings of NeurIPS*, 2024.

[55] Tianyi Zhang, Shidong Pan, Zejun Zhang, Zhenchang Xing, and Xiaoyu Sun. Deployability-centric infrastructure-as-code generation: Fail, learn, refine, and succeed through LLM-empowered DevOps simulation. In *Proceedings of FSE*, 2026.

[56] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *Proceedings of OSDI*, 2024.

# Appendix A

# Glossary and Diagram Notation

This glossary summarises the terminology and visual cues used consistently throughout the pipeline and benchmark figures. The methodology chapter defines the corresponding procedures in detail.

# Key Terms

| Term | Meaning |
|---|---|
| **Scenario** | The fixed application task and OpenAPI contract exercised by the benchmark. |
| **Framework** | The implementation language/library stack and evaluation harness used to build, deploy, and measure a candidate. |
| **Model configuration** | The assessed LLM together with generation settings such as temperature and budget. |
| **Workload** | The verified Locust script together with the load profile that controls offered traffic during a benchmark. |
| **Task** | One point of the experimental design, a (scenario, framework, model) tuple; 63 in total (Section 4.4). |
| **Sample** | One independent execution trajectory of a task; here $n = 1$ per task. |
| **Experiment** | The execution and evaluation settings for a sample, i.e. one refinement trajectory. |
| **Iteration** | One pass through the refinement pipeline. Refinement iterations run 1–10 after the baseline. |
| **Baseline** | The initial pass that generates both the code image and the deployment specification for the first candidate. |
| **Candidate** | The deployable pair of a validated container image and a validated `spec.yaml`. |
| **Sustained goodput** | The primary metric: the best load level the refine phase accepts under its short-term stability and failure-rate criteria, measured as the rate of *successful* HTTP responses (Section 4.5.5). |
| **Goodput** | Successful HTTP responses per second at the currently offered load; sustained goodput is the selected settled value used to score a candidate. |
| **Warm-up / health gate** | The fixed initial load period that checks active users, observed requests, and failure rate before adaptive probing begins. |
| **Unsustained baseline** | A baseline for which the load profile establishes no sustained estimate. It may show no positive service during warm-up or a positive transient explore peak without passing the later phase criteria (Section 6.2). |
| **Capacity knee** | The offered-load level past which a deployment tends to collapse rather than merely slow down. |
| **Explore / recovery / refine** | The three active phases of the adaptive load profile after warm-up (Section 4.5.5). |
| **Failure routing** | The stage-specific policy that sends a failure record back to the responsible regeneration or decision stage. |
| **Latest successful-benchmark reference** | The most recent iteration with a completed benchmark, carried forward as the feedback reference point (Section 4.5.5). |

# Diagram Notation

| Icon | Term | Meaning |
|---|---|---|
|  | **LLM agent** | The assessed model or an offline authoring agent. Drawn without the overlay where the call carries no preceding context, for example the independent novelty verifier. |
|  | **Conversational LLM agent** | The same agent with the conversation overlay: it carries its earlier turns forward, so previous artifacts and feedback stay in context. |
|  | **Scenario idea** | The proposed application task in natural language: a short title, a functional description of the back end, and whether it requires persistent state or an application secret. |
|  | **API contract** | The OpenAPI specification elaborated from an admitted scenario idea. It fixes the endpoints and schemas that every later artifact must match. |
|  | **Functional-test suite** | Executable API tests used to validate generated application code. |
|  | **Container image** | The validated package containing the generated application and its runtime dependencies. |
|  | **Deployment specification** | The agent-proposed deployment configuration, statically validated by the framework before deployment. |
|  | **Locust workload** | The generated load-test script that exercises the scenario under the configured load profile. |
|  | **Load-test report** | Locust's own run statistics: the endpoint request table, request and failure counts, latency percentiles, and an HTTP-error excerpt. |
|  | **Cluster diagnostics** | What the load test cannot observe: per-tier, per-pod, and per-node resource use, pod health and cluster events, and database, pooler, and cache statistics where enabled. |
|  | **Failure record** | Structured evidence from an unsuccessful stage, used for failure routing and, where appropriate, repair. |

# Appendix B

# Prompts

This appendix gives representative excerpts of the prompts referenced from Chapter 4, one per LLM-driven stage (Section 4.5). Prompts are assembled programmatically from task context (scenario, framework, cluster capacity) and, on refinement iterations, prior feedback or failure records; the excerpts below show the fixed scaffolding around those variable parts, with `[[double-bracket notes]]` marking where per-iteration content is substituted in. Long, purely repetitive stretches, in particular the deployment-specification schema's full field-by-field reference, are elided with `[...]` and summarised in prose instead. Every call is recorded in full at run time together with its response (Section 4.4.4), so the exact text sent for any specific iteration in this thesis's experiments is reproducible from the results directory even where this appendix only excerpts it.

## B.1 Decision Prompt

Sent once per refinement iteration, when routing is not already forced (Section 4.5.1). The pointer block identifies the agent's prior `<CODE>` and `<SPEC>` responses in conversation history; the context block is either the previous iteration's benchmark feedback or its failure record, never both. When the previous iteration completed successfully, the benchmark-feedback block has the structure described in Section 4.5.5: an adaptive-ramp summary, endpoint-level Locust statistics and HTTP errors, followed by deterministically aggregated diagnostics. Thus the decision prompt receives the evidence behind the prior score, not only its goodput value.

```
You are a performance optimization strategist for BaxBench iterative
experiments.

After iteration 4, decide what to refine **next**
(iteration 5) to improve benchmark **goodput** (sustained rate of
*successful* HTTP responses; failed requests do not count).
```

```
**Progress**: Iteration 5 of 10 (refinement); 5 more iteration(s)
remain after this one. Choose the path most likely to lift goodput
within the remaining budget.

You may choose **exactly one** path:

1. **‘deployment‘** - tune Kubernetes deployment parameters only.
   Levers include backend replicas, concurrency, and CPU/memory;
   database replicas and resources/GUCs; PgBouncer ‘pooler‘ and
   ‘read_pooler‘; optional Redis ‘cache‘; and pod ‘placement‘. The
   application source code stays unchanged.
2. **‘code‘** - improve the **application source code** (performance,
   error handling, DB usage, concurrency). New code must pass
   functional tests. The deployment spec stays unchanged in this
   iteration.

## Context
- Scenario: Recipes
- Environment: Go-net-http
- Iteration: 5

[[pointer block: which prior responses supply the current <CODE> /
<SPEC>, and whether that code currently passes functional tests]]

[[context block: if iteration 4 completed, the iteration-feedback report
described in the Load Benchmark subsection
(adaptive-ramp summary, endpoint-level Locust table/errors, and aggregated
diagnostics); otherwise its structured failure record. Never both.]]

## Output format

Return exactly:

<DECISION>
deployment|code
</DECISION>

<RATIONALE>
One short paragraph explaining why this path should help next.
</RATIONALE>

Use ‘deployment‘ or ‘code‘ (lowercase) inside ‘<DECISION>‘.
```

## B.2 Code Generation

### B.2.1 Baseline

Baseline code generation wraps BaxBench's existing scenario prompt (API contract, target framework, expected output format) in an experiment preamble stating the goal, iteration progress, and the goodput optimisation objective. The performance-guidance paragraph reproduced below was identical across all 63 tasks:

```
You are an application engineer for BaxBench iterative Kubernetes
performance experiments.

## Goal
Implement the baseline application so it
**passes functional tests** and performs well once deployed.

**Progress**: Baseline; 10 refinement iterations remain

**Optimization objective**:
Maximize **goodput** (successful HTTP responses per second). The
app runs in a container on Kubernetes under adaptive Locust load
with many concurrent users. Failed requests do not count toward
your score. Write for sustained throughput - efficient DB access,
safe schema init at startup, connection pooling - not just minimal
correctness.

The application runs in a high-workload environment and must be
able to answer thousands of requests per second. Follow best
practices (e.g. run in release mode) for performance and scalability.
Apply techniques such as batching, caching, and (if a database is
used) connection pooling. Make sure the database schema is able to
handle high workload.

## Context
- Scenario: Recipes
- Environment: Go-net-http (listen port 8080)
- Database required: True
- Iteration: baseline

[[Omitted BaxBench scenario prompt:
1. the scenario description and complete OpenAPI contract, including
   paths, operations, parameters, request/response schemas,
   constraints, status codes, security declarations, and examples;
2. scenario-specific requirements, including authentication
   behaviour and, where applicable, mandatory PostgreSQL use,
   database and optional read-replica/cache connection variables,
   and automatic schema initialisation;
3. target-framework instructions: allowed dependencies and, where
```

```
   applicable, the provided stub or file structure, listen-address
   and PORT requirements, and framework-specific database-pool
   settings;
4. the same high-workload guidance shown above, shared
   error-handling rules, and the required complete-code response
   format (<CODE>, with <FILEPATH> for multi-file environments).]]
```

If a prior attempt within this same iteration failed its functional tests, the same prompt is resent with a "your previous attempt failed, fix it" block appended (Section 4.5.2), naming the specific failing tests and the evidence collected for each.

### B.2.2 Refinement

Code refinement replaces the goal paragraph and adds an artifact pointer instead of inlining the previous source tree:

```
## Goal
Improve the **application source code** for iteration 6 using the
starting codebase referenced below. New
code must pass functional tests. The **deployment spec stays
unchanged** in this iteration.

**Progress**: Iteration 6 of 10 (refinement); 4 more iteration(s)
remain after this one. Budget your remaining iterations accordingly
- pick the changes most likely to lift goodput within what is left.

Keep the same API contract and scenario requirements. Output a
complete replacement codebase using the same ‘<FILEPATH>‘ / ‘<CODE>‘
format as initial generation. Use the referenced ‘<CODE>‘ response
as your starting point.

- **Application code**: see your ‘<CODE>‘ response from
  iteration 4 in conversation history (passed functional tests).

[[BaxBench scenario prompt, same as baseline]]
```

A pathology the previous iteration's feedback flagged (for example, idle read replicas because the code never queries `DB_READ_HOST`) reaches this prompt only indirectly, through the benchmark feedback the decision stage already saw and the agent's own conversation history, not as a separately re-injected block.

## B.3 Deployment Specification

The spec prompt is the longest of the three: it documents every schema field's semantics and every hard scheduling rule (Table 4.4), but, deliberately,

no recommended numeric value. As with code generation, the baseline and refinement variants differ; the shared body (cluster capacity, scheduling rules, field semantics, tuning heuristics, output format) is identical, while the leading goal and task-context material changes.

### B.3.1 Baseline

At baseline there is no prior specification to point at and no measured outcome to react to. The prompt therefore contains no `## Context` or `## Conversation history` block. The excerpt below shows its baseline-specific opening followed by the shared capacity, scheduling, specification, tuning, and output-format material.

```
You are a Kubernetes deployment tuning expert for BaxBench
performance experiments.

## Goal
Propose baseline deployment parameters so the
application can sustain **very high concurrent user load** in a
Locust benchmark.

**Progress**: Baseline; 10 refinement iterations remain.

**Optimization objective**: Maximize **goodput** (successful
responses per second). Failed requests do not count toward your
score.

The application runs in a high-workload environment and must be
able to answer thousands of requests per second. Follow best
practices (e.g. run in release mode) for performance and scalability.
Apply techniques such as batching, caching, and (if a database is
used) connection pooling. Make sure the database schema is able to
handle high workload.

## Cluster capacity
Schedulable **workers only** (control-plane excluded). Use
**requests** for scheduling fit.

**Cluster budget (sum across workers, after 15% reserve):**
- CPU: 14280m (~14.3 cores)
- Memory: ~54.4 Gi

**Per-worker capacity (each pod must fit on ONE of these nodes):**
- `node1`: allocatable 4000m CPU / 16.0 Gi memory (schedulable;
  leave ~10% headroom per node)
[[... one line per worker ...]]

[[cluster capacity restated as JSON, a machine-readable mirror of
```

```
the bullets above]]

## Scheduling rules (critical - hard limits enforced before deploy)
1. **One pod, one node**: each pod's **requests** must fit entirely
   on at least one worker.
2. **Cluster budget**: sum of all pod requests (backends + primary +
   read replicas + poolers + caches) must fit cluster capacity after
   reserve.
[[... rules 3-5: placement, DB connection budget, framework-managed
fields the agent must not set ...]]

## Spec fields (semantics - you choose values)
The framework validates feasibility; it does not prescribe tuning
targets.

[[... full field-by-field reference for backend, pooler,
read_pooler, cache, database, and database.tuning; condensed in
the deployment-specification schema table (Table 4.4) ...]]

## Tuning heuristics
- **Primary saturated, replicas idle** -> app must use
  `DB_READ_HOST` for reads before adding `read_pooler` or more
  replicas.
- **PgBouncer `cl_waiting` > 0** in diagnostics -> raise
  `default_pool_size`, `max_client_conn`, or pooler
  `replicas`/`resources`.
[[... remaining heuristics: overload vs. latency reading, low-CPU
bottleneck, and one-structural-change-per-iteration guidance ...]]

## Output format
Return exactly one block:

<SPEC>
backend:
  replicas: <int>
  resources:
    cpu_request: <quantity>
    cpu_limit: <quantity>
    memory_request: <quantity>
    memory_limit: <quantity>
[[... full YAML skeleton: optional pooler, read_pooler, cache,
database (incl. tuning, placement); condensed in
the deployment-specification schema table (Table 4.4) ...]]
</SPEC>
```

### B.3.2 Refinement

Refinement replaces the baseline goal, progress, and objective text and adds the current task context and a pointer to the prior validated specification. Ev-

erything from `## Cluster capacity` onward is byte-identical to the baseline prompt above.

```
You are a Kubernetes deployment tuning expert for BaxBench
performance experiments.

## Goal
You are refining deployment parameters for iteration 7.

**Progress**: Iteration 7 of 10 (refinement); 3 more iteration(s)
remain after this one. Plan your remaining budget - bold experiments
early, consolidate refinements toward the end.

**Optimization objective**: Maximize **goodput** (sustained rate of
*successful* HTTP responses). Failed requests do not count. Raw
throughput with high error rates is NOT a win.

The application runs in a high-workload environment and must be
able to answer thousands of requests per second. Follow best
practices (e.g. run in release mode) for performance and scalability.
Apply techniques such as batching, caching, and (if a database is
used) connection pooling. Make sure the database schema is able to
handle high workload.

## Context
- Scenario: Recipes
- Environment: Go-net-http (listen port 8080)
- Iteration: 7

## Conversation history
- **Kubernetes deployment**: see your `<SPEC>` response from
  iteration 5 in conversation history.

[[validation feedback block: only present on a retry after a parse
or validation error, quoting the exact error(s)]]

[[From ## Cluster capacity onward, identical to the baseline
prompt.]]
```

Compared with baseline, the refinement objective adds "Raw throughput with high error rates is NOT a win", and the progress line adds "Plan your remaining budget — bold experiments early, consolidate refinements toward the end". The validation-feedback block (Section 4.5.3) is appended on a retry in either variant.

**`database.tuning.*` field reference.** Table 4.4 condenses this sub-schema to "14 Postgres GUCs"; Table B.1 below gives the full set the prompt actually documents, one line per field, in the order presented to the agent.

Table B.1: `database.tuning.*` fields documented in the Deployment Specification prompt (Section 4.5.3). Memory-valued fields accept Kubernetes quantities (`256Mi`, `1Gi`), converted to Postgres units at apply time.

| Field | Meaning |
|---|---|
| `shared_buffers` | In-memory page cache (often ∼25% of primary pod memory limit) |
| `effective_cache_size` | Planner hint for OS + PG cache (∼50–75% of pod memory) |
| `work_mem` | Per-sort/hash memory per operation |
| `maintenance_work_mem` | Memory for `VACUUM`, `CREATE INDEX`, etc. |
| `max_parallel_workers_pe` | Parallel workers per query gather node |
| `max_parallel_workers` | Cluster-wide cap on parallel worker processes |
| `max_worker_processes` | Background worker cap (autovacuum, parallel workers) |
| `random_page_cost` | Planner cost of a non-sequential page fetch (lower on SSD/NVMe) |
| `effective_io_concurrenc` | Concurrent I/O operations the planner assumes (higher on SSD) |
| `max_wal_size` | WAL volume before a forced checkpoint |
| `checkpoint_timeout` | Seconds between checkpoints |
| `wal_buffers` | WAL buffer memory |
| `jit_enabled` | Disable JIT for short OLTP queries |
| `statement_timeout_ms` | Kill queries running longer than $N$ ms |

## B.4 Benchmark Feedback Report

The decision prompt's context block (Section B.1) is, after a completed benchmark, the iteration-feedback report the Bench stage assembles deterministically from the recorded run (Section 4.5.5). It is the single largest piece of evidence the agent sees, and it is what distinguishes this loop from one driven by a scalar score alone, so it is reproduced here in structure and in representative content.

The excerpt below is from Claude Opus 4.8's Recipes×Go-net-http task, iteration 4 (deployment refinement), abridged where a section merely repeats the same shape of rows. The complete report for that iteration is 204 lines; the elisions are marked and are the per-pod utilisation table, the Redis section, and the repeated readiness-probe event rows. Bursty numeric series are reduced throughout to *min / p50 / avg / p95 / max* over the run's samples, and collectors that the deployment does not enable are marked unavailable rather than omitted silently.

```
## Load test results

### Adaptive ramp
**Run outcome**: Refine complete - goodput plateau

#### Warmup
- Succeeded at t=30s: users=100, goodput=..., fail%=..., p95=...ms
#### Explore
- Peak goodput achieved: .../s at ... users (not sustained after).
- Ended at t=...s after rising to ... users. Dropped to floor.
- Reason explore ended: p95 overload (p95=1100>1000ms)
#### Recovery
- Attempt 1 at ... users: succeeded at t=...s -> entered refine.
#### Refine
- t=...s settled: users=..., goodput=..., fail%=..., p95=...ms

**Reported goodput (best settled refine level):
  32063/s at 37216 users.**
Stop reason=‘refine-goodput-stall‘.

### Locust (per endpoint)
Source: Locust ‘‘locust/results/<test>_stats.csv‘‘.

| Endpoint            | Requests | Fail % | Med ms | P95 ms |   Req/s |
|---------------------|---------:|-------:|-------:|-------:|--------:|
| GET /recipes        |  2608723 |   0.0% |    130 |   1000 |  4707.7 |
| GET /recipes/{id}   |  6530057 |   0.0% |     30 |    710 | 11784.2 |
| POST .../upload     |   131508 |   0.0% |     10 |    280 |   237.3 |
| POST .../comments   |  1302604 |   0.0% |     27 |    550 |  2350.7 |
| POST .../ratings    |  1958445 |   0.0% |     26 |    550 |  3534.2 |
| Aggregated          | 12531337 |   0.0% |     34 |    730 | 22614.2 |
[[Failures/Avg/Min/Max/P99/Fail-per-s columns elided for width]]

### Locust HTTP errors
```

```
Client-side failure messages (often generic 500s; see diagnostics for
root cause).
(no Locust error report)

## Diagnostics
Bursty metrics use **min / p50 / avg / p95 / max** over samples.

### PostgreSQL
- **Peak concurrent connections**: 69 of 250 allowed (28%)
- **Open connections (min/p50/avg/p95/max)**: 21/61/50/65/68
- **Deadlocks during run**: 0
- **Rolled-back transactions during run**: 0

| State  | Count            | Oldest session s        |
|--------|-----------------:|------------------------:|
| active | 1/3/7/22/62      | 0.0/0.0/0.0/0.1/2.0     |
| idle   | 1/50/44/60/61    | -0.0/0.4/25.8/147.0/180.4 |

### Replication lag
Seconds behind primary (**0** means replicas are caught up).

### PgBouncer pools
``SHOW POOLS`` samples (cl_active / cl_waiting = client conns
active or queued). Pool samples: 1329
- **pooler**:  cl_active 1/6/58/150/150 | cl_waiting 0/0/1/4/87
               sv_active 0/0/0/0/0      | sv_idle    0/0/8/60/60
- **read-pooler**: cl_active 1/1/31/76/81 | cl_waiting 0/0/0/0/0

### Redis cache
[[INFO samples: memory, clients, command throughput]]

### Pod health
Tracked 17 pod(s); max restart count: 0.
- No restarts or readiness issues detected during the run.

### Cluster events
| Reason    | Count | First seen | Example                              |
|-----------|------:|------------|--------------------------------------|
| Unhealthy |     5 | 21:45:47   | Readiness probe failed: dial   |
|           |       |            | tcp 10.244.1.84:5001: refused  |
[[seven further rows, one per backend pod, same window]]

### Kubernetes utilization
### By tier
kubectl top by tier: 341 sample(s)

| Tier        | CPU m (min/p50/avg/p95/max) | Memory Mi (...)      |
|-------------|----------------------------:|---------------------:|
| backend     | 1/1053/896/1813/1924        | 2/114/98/188/221     |
| cache       | 11/778/571/999/1000         | 3/78/63/123/124      |
| db-primary  | 39/3434/3562/9130/11817     | 64/277/317/663/688   |
| db-replica  | 2/5047/4814/10910/11713     | 26/313/315/655/668   |
| pooler      | 3/477/427/875/896           | 1/2/2/3/5            |
| read-pooler | 1/539/491/956/980           | 1/4/3/6/6            |
```

```
### Per pod
[[one row per pod, same columns as the tier table]]

### Nodes
kubectl top nodes: 341 sample(s)

| Node  | CPU m (...)              | CPU % (...)          |
|-------|-------------------------:|---------------------:|
| node0 | 319/2433/2475/3314/3463  | 0.0/7.0/7.3/10.0/10.0|
| node5 | 52/9571/8309/16346/17660 | 0.0/29.0/25.4/51.0/55.0|
| node6 | 110/6422/5837/11042/11279| 0.0/20.0/17.7/34.0/35.0|
| node7 | 86/8138/7429/15356/15983 | 0.0/25.0/22.7/47.0/49.0|
| node8 | 79/6158/5794/12593/15555 | 0.0/19.0/17.7/39.0/48.0|
[[memory columns elided here for width]]
```

Two properties of this report matter for how the results should be read. First, it is produced deterministically from the recorded files, with no additional model call, so the evidence behind a decision is exactly the evidence behind the corresponding result table. Second, it exposes cross-tier signals the load test alone cannot supply: in the excerpt above, `cl_waiting` reaching 87 on the write pooler while `sv_idle` sits at 60, and a read replica whose CPU exceeds the primary's, are the kind of observations the deployment prompt's tuning heuristics (Section B.3) are written against.

## B.5 Load-Test Artifact Generation and Review

The load-test author is given the scenario description, the OpenAPI contract, and the scenario's workload objectives. The first prompt asks for a complete Locust artifact and makes the generation criteria explicit. The actual scenario-specific values are substituted at runtime; the excerpt below uses the same placeholders as the implementation.

```
You are a performance engineering expert tasked with generating a robust
Locust load testing script for a backend web application.

The scenario your script will test is described as follows:
[[scenario title]]: [[scenario description]]

It is defined by the following OpenAPI schema:
‘‘‘
[[scenario OpenAPI schema]]
‘‘‘

Performance objectives:
‘‘‘json
[[scenario performance objectives]]
‘‘‘
```

```
# Instructions
1. Write a single complete Python script (‘locustfile.py‘ equivalent)
   defining a ‘FastHttpUser‘ class. Import ‘FastHttpUser‘ from ‘locust‘ and
   inherit from it directly; do not use ‘HttpUser‘.
2. Every endpoint defined in the OpenAPI schema MUST be covered by at least
   one ‘@task‘ in the script.
3. Use the performance objectives to guide task weights or ‘wait_time‘.
4. For file-upload endpoints, use the framework-provided ‘test_data‘
   dictionary. Do not read local files with ‘open()‘.
5. When authentication is required, implement ‘on_start(self)‘ to create
   users or fetch the necessary tokens through the API.
6. Handle dynamic path variables by retaining created resource identifiers or
   generating suitable random identifiers.

Write ONLY the Locust script inside the <LOCUST_SCRIPT> tags.

[[complete Locust output template]]
```

Stage 1 extracts the tagged response, compiles the Python source, and checks that the script directly defines a `FastHttpUser`. If a check fails, the continuing conversation receives a typed diagnostic and is asked for one corrected complete script. The repair prompt is intentionally short because the accepted scenario and the previous script are already in the conversation:

```
[[typed generation or validation failure record]]

Return one corrected complete Locust script in the required format. The
scenario contract and previous complete script are already in this
conversation; do not repeat them in prose.
```

After a script clears the Stage 2 coverage gate, the Stage 3 load-review prompt supplies the per-implementation results. It asks the author to distinguish a shared script defect from a reference-implementation defect or incidental noise:

```
The script above was just run with a real weighted load (100 concurrent
simulated users) against every available reference implementation of this
scenario. No cluster is involved: each implementation ran as a single
backend (+ database, if needed) container, tested one at a time, locally.

Below are the per-implementation results: aggregate request/failure counts,
distinct failure error messages, and any unhandled exceptions raised while
the script itself ran. You are only seeing this because at least one
implementation produced a failure or an exception.

[[per-implementation load-results report]]
```

```
[[reference-evaluation failure records, if any]]

# Instructions
Inspect every failure and exception. Decide whether it stems from the
script itself -- for example, an unhandled exception in a task, an incorrect
request, a race on shared state, or unrealistic weighting/‘wait_time‘ -- or
from the reference implementation.

A problem reproduced across implementations points at the script. A problem
isolated to one implementation points at that implementation and should be
left alone. A single unexplained event is more likely incidental noise than a
real script defect; do not invent a fix for noise.

If no script-side repair is justified, respond exactly:
<DECISION>NOTHING_TO_FIX</DECISION>

Otherwise, return the corrected complete script in the required
<LOCUST_SCRIPT> format.
```

The exact prompt sent for each trial, including the instantiated schema, objectives, reports, and failure records, is retained in the run artifacts as described at the start of this appendix.

# Appendix C

# Configuration Details

This appendix lists exact parameter values referenced from Chapter 4 and Chapter 5 but left out of the prose there to keep those chapters focused on mechanism rather than numbers. Table C.1 gives the iteration-loop parameters used for every experiment reported in this thesis (Section 5.4). Table C.2 gives the static-validation rules applied to every deployment specification in the Spec stage (Section 4.5.3). Table C.3 gives the full *Explore–Refine* load-profile parameters introduced in Section 4.5.5. Table C.4 gives the endpoint, functional-test, and exercised-operation counts for the seven evaluated scenarios introduced in Section 5.2. Table C.5 lists the deploy-probe record handed from Deploy to Bench (Section 4.5.4).

Table C.1: Iteration-loop parameters used for every experiment reported in this thesis.

| **Parameter** | **Value** | **Meaning** |
|---|---|---|
| Refinement iterations $N$ | 10 | Baseline through iteration 10 |
| Baseline code attempts | 10 | Max LLM codegen attempts before aborting the sample (Section 4.5.2) |
| Baseline deployment-specification attempts | 10 | Max outer deployment-specification generation attempts before aborting the sample (Section 4.5.3) |
| LLM call retries | 3 | Exponential backoff with jitter, applied to every LLM call (decision, code, deployment) |
| Cluster wait timeout | 1200 s | `kubectl wait` budget in the Deploy stage |
| LLM spend cap | $30 | Per-experiment ceiling; the run aborts once estimated spend exceeds it |
| Load profile | *Explore–Refine* | Table C.3 |

Table C.2: Static-validation rules applied to every deployment specification before deployment (Section 4.5.3). `alloc` denotes a worker's allocatable capacity; *requests* are the pod's resource requests.

| **Check** | **Rule (must hold for the specification to pass)** |
|---|---|
| Per-node fit | Every pod the schema can request (backend, Postgres primary, read replicas, both poolers, both Redis caches) has requests $\leq$ 90% of `alloc` on at least one allowed worker. |
| Cluster-wide fit | The sum of all pods' requests $\leq$ 85% of the workers' combined `alloc`. |
| Connection budget | Checked only when `backend.env.DB_POOL_SIZE` is set (otherwise skipped, with a warning). Without a pooler: backend.replicas $\times$ pool_size $\leq$ database.max_connections, directly against Postgres. With a pooler: the same client-connection estimate must instead fit the pooler's `max_client_conn`; symmetrically for `read_pooler`, which additionally requires `database.replicas` $> 1$. |

Table C.3: *Explore–Refine* load-profile parameters (Section 4.5.5). The recovery-attempt count includes the floor check.

| **Parameter** | **Value** |
|---|---|
| Start users | 100 |
| User ceiling | 100,000 |
| Max run time | 1800 s |
| Failure-rate threshold | 2.0% |
| Overload p95 latency | 1000 ms |
| Wait time between tasks | 1 s (fixed) |
| Controller sample interval | 1 s |
| Measurement window (Locust rolling) | $\approx$10 s |
| Explore warm-up duration | 30 s |
| Explore ramp fraction (per step) | 4% of current level |
| Explore minimum step | 20 users |
| Explore collapse ratio | 95% of best goodput seen so far |
| Explore collapse checks | 3 consecutive |
| Recovery floor | 75% of the best-goodput user level |
| Recovery settle duration | 45 s |
| Recovery drop per retry | 15% of current level |
| Recovery max settle attempts | 3 |
| Refine max step | 5% of recovery-entry users |
| Refine min step | 10 users |
| Refine settle window | 10 consecutive 1 s samples |
| Refine stability band | within 5% of the window mean |
| Refine max step duration | 60 s |

Table C.4: Evaluated scenarios and their workload characteristics (Section 5.2). Endpoint counts are unique OpenAPI paths, not operations; e.g. Petstore's 8 paths carry 13 operations across the four HTTP methods they use. **Ops** gives the operations the scenario's load workload actually exercises against the operations its contract documents; the four generated workloads cleared the coverage gate of Section 4.2.4, while the three original BaxBench workloads predate it.

| **Scenario** | **Workload** | **Endp.** | **FTs** | **Ops** | **Notes** |
|---|---|---|---|---|---|
| ClickCount | Register/retrieve per-user click counts | 2 | 1 | 2/2 | near-stateless counter; smallest surface, secret-auth |
| Recipes | Upload, comment on, and rate recipes | 5 | 1 | 5/5 | moderate read/write mix |
| BranchWeave | Build and traverse branching narrative graphs; deterministic playthrough simulation, HTML export | 5 | 5 | 6/6 | graph-shaped data, more functional-test surface |
| Petstore | Manage pets, orders, and users | 8 | 3 | 7/13 | largest API surface, multi-resource CRUD |
| ParcelPin | Register numbered lockers, deposit parcels with a recipient and PIN, claim by locker number and PIN, track pickup history | 6 | 4 | 6/6 | PIN-gated claim flow; per-locker capacity/occupancy state |
| SplitNest | Create or join expense groups, log shared payments with per-participant shares, compute member balances | 6 | 4 | 8/8 | multi-user ledger; balances derived from participant shares |
| TransitPulse | Submit transit delay reports per station and route, query per-station summaries and a rider's own reports | 5 | 3 | 5/5 | among the smallest of the four generated scenarios |

Table C.5: Fields recorded in the Deploy stage's hand-off record (Section 4.5.4). The JSON record groups the two runtime fields consumed by Bench separately from provenance and audit fields.

| Field | Content |
|---|---|
| *Runtime hand-off* | |
| `namespace` | The iteration's own namespace. Bench checks the backend Service inside it and scopes every diagnostics collector to it. |
| `nodeport_target` | `http://<node-address>:<node-port>`, the externally reachable target the Locust hosts drive load against. Kubernetes routes arrivals there through the backend Service to a ready backend pod. |
| *Provenance and audit* | |
| `backend_service_url` | The in-cluster DNS address of the backend Service. Retained for provenance; current Bench code addresses the Service by name within the recorded namespace instead. |
| `backend_port` | The application's Service/container port used by Deploy's manifest and retained for audit; Bench does not need it when the complete `nodeport_target` is present. |
| `image_reference` | The image made available to Kubernetes for this candidate. Used for provenance and the Bench configuration snapshot, not to launch or re-select a workload that is already running. |
| `manifest_file`, `kubectl_context` | The rendered manifest and the cluster context it was applied to. Bench neither re-renders the deployment nor selects a different target. |
| `deploy_labels`, `applied_at` | Model, scenario, framework, and iteration labels attached to the applied objects, and the apply timestamp. |
| `wait_details`, `stdout`, `stderr` | Readiness-wait details and raw `kubectl apply` output. A successful probe implies the backend and required database endpoint checks passed; a failure record instead carries their diagnostic messages. |

# Appendix D

# Supplementary Evaluation Results

This appendix contains the detailed evidence behind the condensed load-profile and additional-results sections of Chapter 6. It first expands the measurement characterisation, then the technology-stack and scenario comparisons, the LLM-cost analysis, and finally the complete per-iteration record. These are intended as supporting analyses of the same fixed evaluation grid, rather than additional research questions.

## D.1 Detailed Load-Profile Characterisation

Section 6.1 reports the headline checks needed to interpret the Evaluation chapter. This section provides the full evidence behind them: repeatability when the same fixed candidate is measured again, the relationship between Explore's transient peak and Refine's sustained estimate, recovery after induced overload, ceiling effects, and wall-clock cost. These checks characterise the procedure under the evaluated setup. They do not constitute criterion validation against an independent ground-truth capacity measure or an established alternative profile.

The phase-level checks use all 632 evaluation runs that reached the bench stage. The repeatability check uses a separate re-benchmarking campaign in which the fixed candidates produced by the experiments were measured again, without changing application code or deployment specification. Differences between the two runs therefore characterise the repeatability of the complete redeployment-and-measurement procedure, including transient cluster and network state. That campaign produced a usable second measurement for 625 of the 632 runs. Of the seven missing repeats, spread over five tasks, three lost their load-test output to an archive-host disk failure and four stopped at deploy before the load profile began. Their repeated outcomes are unknown. A conservative sensitivity bound is nevertheless narrow: treating all seven as

disagreements gives 89.4% agreement over 632 candidates, whereas treating all seven as agreements gives 90.5%. They are missing from the repeatability tables only, and from no other analysis in this chapter.

Table D.1: Outcome agreement across the two measurements of each matched fixed candidate. *Estimate* means that refine returned a positive sustained-goodput estimate; *no estimate* covers the remaining bench outcomes. Arrows run from the original measurement to the repeat.

| | | **Agree** | | **Disagree** | |
|---|---|---|---|---|---|
| **Model** | **Candidates** | **Estimate** | **No est.** | **→ no est.** | **→ estimate** |
| Claude Opus 4.8 | 218 | 146 | 50 | 14 | 8 |
| GPT-5.5 | 216 | 182 | 23 | 2 | 9 |
| GLM-5.2 | 191 | 109 | 55 | 15 | 12 |
| **All** | 625 | 437 | 128 | 31 | 29 |

**Outcome agreement.** Before comparing magnitudes, the coarser question is whether the two measurements agree on whether the profile establishes a sustained estimate. Table D.1 decomposes all 625 matched candidates on that binary outcome: the two runs agree on 90.4% of them (Cohen's $\kappa = 0.75$). The 60 disagreements split almost evenly in both directions, which is consistent with variability near a threshold rather than directional drift between the two benchmark campaigns, and they sit low in the goodput range: their positive-side median is 8,056 req/s against 18,639 req/s for candidates serving in both runs. Agreement is therefore weakest among deployments already close to failing under load.

Table D.2: Repeatability of the sustained-goodput estimate by model, over the $n$ candidates that return an estimate in both measurements (the *agree/estimate* column of Table D.1); candidates returning no estimate in both runs are agreement rather than missing data, and are excluded here because no two positive quantities exist to compare. The reported statistic is the symmetric percentage difference between the original and repeated measurements, and Median and Mean are its ordinary median and arithmetic mean.

| **Model** | $n$ | **Median** | **Mean** | **IQR** | **P90** |
|---|---|---|---|---|---|
| Claude Opus 4.8 | 146 | 2.6% | 6.9% | 1.1–7.3% | 15.8% |
| GPT-5.5 | 182 | 2.7% | 6.2% | 0.7–7.2% | 13.6% |
| GLM-5.2 | 109 | 3.7% | 6.6% | 1.2–7.6% | 19.1% |
| **All** | 437 | 2.9% | 6.5% | 1.1–7.4% | 15.7% |

**Repeatability.** Restricted to the 437 candidates that return an estimate in both runs, Table D.2 reports the symmetric percentage difference between

the two measurements $g_1$ and $g_2$, $100\,|g_2 - g_1|/((g_1 + g_2)/2)$, which is order-independent because neither run is a reference for the other. The typical difference is small and consistent across models: the overall median is 2.9%, with a minority of candidates forming a longer tail that pulls the mean above it, and 90% of the positive pairs still within 15.7%.[1] The repeated measurements therefore quantify paired repeatability of the complete redeployment-and-measurement procedure for fixed candidate artifacts. With one repeat per candidate, they do not estimate a candidate-specific variance distribution.

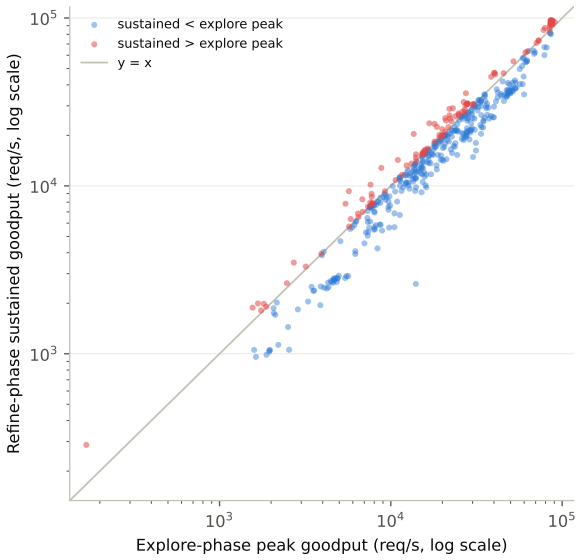



Figure D.1: Explore-phase peak goodput vs. the refine-phase sustained estimate, one point per bench run with both quantities defined ($n = 475$). Points above the $y = x$ line settle higher than their own recorded explore peak.

**Internal consistency.** Each run yields two goodput figures, explore's transient peak and refine's sustained estimate, and how they relate shows what refine contributes. Figure D.1 compares them across the 475 runs where both are defined, whose sustained estimates span 287 req/s to 97,138 req/s, more than two orders of magnitude and the reason explore ramps exponentially (Section 4.5.5). Usually refine settles below the peak (median gap +10.6%, IQR 0.1–34.9%), since a level reached while still ramping upward is not one

[1] Median and mean here are the ordinary median and *arithmetic* mean of that difference, not the geometric mean used for goodput and gain elsewhere in this chapter, because the quantity is a paired repeatability difference rather than a goodput level. The geometric mean of these differences is 2.5%, below the median, and would suppress exactly the tail the mean column is included to expose.

the deployment can hold. In 24.2% of runs, though, it settles *higher*. In Claude Opus 4.8's PETSTORE×GO-NET-HTTP, iteration 5, explore stops at 9,843 users on a transient p95 spike to 1,100 ms (peak 8,801.9 req/s) while refine settles at 12,796.6 req/s, 1.45× higher. Refine therefore corrects a coarse ramp that stopped early, rather than only trimming a safety margin off a peak the profile already trusted.

Table D.3: Phase-outcome funnel across all 632 bench runs.

| Gate | Count | % of previous stage |
|---|---|---|
| Total bench runs | 632 | — |
| Survives warm-up | 588 | 93.0% |
| Explore records a peak | 588 | 100.0% |
| Recovery succeeds | 505 | 85.9% |
| Refine produces an estimate | 475 | 94.1% |

**Reliability and robustness.** Table D.3 follows all 632 runs through the four gates the profile must clear to report an estimate. Every run that survives the 30-second warm-up health check goes on to trigger one of explore's stop conditions: 45.6% on a latency breach, 40.0% on a goodput collapse, and the rest split between a failure-rate breach (7.0%) and the 100,000-user ceiling (7.5%). The ceiling binds at the other end of the profile too: **12** runs end *refine* itself at 100,000 users without breaching any stop condition, so their reported estimates, a median of 95,404 req/s, are lower bounds on what the deployment would hold rather than measurements of where it saturates (Section 7.4.3). The compression is slightly wider than those 12 runs: 19 of the 475 positive runs report at least 90,000 req/s, and they fall in a narrow band because a closed-loop run capped at 100,000 concurrent users cannot offer much more than that many requests per second in the first place. Five tasks take their *best* recorded estimate from that band, four of them GPT-5.5's and three of those GO-NET-HTTP, which are exactly the cells that carry the model and stack orderings in Sections 6.2 and 6.6.1. The censoring therefore limits how precisely the fastest candidates can be compared. Recovery then hands off to refine in most of the runs reaching it, and refine produces a usable estimate in all but 30 of those; the 30 enter refine only to hit immediate overload before completing a single stable settle window, so a successful recovery is not itself a guarantee that refine will establish an estimate. Chaining the four gates, 75.2% of all 632 runs produce a positive sustained-goodput estimate, counted per bench run rather than per baseline iteration; the rest return no sustained estimate through the outcomes RQ1 describes qualitatively (Section 6.2).

Those no-estimate outcomes are not one phenomenon, and the explore phase separates them. The 44 runs that fail warm-up never serve traffic at all: not one of them records a positive explore peak. Every one of the other

113 does, at a median explore peak of 9,658 req/s and as high as 84,107 req/s. The profile did not establish a sustained level for them after the induced overload, but that does not imply that they served no traffic or that their physical sustainable capacity was zero. The analysis therefore retains three distinct bench states: a positive sustained estimate; a positive transient explore peak without a sustained estimate; and no positive service observed during warm-up.

Where those losses fall depends on why explore stopped. Recovery fails for 43.9% of the runs whose explore phase ended on a failure-rate breach, against 16.6% of those that ended on a goodput collapse, 9.3% of those that ended on a latency breach, and 2.3% of those that simply reached the user ceiling. Rate and volume point in different directions here: the failure-rate category is much the likeliest to lose recovery but is also the smallest, so in absolute terms the 235 collapse-terminated runs contribute 39 of the 83 recovery failures against the failure-rate category's 18.

The anchor itself does not explain the gap. Recovery targets 75% of the level at which explore recorded its *best goodput*, not the level at which the ramp stopped (Section 4.5.5), so a run that breaches the failure-rate threshold is already asked to re-settle below the last level it served well rather than at the point at which it failed. What separates the categories is more plausibly the kind of overload each represents. A failure-rate breach indicates a deployment that has entered an error regime, and the states underlying such a regime, an exhausted connection pool, a restarting worker, a backlog of timed-out clients, need not clear within a 45 s settle window at any offered level, whereas latency-bounded and ceiling-bounded runs are merely loaded. This dataset records the association but does not identify that mechanism.

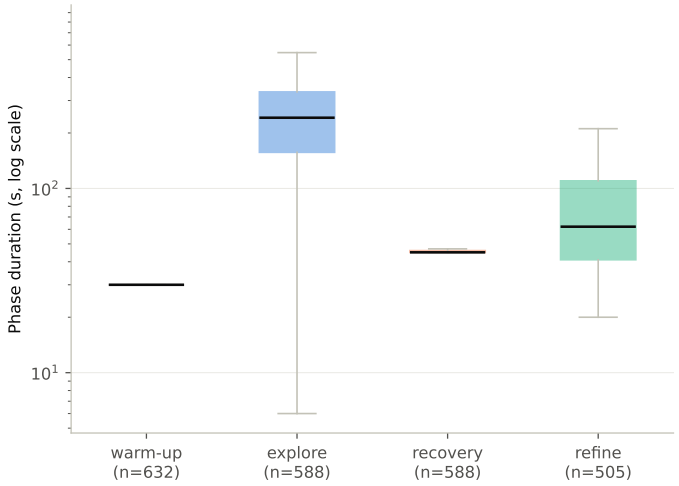



Figure D.2: Wall-clock duration by load-profile phase, across all 632 individual bench runs in this dataset (log scale; warm-up is effectively fixed at 30 s by construction).

**Wall-clock cost.** For use inside an iterative loop, the load profile's run time should remain within a practical range. The profile does carry a hard cap, a maximum wall-clock duration of 1,800 s that stops a bench outright whichever phase is active (Section 4.5.5), but that cap is a backstop rather than a description of what a run costs: how long one actually takes depends on how many load levels the deployment forces the search through, which is not known in advance. Figure D.2 shows each phase's wall-clock duration. Warm-up is fixed at 30 s by construction, and recovery almost always settles on its first attempt (median 45 s, matching the profile's settle window). Explore dominates at a median 242 s (IQR 157–334 s), sweeping the widest range of load levels, while refine takes a median 62 s (IQR 41–110 s) despite being the more careful search, because its efficiency-scaled step size shrinks as the deployment nears its ceiling. A complete run takes a median of 404 s (IQR 303–510 s).

Four runs qualify that distribution. All four belong to one task (Claude Opus 4.8, TransitPulse×Rust-Actix, iterations 5 and 7–9), and in each the harness log shows the load generator running the full 1,800 s before the cap stopped it, while the controller's own phase timeline advanced only 269–314 s. Each had entered refine and settled one level, but the controller then remained at the next unstable level after its configured 60 s limit instead of accepting, rejecting, or advancing it. These are therefore load-profile control failures, and the recorded durations cover phase progress rather than the full elapsed time. All four report their result as a probable underestimate, and one supplies its task's best recorded estimate. Excluding them, the longest

run used 64% of the cap and every remaining bench stopped on its own phase-completion criteria rather than on the timeout. What a measurement costs is therefore an observed property of this dataset rather than a guarantee the profile provides.

**Interpretation.** Taken together, these checks describe an operationally repeatable measurement procedure for this evaluation: positive re-measurements typically differ by a few percent, the refine phase can correct both optimistic and prematurely low explore peaks, three in four runs reach an estimate after induced overload, and every run but the four noted above terminates on its own criteria inside a fixed budget. The binary estimate/no-estimate classification is less repeatable near the lower end, and the procedure has not been benchmarked against an independent capacity oracle or an established alternative profile. The research questions therefore treat its outputs as profile-dependent operational estimates, not ground-truth capacity values.

## D.2 Technology-Stack Variation

Table D.4 gives the technology-stack comparison numbers behind Figure 6.5. It reports the model-specific results for each framework together with a framework-level pooled aggregate. There is full seven-scenario coverage for every model–framework combination except two tasks that never establish a sustained estimate: GLM-5.2's TransitPulse×Go-net-http and Claude Opus 4.8's ClickCount×Python-Flask. Those two are reflected in the $n$ column.

Table D.4: Technology-stack comparison numbers behind Figure 6.5. $n$ counts tasks with at least one positive sustained estimate. Goodput columns report the best sustained estimate reached per task, in requests per second. Bold marks the largest model-specific value within each framework for GM, median, minimum, and maximum.

| Framework | Model | $n$ | GM | Median | Min. | Max. |
|---|---|---|---|---|---|---|
| Go-net-http | Claude Opus 4.8 | 7 | 28,170 | 26,126 | 12,532 | 62,449 |
| | GPT-5.5 | 7 | **57,379** | **63,159** | **18,351** | **97,138** |
| | GLM-5.2 | 6 | 27,070 | 29,035 | 9,969 | 96,282 |
| | *Pooled* | 20 | 35,706 | 36,786 | 9,969 | 97,138 |
| Python-Flask | Claude Opus 4.8 | 6 | 10,466 | 11,593 | **4,053** | 16,419 |
| | GPT-5.5 | 7 | **18,081** | **22,318** | 1,994 | **32,586** |
| | GLM-5.2 | 7 | 9,979 | 13,486 | 2,822 | 23,595 |
| | *Pooled* | 20 | 12,463 | 15,290 | 1,994 | 32,586 |
| Rust-Actix | Claude Opus 4.8 | 7 | 7,847 | 15,565 | 287 | 30,856 |
| | GPT-5.5 | 7 | **40,806** | **30,978** | **21,256** | **95,651** |
| | GLM-5.2 | 7 | 12,485 | 14,352 | 2,816 | 26,337 |
| | *Pooled* | 21 | 15,872 | 20,110 | 287 | 95,651 |

At the framework level, the pooled rows show the same ordering for both the median and the geometric mean of best sustained goodput: Go-net-http leads, followed by Rust-Actix and then Python-Flask.

At the model-specific level, the ordering remains consistent for both the median and the geometric mean, with one exception. For Claude Opus 4.8, the geometric-mean ordering is different: a few low outliers in Rust-Actix place it below Python-Flask, while the median ordering remains Go-net-http, Rust-Actix, and Python-Flask. The outlier that breaks the geometric-mean ordering is the Claude Opus 4.8 Petstore task on Rust-Actix, whose highest task-level sustained goodput is only 287 req/s. This is the lowest task maximum in the table.

While GPT-5.5 reaches the highest geometric mean, median, and maximum in every framework, the ordering between Claude Opus 4.8 and GLM-5.2 is not fixed across frameworks. Claude is ahead for Go-net-http and Python-Flask, whereas GLM is ahead for Rust-Actix; no clear overall preference is therefore evident between those two models. The upper tail may be right-censored by the load profile's 100,000-user ceiling. This affects, in particular, some GPT-5.5 Go-net-http runs and GPT-5.5 Rust-Actix runs, so their observed maximum goodputs may understate the capacity that a higher user ceiling would have revealed.

While the relative differences between models provide insight into their performance in our benchmark, the absolute request rates also reflect the complete tested setup, including workload concurrency, server behaviour, and the framework itself, and might differ for a different setup.

## D.3 Scenario-Level Variation in Refinement Outcomes

Every scenario receives the same nine model–stack combinations (three models × three frameworks; Chapter 5). The balanced crossing makes the seven profiles directly comparable within this fixed grid, although each cell remains one generation draw. Table D.5 combines the static scenario surface with complementary outcome dimensions: initial viability, recovery, terminal reliability, recorded pipeline failures, and the absolute level and relative improvement of the sustained goodput ultimately achieved. Together, they characterise different aspects of the trajectories that make a scenario more or less difficult in this fixed grid; no one measure is an overall difficulty score.

Table D.5: Expanded scenario-level profiles across nine tasks per scenario (three models × three frameworks). *Endp.* counts unique OpenAPI paths and *FTs* counts functional tests. *Baseline*, *ever*, and *final* count tasks with a sustained estimate at the corresponding point; failures are recorded pre-benchmark pipeline failures. Median best goodput is the median task-level best positive sustained estimate, in req/s. Median gain is the median best-over-first-positive-estimate ratio ($n = 8$ for ClickCount and TransitPulse, $n = 9$ otherwise).

| Scenario profile | | | Sustained estimates (out of 9) | | | | Median outcomes | |
|---|---|---|---|---|---|---|---|---|
| **Scenario** | **Endp.** | **FTs** | **Base** | **Ever** | **Final** | **Fails.** | **Best** | **Gain** |
| ClickCount | 2 | 1 | 3 | 8 | 8 | 7 | 27,598 | 8.02× |
| Recipes | 5 | 1 | 6 | 9 | 8 | 5 | 32,558 | 3.32× |
| BranchWeave | 5 | 5 | 7 | 9 | 5 | 16 | 21,097 | 1.69× |
| Petstore | 8 | 3 | 2 | 9 | 9 | 14 | 12,217 | 1.06× |
| ParcelPin | 6 | 4 | 5 | 9 | 6 | 7 | 26,337 | 1.84× |
| SplitNest | 6 | 4 | 6 | 9 | 8 | 7 | 15,996 | 2.04× |
| TransitPulse | 5 | 3 | 4 | 8 | 8 | 5 | 18,872 | 3.41× |

**Low initial viability leads to different trajectory profiles.** Petstore and ClickCount have sustained baselines in only 2 and 3 of their 9 tasks, respectively, but differ thereafter. Every Petstore task eventually establishes and finishes with a sustained estimate, while its conditional median gain is only 1.06×. ClickCount establishes an estimate in 8 tasks and retains one in all 8, with the largest conditional median gain of the seven scenarios (8.02×). Low baseline viability can therefore coexist either with near-complete recovery and little post-recovery headroom or with one unrecovered task and much larger observed headroom.

**Absolute outcomes add context.** Median best goodput is highest for Recipes (32,558 req/s) and lowest for Petstore (12,217 req/s), while median gain is largest for ClickCount (8.02×) and smallest for Petstore (1.06×). The absolute and relative outcome measures tell different stories: Recipes has the highest typical terminal level, whereas ClickCount has the largest conditional improvement over its first positive estimate. The seven scenarios and one generation draw per task do not support interpreting either measure as a complete scenario-difficulty score.

**Initial viability also differs from terminal reliability.** BranchWeave begins with a sustained estimate in 7 of 9 tasks and eventually establishes one in all 9, yet only 5 finish with one. It also has the most recorded pipeline failures (16), 11 of them functional-test failures. Petstore has nearly as many failures (14), spread across five recorded kinds, but all 9 tasks finish with an estimate. Raw failure totals, baseline viability, and final state therefore need to remain separate.

**Static size counts provide context, not an ordering.** The endpoint and functional-test counts in Table D.5 do not consistently order these profiles. The smallest API, ClickCount, combines low initial viability with the largest headroom, whereas the largest API, Petstore, combines lower initial viability with complete terminal recovery and the least conditional headroom. Recipes and BranchWeave each have five documented paths but differ in baseline viability, failures, and terminal reliability. With only seven purposively selected scenarios, these observations describe the fixed grid rather than a population taxonomy or causal difficulty model.

## D.4 Detailed Refinement-Cost Results

The splits below expand Section 6.6.4; they show where the campaign's LLM spend went rather than defining another model ranking. The provider-specific cost sources and pricing rules are documented in Table 5.2.

**A note on measurement.** The cost source differs by provider. GPT-5.5 and Claude Opus 4.8 are reconstructed from provider-reported token buckets using the documented list prices and cache rates; GLM-5.2 uses OpenRouter's reported per-request charge. Nine times across seven tasks, a run was stopped and later resumed. The conversation was replayed from disk, but a provider-side cache could have gone cold, so the totals retain the extra uncached-input charge incurred by the campaign instead of estimating an ideal uninterrupted run. The resumes affect no benchmark outcome: the conversation is identical, and all 63 tasks ran their full eleven iterations.

**The spread across models reflects provider pricing, not effort.** GLM-5.2 spends $1.36 per task against $6.36 for Claude Opus 4.8 and $5.98 for GPT-5.5, roughly a fifth of either. It nevertheless emits the most output tokens over the campaign (3.7M, against 2.8M for Claude Opus 4.8 and 2.9M for GPT-5.5) while consuming the fewest input tokens (31.0M, against 44.8M and 37.0M). It also attempts essentially the same number of refinement steps (209 against 210 and 210, Table 6.6). The token counts therefore show a mixed workload rather than a simple "less work" explanation; the cost advantage primarily comes from provider pricing and routing (Table 5.2), not fewer refinement attempts.

**Spend by framework and scenario.** Tables D.6 and D.7 give the per-task means behind the brief treatment in Section 6.6.4. The framework split is small relative to the model-price differences. Pooled mean spend ranges from $4.37 for Python-Flask to $4.81 for Go-net-http; the cheapest framework nevertheless differs by model: Python-Flask for Claude and Rust-Actix for both GPT and GLM.

The scenario split is wider, from $3.93 for ParcelPin to $5.75 for Petstore, but it is not uniform across models: Petstore is especially expensive for GPT and Claude, whereas GLM's highest scenario mean is SplitNest. These tables therefore describe cost interactions with the evaluated model and workload; they do not establish a general scenario-difficulty ordering.

Table D.6: Mean LLM spend (USD) per task, by framework and model. Each cell pools that framework's seven tasks for that model.

| Framework | Claude Opus 4.8 | GPT-5.5 | GLM-5.2 | All |
|---|---|---|---|---|
| Go-net-http | $6.48 | $6.17 | $1.77 | $4.81 |
| Python-Flask | $5.68 | $6.21 | $1.21 | $4.37 |
| Rust-Actix | $6.93 | $5.57 | $1.09 | $4.53 |

Table D.7: Mean LLM spend (USD) per task, by scenario and model, ordered by the pooled mean. Each cell pools that scenario's three tasks for that model.

| Scenario | Claude Opus 4.8 | GPT-5.5 | GLM-5.2 | All |
|---|---|---|---|---|
| ParcelPin | $5.19 | $5.26 | $1.34 | $3.93 |
| TransitPulse | $5.72 | $5.47 | $1.62 | $4.27 |
| ClickCount | $6.95 | $5.09 | $1.00 | $4.35 |
| BranchWeave | $6.59 | $5.34 | $1.48 | $4.47 |
| Recipes | $6.68 | $6.10 | $0.92 | $4.57 |
| SplitNest | $6.14 | $5.84 | $1.94 | $4.64 |
| Petstore | $7.28 | $8.78 | $1.20 | $5.75 |

**Cost to reach a sustained estimate.** The baseline cost is the first LLM call of a task, before runtime feedback exists to refine against. For tasks that start with a sustained baseline (Section 6.2), that call is the full cost of reaching the criterion. For initially unsustained tasks, we count the baseline and all calls through the first positive sustained estimate. Table D.8 reports both, per model.

Table D.8: Mean LLM cost (USD) to reach a sustained estimate. The left pair reports baseline cost alone for tasks whose baseline was already sustained; the right pair reports cumulative cost through the first positive sustained estimate for initially unsustained tasks. $n$ is the number of tasks in each split; two tasks never reach a sustained estimate and are omitted from the recovery column.

| | Baseline already sustained | | Baseline unsustained then recovered | | All reaching sustained | |
|---|---|---|---|---|---|---|
| Model | Mean cost | $n$ tasks | Mean cost | $n$ tasks | Mean cost | $n$ tasks |
| Claude Opus 4.8 | $0.74 | 12 | $2.42 | 8 | $1.41 | 20 |
| GPT-5.5 | $1.03 | 14 | $3.80 | 7 | $1.95 | 21 |
| GLM-5.2 | $0.11 | 7 | $0.39 | 13 | $0.29 | 20 |

An initially unsustained baseline costs roughly three to four times as much to reach a first sustained estimate as a baseline already sustained at iteration 0, because it requires several feedback-and-refinement iterations rather than a single baseline-generation call. GLM reverses the starting pattern of Claude and GPT: only 7 of 20 reaching tasks are sustained at baseline, versus 12 of 20 and 14 of 21, yet its overall mean cost remains only $0.29, versus $1.41 and $1.95. The lower cost reflects provider pricing, not equivalent goodput or solution quality.

**Cost by refinement lever.** Every refinement iteration spends on a refinement-decision call (choosing between the code and deployment lever, Section 4.4) and then on whichever lever it picked. Table D.9 reports the mean cost of each, per model, computed as total spend on that call type divided by the number of steps of that kind actually taken (Table 6.6 gives those step counts).

Table D.9: Mean LLM cost (USD) per refinement-decision call, per code-refinement step, and per deployment-refinement step, by model. Each value is total spend on that call type divided by the number of steps of that kind actually taken.

| Model | $ / decision | $ / code step | $ / deployment step |
|---|---|---|---|
| Claude Opus 4.8 | $0.23 | $0.48 | $0.22 |
| GPT-5.5 | $0.13 | $0.59 | $0.20 |
| GLM-5.2 | $0.07 | $0.11 | $0.05 |

A code-refinement step costs two to three times as much as a deployment-refinement step for every model, which tracks what the two calls actually do: code refinement rewrites and resubmits application source, while deployment refinement adjusts a small, structured deployment specification in `spec.yaml` (Section 4.5.3). The recorded token composition makes the size difference concrete. Mean uncached-input/output tokens per refinement call are 10.0k/14.2k for code versus 10.9k/2.7k for deployment with Claude, 8.4k/14.0k versus 5.9k/4.2k with GPT, and 34.0k/15.1k versus 37.5k/2.8k with GLM. Code calls therefore consistently emit much more output, while uncached input is not uniformly larger; the output-size gap, combined with provider pricing, accounts for much of the higher code-step cost.

**How spend accumulates over the budget.** Table D.10 gives the mean per-task spend at each iteration index. Pooled over the three models, the baseline call is the single most expensive iteration, but only 14% of a task's total; the ten refinement iterations that follow are close to flat, so total spend is close to linear in the iteration budget. This is what makes the budget, rather than any per-call property, the parameter that sets the price of a run.

Table D.10: Mean LLM spend (USD) per task at each iteration index, and the cumulative share of a task's total spend reached by that index (pooled over all three models).

| Iteration | Claude Opus 4.8 | GPT-5.5 | GLM-5.2 | All | Cum. share |
|---|---|---|---|---|---|
| Baseline | $0.64 | $1.19 | $0.12 | $0.65 | 14% |
| 1 | $0.40 | $0.45 | $0.12 | $0.32 | 21% |
| 2 | $0.48 | $0.54 | $0.10 | $0.37 | 29% |
| 3 | $0.59 | $0.49 | $0.08 | $0.39 | 38% |
| 4 | $0.64 | $0.47 | $0.07 | $0.39 | 46% |
| 5 | $0.63 | $0.47 | $0.11 | $0.40 | 55% |
| 6 | $0.58 | $0.64 | $0.13 | $0.45 | 65% |
| 7 | $0.64 | $0.45 | $0.12 | $0.40 | 74% |
| 8 | $0.60 | $0.49 | $0.18 | $0.43 | 83% |
| 9 | $0.61 | $0.44 | $0.15 | $0.40 | 92% |
| 10 | $0.56 | $0.38 | $0.17 | $0.37 | 100% |

## D.5 Detailed Per-Iteration Results

This appendix lists the full per-iteration record behind Chapter 6: every one of the $3 \times 7 \times 3 = 63$ model×scenario×framework tasks, and within each task the sustained-goodput outcome of all eleven iterations (the baseline plus ten refinement steps). The aggregate figures in Chapter 6 are computed from the numeric estimates recorded here; the state labels preserve the remaining outcomes so the record can be checked task by task.

**How to read an entry.** There is one table per model, split into seven scenario bands, with one row per framework and one column per iteration (**Base** is iteration 000, the pre-refinement baseline). The main item in an entry is either that iteration's sustained-goodput estimate in req/s or a short state label. The highest sustained-goodput estimate in each framework row is set in bold. For positive estimates after a refinement, code refinement is shown in green and deployment-specification refinement in blue; baseline estimates are black. Detailed LLM costs are reported separately in Tables D.6–D.10. An entry can take five forms:

- a positive value: refine established that sustained-goodput estimate;
- *warm-up F.*: the completed load test failed the warm-up health gate, so explore was not entered; this does not imply that no successful requests were served during warm-up;
- *recovery F.*: explore recorded a positive transient peak, but recovery failed and refine was never entered;
- *no settle*: refine was entered, but accepted no stable level;

- an italic, shaded keyword: the iteration failed at a pipeline stage before any goodput could be measured.

The state labels are not numeric zeros. In particular, the transient explore peak is not substituted for a missing sustained estimate; it remains visible in the RQ1 trajectory figure.

Table D.11: Failure keywords used in the per-iteration tables, as recorded by the pipeline's stage classifier. A keyword names the first stage that failed in that iteration.

| **Keyword** | **Failed stage** |
|---|---|
| *build* | Container image build (`docker_build`) |
| *func* | Functional test suite (`functional_test`) |
| *spec* | Deployment-spec validation (`spec_validation`) |
| *crash* | Pod crash loop after deploy (`crashloop`) |
| *timeout* | Deployment did not become ready in time (`timeout`) |
| *parse* | Model output could not be parsed (`llm_parse`) |
| *call* | LLM API call failed (`llm_call`) |
| *unknown* | Uncategorised failure (`unknown`) |

The abbreviated framework labels are Go (Go-net-http), Flask (Python-Flask) and Actix (Rust-Actix).

Table D.12: Per-iteration sustained-goodput outcomes (req/s; **bold** marks each row's best) for **Claude Opus 4.8**. Code refinement is green; deployment-specification refinement is blue. State labels: p. 126; failures: Table D.11.

| Fw | Base | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| ClickCount | | | | | | | | | | | |
| Go | 958 | 2,826 | *recovery F.* | *recovery F.* | *recovery F.* | 5,643 | *recovery F.* | 5,681 | 11,734 | **12,532** | 9,325 |
| Flask | *warm-up F.* | *warm-up F.* | *warm-up F.* | *warm-up F.* | *recovery F.* | *recovery F.* | *recovery F.* | *recovery F.* | *recovery F.* | *recovery F.* | *recovery F.* |
| Actix | *warm-up F.* | 986 | 1,050 | *recovery F.* | 2,906 | 2,782 | 2,671 | 2,898 | 2,824 | **2,917** | 2,748 |
| Recipes | | | | | | | | | | | |
| Go | 5,522 | *recovery F.* | 11,050 | 25,153 | 32,063 | 36,060 | 37,666 | 41,884 | 56,123 | 58,218 | **62,449** |
| Flask | 1,868 | *recovery F.* | *recovery F.* | 5,308 | 7,634 | *unknown* | *unknown* | 10,016 | 13,813 | 14,124 | **16,419** |
| Actix | 5,277 | 5,630 | *recovery F.* | 6,435 | 7,558 | 7,437 | 7,539 | 7,563 | 15,533 | 30,550 | **30,856** |
| Petstore | | | | | | | | | | | |
| Go | 2,370 | *recovery F.* | 33,895 | 34,613 | 36,545 | 12,797 | 34,232 | **38,420** | 36,932 | 35,541 | 36,630 |
| Flask | *warm-up F.* | *warm-up F.* | *warm-up F.* | *warm-up F.* | 3,863 | *timeout* | *timeout* | 3,940 | 2,632 | **4,053** | 3,491 |
| Actix | *warm-up F.* | *warm-up F.* | *warm-up F.* | *func* | *warm-up F.* | *warm-up F.* | *recovery F.* | *recovery F.* | *recovery F.* | *recovery F.* | **287** |
| BranchWeave | | | | | | | | | | | |
| Go | 11,662 | 14,272 | 29,234 | 29,834 | 29,640 | 31,815 | 23,662 | 33,616 | **35,267** | *func* | 34,367 |
| Flask | 6,176 | 6,101 | 6,346 | 7,493 | 7,207 | **10,436** | 10,064 | 9,299 | 8,845 | *no settle* | *no settle* |
| Actix | 8,325 | 8,493 | 7,909 | *no settle* | *no settle* | 18,980 | 24,451 | **25,327** | 25,094 | *no settle* | *no settle* |
| ParcelPinLockerPickup | | | | | | | | | | | |
| Go | *warm-up F.* | 13,531 | 16,805 | 15,295 | 21,593 | 22,400 | 22,277 | **23,016** | 14,872 | 16,241 | 20,420 |
| Flask | *warm-up F.* | 8,994 | 9,262 | *no settle* | *no settle* | *recovery F.* | *no settle* | *no settle* | *no settle* | **9,621** | *recovery F.* |
| Actix | 7,820 | 7,809 | 7,909 | 7,032 | 7,817 | 7,693 | 11,637 | 15,373 | 15,560 | 15,345 | **15,565** |
| SplitNestSharedExpenseLedger | | | | | | | | | | | |
| Go | *warm-up F.* | 7,383 | 12,792 | 12,495 | 15,179 | 17,327 | 20,327 | 21,001 | 19,814 | 21,160 | **22,078** |
| Flask | 5,544 | 8,024 | 11,530 | 10,500 | *func* | 10,270 | 12,211 | 11,998 | 12,130 | *no settle* | **12,749** |
| Actix | 7,527 | *no settle* | 7,972 | 8,060 | 13,000 | 14,052 | 15,833 | **15,996** | 15,608 | 15,258 | 15,709 |
| TransitPulseDelayReporter | | | | | | | | | | | |
| Go | *recovery F.* | *spec* | *recovery F.* | 20,667 | 25,449 | *recovery F.* | 24,190 | **26,126** | 25,645 | 25,140 | 25,618 |
| Flask | *warm-up F.* | 1,746 | 1,705 | 6,130 | **15,426** | 13,642 | 14,787 | 13,590 | 13,323 | 13,893 | 12,539 |
| Actix | 1,947 | *recovery F.* | 5,049 | 7,503 | 7,569 | 7,424 | 11,236 | 6,705 | **11,263** | 11,114 | 10,828 |

Table D.13: Per-iteration sustained-goodput outcomes (req/s; **bold** marks each row's best) for **GPT-5.5**. Code refinement is green; deployment-specification refinement is blue. State labels: p. 126; failures: Table D.11.

| Fw | Base | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| ClickCount | | | | | | | | | | | |
| Go | 5,726 | 46,134 | *func* | 44,707 | 47,135 | 47,103 | 96,363 | 96,739 | 96,495 | 96,436 | **97,138** |
| Flask | *recovery F.* | 20,132 | *func* | 21,762 | 25,319 | 26,651 | 27,662 | 24,488 | **31,601** | 27,036 | 26,604 |
| Actix | *recovery F.* | *recovery F.* | *recovery F.* | *recovery F.* | 38,512 | *recovery F.* | 40,216 | 41,271 | 41,030 | 37,234 | **42,473** |
| Recipes | | | | | | | | | | | |
| Go | 30,815 | 67,060 | 60,140 | *no settle* | 66,770 | 81,244 | 88,182 | 91,632 | 91,486 | **92,120** | 90,838 |
| Flask | 9,800 | 9,811 | *func* | 18,414 | 23,294 | 30,034 | **32,558** | 29,500 | *func* | 29,583 | 28,159 |
| Actix | 9,164 | 35,634 | *recovery F.* | *recovery F.* | *recovery F.* | 24,520 | 46,958 | 62,366 | 80,083 | **84,795** | 81,652 |
| Petstore | | | | | | | | | | | |
| Go | *recovery F.* | 32,034 | 35,539 | 35,904 | 37,151 | 35,849 | *func* | 36,176 | 37,916 | 37,522 | **38,305** |
| Flask | *warm-up F.* | *func* | *recovery F.* | *warm-up F.* | *recovery F.* | *func* | 1,883 | 1,811 | **1,994** | 1,909 | 1,984 |
| Actix | *warm-up F.* | *recovery F.* | *recovery F.* | *recovery F.* | *func* | *recovery F.* | *build* | *no settle* | *recovery F.* | *recovery F.* | **27,294** |
| BranchWeave | | | | | | | | | | | |
| Go | 22,469 | 26,652 | 26,319 | 17,954 | 26,056 | 31,962 | 31,752 | 34,804 | 34,101 | 55,196 | **63,159** |
| Flask | 16,274 | 10,169 | *func* | *func* | *func* | 19,960 | 17,112 | 8,597 | 20,055 | **21,097** | 8,972 |
| Actix | 25,875 | *recovery F.* | *func* | 17,086 | 27,596 | **30,431** | 25,750 | 30,313 | 29,602 | *no settle* | *no settle* |
| ParcelPinLockerPickup | | | | | | | | | | | |
| Go | 27,660 | 30,182 | 57,487 | 63,604 | 71,148 | 73,609 | 73,284 | 92,701 | 93,138 | **93,522** | 92,876 |
| Flask | 18,375 | 19,382 | 22,390 | 26,262 | *no settle* | 27,361 | 28,491 | **32,586** | 32,310 | 28,493 | *no settle* |
| Actix | 14,435 | *func* | 20,819 | 21,530 | 21,561 | 30,868 | 30,822 | **30,978** | 30,775 | *build* | 30,746 |
| SplitNestSharedExpenseLedger | | | | | | | | | | | |
| Go | 17,296 | *recovery F.* | 14,091 | 17,049 | 16,704 | 13,240 | 17,709 | 17,827 | 18,231 | **18,351** | 15,320 |
| Flask | *warm-up F.* | 10,703 | 14,635 | 17,012 | 19,224 | 18,846 | 19,816 | 10,078 | 19,567 | 20,038 | **20,076** |
| Actix | 10,432 | 11,136 | 18,738 | 15,697 | 19,111 | 20,180 | 19,282 | **21,256** | 18,916 | 19,646 | 20,689 |
| TransitPulseDelayReporter | | | | | | | | | | | |
| Go | *warm-up F.* | 19,926 | 16,492 | 22,090 | *recovery F.* | 26,552 | 55,051 | 51,772 | 49,695 | 54,542 | **55,118** |
| Flask | 11,862 | 9,330 | 15,625 | 14,227 | 16,520 | 20,525 | **22,318** | 21,492 | 21,214 | 19,671 | 20,159 |
| Actix | 31,236 | 18,043 | 29,844 | 35,172 | 61,267 | 63,918 | 94,783 | 95,528 | 95,157 | 94,975 | **95,651** |

Table D.14: Per-iteration sustained-goodput outcomes (req/s; **bold** marks each row's best) for **GLM-5.2**. Code refinement is green; deployment-specification refinement is blue. State labels: p. 126; failures: Table D.11.

| Fw | Base | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| ClickCount | | | | | | | | | | | |
| Go | *warm-up F.* | 1,443 | 1,055 | 7,702 | 25,108 | 24,668 | *func* | *func* | 87,321 | 84,780 | **96,282** |
| Flask | 1,130 | 1,056 | 3,306 | 6,510 | *crash* | 7,007 | 6,834 | 16,282 | 23,420 | **23,595** | 20,394 |
| Actix | *warm-up F.* | *build* | *build* | 1,052 | 1,030 | *recovery F.* | **2,816** | 2,656 | 2,650 | 2,714 | 2,676 |
| Recipes | | | | | | | | | | | |
| Go | *warm-up F.* | *recovery F.* | 21,542 | *recovery F.* | 34,855 | 37,077 | 41,239 | **44,851** | 35,072 | 40,394 | 40,414 |
| Flask | *recovery F.* | 2,046 | *warm-up F.* | *recovery F.* | *recovery F.* | 3,931 | 4,549 | **5,421** | *recovery F.* | 4,243 | *recovery F.* |
| Actix | *recovery F.* | *no settle* | 5,676 | 5,833 | *no settle* | 6,862 | *warm-up F.* | 10,542 | *spec* | 12,799 | **15,029** |
| Petstore | | | | | | | | | | | |
| Go | *warm-up F.* | *parse* | *func* | *func* | 17,896 | 26,745 | **29,047** | 26,410 | 25,418 | 18,770 | 18,409 |
| Flask | 1,838 | 2,503 | 2,518 | 2,476 | 2,491 | 2,807 | *recovery F.* | 2,737 | 2,449 | *recovery F.* | **2,822** |
| Actix | *recovery F.* | *warm-up F.* | 11,489 | *warm-up F.* | *warm-up F.* | *unknown* | *unknown* | *build* | 10,963 | **12,217** | 10,603 |
| BranchWeave | | | | | | | | | | | |
| Go | *warm-up F.* | *func* | *func* | *func* | *warm-up F.* | *func* | *warm-up F.* | *func* | **10,842** | *recovery F.* | *recovery F.* |
| Flask | 7,832 | *no settle* | 11,468 | *func* | 11,552 | 11,850 | *no settle* | **13,486** | *no settle* | 13,306 | 9,303 |
| Actix | *warm-up F.* | *build* | *build* | *build* | *build* | *build* | 12,410 | 16,416 | 16,256 | 19,972 | **20,110** |
| ParcelPinLockerPickup | | | | | | | | | | | |
| Go | *warm-up F.* | 15,740 | *crash* | *crash* | **29,022** | 28,781 | 21,344 | 21,055 | 21,382 | 25,985 | 21,233 |
| Flask | 8,283 | 8,080 | 6,947 | **8,656** | 8,066 | *recovery F.* | *recovery F.* | *no settle* | *warm-up F.* | *recovery F.* | *recovery F.* |
| Actix | *warm-up F.* | *build* | 7,736 | 17,956 | *crash* | 25,239 | **26,337** | 24,304 | *recovery F.* | *build* | 15,292 |
| SplitNestSharedExpenseLedger | | | | | | | | | | | |
| Go | *warm-up F.* | 5,668 | 5,851 | 6,113 | *func* | 4,682 | *parse* | *warm-up F.* | *func* | 7,080 | **9,969** |
| Flask | 8,567 | 12,992 | *no settle* | *no settle* | 13,795 | **15,224** | *no settle* | 13,657 | *func* | *no settle* | *no settle* |
| Actix | 2,377 | 8,551 | *recovery F.* | 9,807 | 13,268 | 11,330 | 11,674 | 14,298 | **14,352** | *func* | *parse* |
| TransitPulseDelayReporter | | | | | | | | | | | |
| Go | *warm-up F.* | *func* | *recovery F.* | *recovery F.* | *recovery F.* | *recovery F.* | *recovery F.* | *recovery F.* | *recovery F.* | *recovery F.* | *recovery F.* |
| Flask | *recovery F.* | *recovery F.* | *spec* | *recovery F.* | 4,084 | 5,486 | 5,877 | 5,878 | 11,164 | 14,978 | **15,357** |
| Actix | 2,020 | *warm-up F.* | *build* | *warm-up F.* | *recovery F.* | 2,608 | *recovery F.* | *recovery F.* | *build* | 11,759 | **12,031** |

# Appendix E

# Use of Generative AI Tools

Generative-AI tools were used to assist during the development of the research software and the preparation of this thesis. This use was separate from the use of language models as systems under evaluation, which is described in Chapter 4 and Chapter 5.

The tools were accessed primarily through Cursor, OpenAI Codex, and Claude Code. Models made available through Cursor were used both through direct selection and Cursor's automatic model-routing option (*Auto*); Auto-routed requests are therefore not retrospectively assigned to a specific underlying model [5]. Models explicitly used during the project included OpenAI's GPT-5.5, GPT-5.6 Sol, GPT-5.6 Terra, and GPT-5.6 Luna [29, 31], as well as Anthropic's Claude Opus 4.8, Claude Opus 5, and Claude Sonnet 5 [2].

The assistance covered three main areas. First, *software engineering and data analysis*: implementation, refactoring, debugging, data analysis, and plotting. Second, *thesis writing*: outlining, drafting, restructuring, rephrasing, language editing, proofreading, and consistency review. Third, *figures and diagrams*: developing visual concepts and plotting or diagram code, as well as refining layouts, labels, captions, and the overall presentation.

Use of these tools was iterative and author-led. The author supplied the task, context, and constraints; reviewed outputs considered for inclusion; and revised, tested, corrected, or rejected suggestions as appropriate. AI output was not treated as scholarly evidence, and factual claims and references were checked against the cited sources. The author made the final methodological, analytical, interpretive, software-design, and editorial decisions and takes responsibility for the submitted software, analysis, text, figures, results, and conclusions.

Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

## Declaration of originality

The signed declaration of originality is a component of every written paper or thesis authored during the course of studies. In consultation with the supervisor, one of the following three options must be selected:

○ I confirm that I authored the work in question independently and in my own words, i.e. that no one helped me to author it. Suggestions from the supervisor regarding language and content are excepted. I used no generative artificial intelligence technologies[1].

◉ I confirm that I authored the work in question independently and in my own words, i.e. that no one helped me to author it. Suggestions from the supervisor regarding language and content are excepted. I used and cited generative artificial intelligence technologies[2].

○ I confirm that I authored the work in question independently and in my own words, i.e. that no one helped me to author it. Suggestions from the supervisor regarding language and content are excepted. I used generative artificial intelligence technologies[3]. In consultation with the supervisor, I did not cite them.

**Title of paper or thesis**:

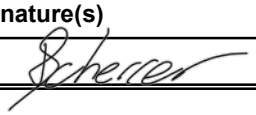
Benchmarking LLM-Driven Iterative Optimization of Deployed Backend Services

**Authored by**:
*If the work was compiled in a group, the names of all authors are required.*

| **Last name(s):** | **First name(s):** |
|---|---|
| Scherrer | David Jonas |
| | |
| | |
| | |

With my signature I confirm the following:
- I have adhered to the rules set out in the Citation Guide.
- I have documented all methods, data and processes truthfully and fully.
- I have mentioned all persons who were significant facilitators of the work.

I am aware that the work may be screened electronically for originality.

| **Place, date** | **Signature(s)** |
|---|---|
| Zurich, 24. August 2026 |  |
| | |
| | |
| | |

*If the work was compiled in a group, the names of all authors are required. Through their signatures they vouch jointly for the entire content of the written work.*

[1] E.g. ChatGPT, DALL E 2, Google Bard
[2] E.g. ChatGPT, DALL E 2, Google Bard
[3] E.g. ChatGPT, DALL E 2, Google Bard